

Queueick Frontend Code

Routes

```
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import { AuthProvider } from "../context/AuthContext";
import ProtectedRoute from "../components/auth/ProtectedRoute";
import AuthLayout from "../components/layout/AuthLayout";
import Home from "../pages/Home";
import Login from "../pages/Login";
import Dashboard from "../pages/Dashboard";
import AdminServicesPage from "../pages/admin/Services";
import AdminServiceWindowsPage from "../pages/admin/ServiceWindows";
import AdminStaffPage from "../pages/admin/Staff";
import AdminSettingsPage from "../pages/admin/Settings";
import { Toaster } from "sonner";
import TicketGeneration from "../pages/TicketGeneration";
import TicketStatus from "../pages/TicketStatus";
import Monitoring from "../pages/Monitoring";
import QueueManagement from "../pages/QueueManagement";
import StaffOnboarding from "../pages/StaffOnboarding";
import NotFound from "../pages/NotFound";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient();

const App = () => {
  return (
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <Router>
          <Routes>
            {/* Public routes */}
            <Route path="/" element={<Home />} />
            <Route path="/login" element={<Login />} />

            <Route
              path="/tickets/:ticketId/status"
              element={<TicketStatus />}
            />
          </Routes>
        </Router>
      </AuthProvider>
    </QueryClientProvider>
  );
};
```

```

<Route path="/monitoring" element={<Monitoring />} />

<Route
  path="/kiosk"
  element={
    <ProtectedRoute allowedRoles={["admin"]}>
      <TicketGeneration />
    </ProtectedRoute>
  }
/>

{/* Authenticated routes WITH sidebar layout (Admin only) */}
<Route
  element={
    <ProtectedRoute allowedRoles={["admin"]}>
      <AuthLayout />
    </ProtectedRoute>
  }
>
  <Route path="/dashboard" element={<Dashboard />} />
  <Route path="/admin/services" element={<AdminServicesPage
/>} />

  <Route
    path="/admin/services/:serviceId/windows"
    element={<AdminServiceWindowsPage />}
  />
  <Route path="/admin/staff" element={<AdminStaffPage />} />
  <Route path="/admin/settings" element={<AdminSettingsPage
/>} />

</Route>

{/* Authenticated routes WITHOUT sidebar layout (Staff only)
*/}

<Route
  path="/staff/onboarding"
  element={
    <ProtectedRoute allowedRoles={["admin", "staff"]}>
      <StaffOnboarding />
    </ProtectedRoute>
  }

```

```

        />
        <Route
          path="/queue-management"
          element={
            <ProtectedRoute allowedRoles={["admin", "staff"]} >
              <QueueManagement />
            </ProtectedRoute>
          }
        />

        <Route path="*" element={<NotFound />} />
      </Routes>
      <Toaster duration={3000} />
    </Router>
  </AuthProvider>

  { /* <ReactQueryDevtools initialIsOpen={false} /> */ }
</QueryClientProvider>
);
};

export default App;

```

Admin api calls

```

import api from "@api/client";
import type { User } from "../user/types";
import type {
  StaffFormValues,
  ListStaffResponse,
  GetSmsSettingsResponse,
  UpdateSmsSettingsPayload,
  GetAdminDashboardAnalyticsResponse,
  GetServiceAnalyticsResponse,
  GetSystemSettingsResponse,
  UpdateSystemSettingsPayload,
  CreateAdminPayload,

```

```

    CreateAdminResponse,
    ListAdminsResponse,
    UpdateAdminPayload,
    UpdateAdminResponse,
    DeleteAdminResponse,
    ResetStaffPasswordResponse,
    WindowAnalyticsResponse,
} from "./types";

export type AnalyticsQueryParams = {
  date?: string;
  start_date?: string;
  end_date?: string;
  service_id?: number;
};

export const adminApi = {
  listStaff: async (): Promise<ListStaffResponse> => {
    const response = await api.get("/auth/admin/staff/list/");
    return response.data;
  },

  getSystemSettings: async (): Promise<GetSystemSettingsResponse> => {
    const response = await api.get("/admin/system-settings/");
    return response.data;
  },

  updateSystemSettings: async (
    data: UpdateSystemSettingsPayload,
  ): Promise<{ success: boolean; message: string; settings: any }> => {
    const response = await api.patch("/admin/system-settings/update/",
data);
    return response.data;
  },

  createStaff: async (data: StaffFormValues): Promise<User> => {
    const response = await api.post("/auth/admin/staff/create/", data);
    return response.data;
  },

```

```

updateStaff: async (
  id: number,
  data: Partial<StaffFormValues>,
): Promise<User> => {
  const response = await api.patch(`/auth/admin/staff/${id}/update/`,
data);
  return response.data;
},

deleteStaff: async (id: number): Promise<void> => {
  await api.delete(`/auth/admin/staff/${id}/delete/`);
},

resetStaffPassword: async (
  userId: number,
  data: {
    new_password: string;
    confirm_password: string;
  },
): Promise<ResetStaffPasswordResponse> => {
  const response = await api.post(
    `/auth/admin/staff/${userId}/reset-password/`,
    data,
  );
  return response.data;
},

getSmsSettings: async (): Promise<GetSmsSettingsResponse> => {
  const response = await api.get("/admin/sms-settings/");
  return response.data;
},

updateGlobalSmsSettings: async (
  data: UpdateSmsSettingsPayload,
): Promise<{ success: boolean; message: string }> => {
  const response = await api.patch("/admin/sms-settings/global/", data);
  return response.data;
},

updateServiceSmsSettings: async (

```

```

    serviceId: number,
    data: UpdateSmsSettingsPayload,
  ): Promise<{ success: boolean; message: string }> => {
    const response = await api.patch(
      `/admin/sms-settings/service/${serviceId}/`,
      data,
    );
    return response.data;
  },

  resetServiceSmsSettings: async (
    serviceId: number,
  ): Promise<{ success: boolean; message: string }> => {
    const response = await api.delete(
      `/admin/sms-settings/service/${serviceId}/reset/`,
    );
    return response.data;
  },

  getDashboardAnalytics: async (
    params?: AnalyticsQueryParams,
  ): Promise<GetAdminDashboardAnalyticsResponse> => {
    const response = await api.get("/admin/analytics/", { params });
    return response.data;
  },

  getServiceAnalytics: async (
    serviceId: number,
  ): Promise<GetServiceAnalyticsResponse> => {
    const response = await
api.get(`/admin/analytics/service/${serviceId}/`);
    return response.data;
  },

  getWindowAnalytics: async (
    serviceId: number,
    params?: {
      date?: string;
      start_date?: string;
      end_date?: string;
    }
  ) => {
    const response = await api.get(
      `/admin/analytics/service/${serviceId}/window/`,
      params,
    );
    return response.data;
  },

```

```

    },
    ): Promise<WindowAnalyticsResponse> => {
        const response = await
api.get(`/admin/analytics/window/${serviceId}/`, {
        params,
    });
    return response.data;
},

exportTicketsCSV: async (params?: {
    date?: string;
    start_date?: string;
    end_date?: string;
    service_id?: number;
}): Promise<Blob> => {
    const response = await api.get("/admin/analytics/export/csv/", {
        params,
        responseType: "blob",
    });
    return response.data;
},

exportWindowPerformanceCSV: async (
    serviceId: number,
    params?: {
        date?: string;
        start_date?: string;
        end_date?: string;
    },
): Promise<Blob> => {
    const response = await api.get(
        `/admin/analytics/export/window/${serviceId}/csv/`,
        {
            params,
            responseType: "blob",
        },
    );
    return response.data;
},

```

```

listAdmins: async (): Promise<ListAdminsResponse> => {
  const response = await api.get("/auth/admin/list/");
  return response.data;
},

createAdmin: async (
  data: CreateAdminPayload,
): Promise<CreateAdminResponse> => {
  const response = await api.post("/auth/admin/create/", data);
  return response.data;
},

updateAdmin: async (
  userId: number,
  data: UpdateAdminPayload,
): Promise<UpdateAdminResponse> => {
  const response = await api.patch(`/auth/admin/${userId}/update/`,
data);
  return response.data;
},

deleteAdmin: async (userId: number): Promise<DeleteAdminResponse> => {
  const response = await api.delete(`/auth/admin/${userId}/delete/`);
  return response.data;
},
};

```

Services api calls

```

import api from "@api/client";
import type {
  ServiceMutationPayload,
  ServiceListResponse,
  ServiceDetailResponse,
  ServiceStatsResponse,
  ServiceWindow,
} from "../types";

type ServiceWindowMutationResponse = {

```

```

    success?: boolean;
    message?: string;
    window?: ServiceWindow;
    service_active?: boolean;
};

export const serviceApi = {
  // — Service Management —————

  getAllServices: async (): Promise<ServiceListResponse> => {
    const response = await api.get("/services/");
    return response.data;
  },

  createService: async (
    data: ServiceMutationPayload,
  ): Promise<ServiceDetailResponse> => {
    const response = await api.post("/services/create/", data);
    return response.data;
  },

  updateService: async (
    id: number,
    data: Partial<ServiceMutationPayload>,
  ): Promise<ServiceDetailResponse> => {
    const response = await api.patch(`/services/${id}/update/`, data);
    return response.data;
  },

  deleteService: async (id: number): Promise<void> => {
    await api.delete(`/services/${id}/delete/`);
  },

  getServiceStats: async (id: number): Promise<ServiceStatsResponse> => {
    const response = await api.get(`/services/${id}/stats/`);
    return response.data;
  },

  // — Window Management —————

```

```

getServiceWindows: async (
  serviceId: number,
): Promise<{ windows: ServiceWindow[] }> => {
  const response = await api.get(`/services/${serviceId}/windows/`);
  return response.data;
},

createServiceWindow: async (
  serviceId: number,
  data: Record<string, unknown>,
): Promise<ServiceWindow> => {
  const response = await api.post(
    `/services/${serviceId}/windows/create/`,
    data,
  );
  const payload = response.data as
    | ServiceWindow
    | ServiceWindowMutationResponse;
  return (
    (payload as ServiceWindowMutationResponse).window ||
    (payload as ServiceWindow)
  );
},

updateWindow: async (
  windowId: number,
  data: Record<string, unknown>,
): Promise<ServiceWindow> => {
  const response = await api.patch(`/windows/${windowId}/update/`,
data);
  const payload = response.data as
    | ServiceWindow
    | ServiceWindowMutationResponse;
  return (
    (payload as ServiceWindowMutationResponse).window ||
    (payload as ServiceWindow)
  );
},

deleteWindow: async (windowId: number): Promise<void> => {

```

```

        await api.delete(`/windows/${windowId}/delete/`);
    },
};

```

Staff api calls

```

import api from "@api/client";
import type { StaffDashboardResponse, CallNextResponse } from "../types";

export type WindowSelectionStatus = "inactive" | "active" | "maintenance";

export interface WindowSelectionWindow {
    id: number;
    name: string;
    number: number;
    status: WindowSelectionStatus;
    is_in_use: boolean;
    is_available: boolean;
    claimed_by: string | null;
    current_staff_name?: string | null;
    current_staff?: number | null;
}

type RawWindowSelectionWindow = Partial<WindowSelectionWindow> & {
    window_number?: number;
    claimed_by?: string | number | null;
};

interface WindowSelectionApiResponse {
    window?: Partial<WindowSelectionWindow>;
    data?: {
        window?: Partial<WindowSelectionWindow>;
    };
}

const normalizeWindowSelectionWindow = (
    raw?: RawWindowSelectionWindow,
): WindowSelectionWindow => ({
    id: Number(raw?.id ?? 0),

```

```

name: String(raw?.name ?? ""),
number: Number(raw?.number ?? raw?.window_number ?? 0),
status: (raw?.status as WindowSelectionStatus) ?? "inactive",
is_in_use: typeof raw?.is_in_use === "boolean" ? raw.is_in_use : false,
is_available:
  typeof raw?.is_available === "boolean"
    ? raw.is_available
    : (raw?.status as WindowSelectionStatus) === "active" &&
      !Boolean(raw?.is_in_use),
claimed_by:
  raw?.claimed_by === null || raw?.claimed_by === undefined
    ? null
    : String(raw.claimed_by),
current_staff_name:
  typeof raw?.current_staff_name === "string" ||
  raw?.current_staff_name === null
    ? raw.current_staff_name
    : undefined,
current_staff:
  typeof raw?.current_staff === "number" || raw?.current_staff === null
    ? raw.current_staff
    : undefined,
});

export const staffApi = {
  getDashboard: async (): Promise<StaffDashboardResponse> => {
    const response = await api.get("/staff/dashboard/");
    return response.data;
  },

  callNext: async (data?: {
    window_id?: number;
  }): Promise<CallNextResponse> => {
    const response = await api.post("/staff/call-next/", data);
    return response.data;
  },

  callSpecific: async (data: {
    ticket_number: string;
    window_id: number;
  }) => {
    const response = await api.post("/staff/call-specific/", data);
    return response.data;
  },
};

```

```

}): Promise<{ success: boolean; message: string }> => {
  const response = await api.post("/staff/call-specific/", data);
  return response.data;
},

completeServing: async (
  ticketId: string,
): Promise<{ success: boolean; message: string }> => {
  const response = await
api.post(`/staff/tickets/${ticketId}/complete/`);
  return response.data;
},

skipTicket: async (
  ticketId: string,
  data?: { reason?: string },
): Promise<{ success: boolean; message: string }> => {
  const response = await api.post(`/staff/tickets/${ticketId}/skip/`,
data);
  return response.data;
},

recallTicket: async (
  ticketId: string,
): Promise<{ success: boolean; message: string }> => {
  const response = await api.post(`/staff/tickets/${ticketId}/recall/`);
  return response.data;
},

removeTicket: async (
  ticketId: string,
  data?: { reason: string },
): Promise<{ success: boolean; message: string }> => {
  const response = await api.post(`/staff/tickets/${ticketId}/remove/`,
data);
  return response.data;
},

claimWindow: async (data: {
  window_id: number;

```

```

    staff_account_id: number;
  }): Promise<WindowSelectionWindow> => {
    const response = await api.post("/sessions/claim", data);
    const payload = response.data as
      | WindowSelectionApiResponse
      | Partial<WindowSelectionWindow>;

    const windowData =
      (payload as WindowSelectionApiResponse).window ||
      (payload as WindowSelectionApiResponse).data?.window ||
      (payload as Partial<WindowSelectionWindow>);

    return normalizeWindowSelectionWindow(windowData);
  },

  releaseWindow: async (data: { window_id: number }): Promise<void> => {
    await api.post("/sessions/release", data);
  },
};

export default staffApi;

```

Window release functions

```

export const CLAIMED_WINDOW_ID_STORAGE_KEY = "claimed_window_id";

const getReleaseEndpointUrl = () => {
  const fallbackPath = "/api/sessions/release";
  const apiBase =
    (import.meta.env.VITE_API_URL as string | undefined) || "/api/";

  try {
    const resolvedBase = new URL(apiBase, window.location.origin);
    const normalizedPath = resolvedBase.pathname.endsWith("/")
      ? resolvedBase.pathname
      : `${resolvedBase.pathname}/`;

    return `${resolvedBase.origin}${normalizedPath}sessions/release`;
  } catch {

```

```

        return new URL(fallbackPath, window.location.origin).toString();
    }
};

const readCookie = (name: string): string | null => {
    if (typeof document === "undefined") {
        return null;
    }

    const escaped = name.replace(/[\.*+?^${}()|[\]\$\\"/g, "\\$&");
    const match = document.cookie.match(new RegExp(`(?:^|;
)${escaped}=(?:^|;|\\s|$)`));
    return match ? decodeURIComponent(match[1]) : null;
};

const toNumberOrNull = (value: string | null): number | null => {
    if (!value) {
        return null;
    }

    const parsed = Number(value);
    return Number.isFinite(parsed) && parsed > 0 ? parsed : null;
};

export const readClaimedWindowIdFromSession = (): number | null => {
    if (typeof window === "undefined") {
        return null;
    }

    return toNumberOrNull(
        window.sessionStorage.getItem(CLAIMED_WINDOW_ID_STORAGE_KEY),
    );
};

export const writeClaimedWindowIdToSession = (windowId: number | null) => {
    if (typeof window === "undefined") {
        return;
    }

```

```

if (windowId == null) {
    window.sessionStorage.removeItem(CLAIMED_WINDOW_ID_STORAGE_KEY);
    return;
}

window.sessionStorage.setItem(
    CLAIMED_WINDOW_ID_STORAGE_KEY,
    String(windowId),
);
};

export const sendReleaseBeacon = (windowId: number): boolean => {
    if (
        typeof window === "undefined" ||
        typeof navigator === "undefined" ||
        typeof navigator.sendBeacon !== "function"
    ) {
        return false;
    }

    const payload = JSON.stringify({ window_id: windowId });
    const body = new Blob([payload], { type: "application/json" });
    return navigator.sendBeacon(getReleaseEndpointUrl(), body);
};

export const sendReleaseKeepalive = (windowId: number): boolean => {
    if (typeof window === "undefined") {
        return false;
    }

    const headers: Record<string, string> = {
        "Content-Type": "application/json",
    };

    const csrf = readCookie("csrftoken");
    if (csrf) {
        headers["X-CSRFToken"] = csrf;
    }
}

```

```
// keepalive requests are best-effort during unload and cannot be
awaited.

fetch(getReleaseEndpointUrl(), {
  method: "POST",
  keepalive: true,
  credentials: "include",
  headers,
  body: JSON.stringify({ window_id: windowId }),
}).catch(() => {
  // ignore - unload path is fire-and-forget
});

return true;
};
```

Shared Hooks

```
import { useEffect, useState } from "react";

export const useCurrentTime = (intervalMs = 1000) => {
  const [currentTime, setCurrentTime] = useState(new Date());

  useEffect(() => {
    const timer = setInterval(() => setCurrentTime(new Date()),
intervalMs);
    return () => clearInterval(timer);
  }, [intervalMs]);

  return currentTime;
};
```

Websocket custom hook

```
import { useEffect, useRef } from "react";

type UseQueueWebSocketOptions = {
  enabled?: boolean;
  path?: string;
  onOpen?: (socket: WebSocket) => void;
  onMessage?: (event: MessageEvent) => void;
};

const DEFAULT_WS_PATH = "/ws/dashboard/";
const MAX_RECONNECT_ATTEMPTS = 8;

const buildWsUrl = (path: string) => {
  const explicitWsUrl = import.meta.env.VITE_WS_URL as string | undefined;
  const backendOrigin = import.meta.env.VITE_BACKEND_ORIGIN as
    | string
    | undefined;

  if (explicitWsUrl) {
    const base = explicitWsUrl.endsWith("/")
```

```

        ? explicitWsUrl.slice(0, -1)
        : explicitWsUrl;
    const cleanPath = path.startsWith("/") ? path : `/${path}`;
    return new URL(`${base}${cleanPath}`).toString();
}

if (backendOrigin) {
    const wsBase = backendOrigin.replace(/^http/i, "ws").replace(/\/$/, "");
    const cleanPath = path.startsWith("/") ? path : `/${path}`;
    return new URL(`${wsBase}${cleanPath}`).toString();
}

const protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
const cleanPath = path.startsWith("/") ? path : `/${path}`;
return new
URL(`${protocol}://${window.location.host}${cleanPath}`).toString();
};

export const useQueueWebSocket = ({
    enabled = true,
    path = DEFAULT_WS_PATH,
    onOpen,
    onMessage,
}: UseQueueWebSocketOptions) => {
    const reconnectTimerRef = useRef<number | null>(null);
    const reconnectAttemptsRef = useRef(0);
    const websocketRef = useRef<WebSocket | null>(null);

    // ✅ Store callbacks in refs so they never trigger re-runs
    const onOpenRef = useRef(onOpen);
    const onMessageRef = useRef(onMessage);

    // ✅ Keep refs updated on every render without re-running effect
    useEffect(() => {
        onOpenRef.current = onOpen;
        onMessageRef.current = onMessage;
    });

    useEffect(() => {

```

```
if (!enabled) return;

let isUnmounted = false;

const clearReconnectTimer = () => {
  if (reconnectTimerRef.current !== null) {
    window.clearTimeout(reconnectTimerRef.current);
    reconnectTimerRef.current = null;
  }
};

const connect = () => {
  clearReconnectTimer();

  const wsUrl = buildWsUrl(path);
  const ws = new WebSocket(wsUrl);
  websocketRef.current = ws;

  ws.onopen = () => {
    reconnectAttemptsRef.current = 0;
    onOpenRef.current?.(ws); // ✅ use ref, not the prop directly
  };

  ws.onmessage = (event) => {
    onMessageRef.current?.(event); // ✅ use ref
  };

  ws.onclose = () => {
    if (isUnmounted) return;

    reconnectAttemptsRef.current += 1;
    if (reconnectAttemptsRef.current > MAX_RECONNECT_ATTEMPTS) {
      return;
    }

    const baseDelay = Math.min(
      1000 * 2 ** (reconnectAttemptsRef.current - 1),
      30000,
    );
    const jitter = Math.floor(Math.random() * 500);
```

```

    const delay = baseDelay + jitter;
    reconnectTimerRef.current = window.setTimeout(connect, delay);
  };

  ws.onerror = () => {
    ws.close();
  };
};

connect();

return () => {
  isUnmounted = true;
  clearReconnectTimer();
  websocketRef.current?.close();
  websocketRef.current = null;
};
}, [enabled, path]); // ✅ only re-run if enabled or path changes
};

```

Staff hooks

```

import { useMutation, useQuery, useQueryClient } from
"@tanstack/react-query";
import { useCallback, useEffect, useRef, useState } from "react";
import { toast } from "sonner";
import { useNavigate } from "react-router-dom";

import { staffApi } from "@features/staff/api";
import type { WindowSelectionWindow } from "@features/staff/api";
import type { StaffDashboardResponse } from "@features/staff/types";
import type { Ticket } from "@features/tickets/types";
import type { ServiceWindow } from "@features/services/types";
import { useStaffWindowStore } from "@store/staffWindowStore";
import { useAuthStore } from "@store/authStore";

```

```

import { useQueueWebSocket } from
"@/features/shared/hooks/useQueueWebSocket";
import { useWindowSelectionSocket } from "./useWindowSelectionSocket";
import { useWindowSelectionStore } from "@/store/windowSelectionStore";
import { useWindowOwnershipCleanup } from "./useWindowOwnershipCleanup";

export interface StaffQueueManagementState {
  dashboard: StaffDashboardResponse["dashboard"] | undefined;
  isLoading: boolean;
  isError: boolean;
  allTickets: Ticket[];
  userName: string | undefined;
  handleLogout: () => Promise<void>;
  selectedWindow: ServiceWindow | null;
  windowStatus: ServiceWindow["status"] | undefined;
  disableCallNext: boolean;
  activeTicketLabel: string | undefined;
  activeTicketId: string | undefined;
  isQueueEmpty: boolean;
  isLastTicket: boolean;
  pending: {
    leaveWindow: boolean;
    callNext: boolean;
    complete: boolean;
    skip: boolean;
    startServing: boolean;
    remove: boolean;
    recall: boolean;
  };
  handleLeaveWindow: () => Promise<void>;
  handleMainActionClick: () => void;
  handleStartServing: (ticketNumber: string) => void;
  handleComplete: (ticketId: string) => void;
  handleSkip: (ticketId: string) => void;
  handleRemove: (ticketId: string) => void;
  handleRecall: (ticketId: string) => void;
}

const getRequestErrorMessage = (error: any, fallback: string): string => {
  const backendMessage =

```

```

    error?.response?.data?.message ||
    error?.response?.data?.detail ||
    error?.response?.data?.error;

    if (backendMessage) {
        return String(backendMessage);
    }

    if (error?.request && !error?.response) {
        return "Request sent but no response came back. Check Django server,
Vite proxy, or route.";
    }

    const status = error?.response?.status;

    if (!status) {
        return "Unexpected error. Please try again.";
    }

    if (status === 400) return "Invalid request. Please refresh and try
again.";
    if (status === 403)
        return "You are not allowed to perform this action right now.";
    if (status === 404)
        return "Endpoint or ticket not found. Check the route and ticket id.";
    if (status === 409)
        return "Action conflict. Please refresh and try again.";
    if (status >= 500)
        return "Server error while processing your request. Check Django
logs.";

    return fallback;
};

export const useStaffQueueManagement = (): StaffQueueManagementState => {
    const navigate = useNavigate();
    const queryClient = useQueryClient();
    const {
        selectedWindow,
        serviceInfo,

```

```

    setSelectedWindow,
    clearSelectedWindow,
  } = useStaffWindowStore();
  const { user, logout } = useAuthStore();
  const { releaseLoading, setSelectedWindowId } =
useWindowSelectionStore();
  const { releaseWindowExplicit } = useWindowOwnershipCleanup();
  const staffServiceId = serviceInfo?.id || user?.assigned_service?.id;
  const [staffSocketAuthFailed, setStaffSocketAuthFailed] =
useState(false);

  const selectedWindowRef = useRef(selectedWindow);
  useEffect(() => {
    selectedWindowRef.current = selectedWindow;
  });

  const handleStaffRealtimeUpdate = useCallback(
    (event: MessageEvent) => {
      if (!event.data) {
        queryClient.invalidateQueries({ queryKey: ["staff-dashboard"] });
        return;
      }

      try {
        const payload = JSON.parse(String(event.data));

        if (
          payload?.type === "error" &&
          typeof payload?.message === "string" &&
          payload.message.toLowerCase().includes("authentication failed")
        ) {
          localStorage.removeItem("ws_access_token");
          setStaffSocketAuthFailed(true);
          queryClient.invalidateQueries({ queryKey: ["staff-dashboard"]
});
          return;
        }

        if (payload?.type === "staff_update" && payload?.data) {
          const data = payload.data;

```

```

const normalizedDashboard = {
  service:
    typeof data.service === "string"
      ? data.service
      : data.service?.name || serviceInfo?.name || "",
  waiting_count: data.waiting_count ?? 0,
  serving_count: data.serving_count ?? data.serving_list?.length
  ?? 0,

  next_ticket: data.next_ticket ?? null,
  currently_serving:
    data.currently_serving || data.serving_list || [],
  waiting_list: data.waiting_list || [],
  skipped_list: data.skipped_list || [],
  windows: data.windows || [],
};

queryClient.setQueryData(["staff-dashboard"], {
  success: true,
  dashboard: normalizedDashboard,
});
return;
}
} catch {
  // fall through
}

queryClient.invalidateQueries({ queryKey: ["staff-dashboard"] });
},
[queryClient, serviceInfo?.name],
);

const handleStaffSocketOpen = useCallback(
  (socket: WebSocket) => {
    if (staffSocketAuthFailed) {
      socket.close();
      return;
    }

    const cookies = document.cookie || "";

```

```

const token = localStorage.getItem("ws_access_token") || undefined;

if (!cookies.includes("access_token=") && !token) {
  socket.send(JSON.stringify({ type: "refresh" }));
  return;
}

socket.send(
  JSON.stringify({
    type: "authenticate",
    ...(cookies.includes("access_token=") ? { cookies } : {}),
    ...(token ? { token } : {}),
  }),
);

socket.send(JSON.stringify({ type: "refresh" }));
},
[staffSocketAuthFailed],
);

useQueueWebSocket({
  enabled: !!staffServiceId && !staffSocketAuthFailed,
  path: staffServiceId ? `/ws/staff/${staffServiceId}/` :
"/ws/staff/0/",
  onOpen: handleStaffSocketOpen,
  onMessage: handleStaffRealtimeUpdate,
});

useQueueWebSocket({
  path: "/ws/dashboard/",
  onMessage: () => {
    queryClient.invalidateQueries({ queryKey: ["staff-dashboard"] });
  },
});

const handleWindowUpdate = useCallback(
  (windows: WindowSelectionWindow[]) => {
    const currentWindow = selectedWindowRef.current;
    if (!currentWindow) return;

```

```

const matchedWindow = windows.find(
  (windowItem) =>
    Number(windowItem.id) === Number(currentWindow.id) ||
    Number(windowItem.number) ===
Number(currentWindow.window_number),
);

if (!matchedWindow) return;

if (matchedWindow.status !== "active") {
  clearSelectedWindow();
  setSelectedWindowId(null);
  toast.error(
    "Your claimed window is not usable right now. Please select a
window again.",
  );
  navigate("/staff/onboarding", { replace: true });
  return;
}

const claimedByAnotherStaff =
  matchedWindow.is_in_use &&
  typeof matchedWindow.current_staff === "number" &&
  typeof user?.id === "number" &&
  matchedWindow.current_staff !== user.id;

if (claimedByAnotherStaff) {
  clearSelectedWindow();
  setSelectedWindowId(null);
  toast.error(
    "This window is now claimed by another staff member. Please
choose another window.",
  );
  navigate("/staff/onboarding", { replace: true });
  return;
}

setSelectedWindow({
  ...currentWindow,
  status: matchedWindow.status,

```

```

        is_available: matchedWindow.is_available,
        is_in_use:
          typeof matchedWindow.is_in_use === "boolean"
            ? matchedWindow.is_in_use
            : currentWindow.is_in_use,
        claimed_by: matchedWindow.claimed_by ?? currentWindow.claimed_by,
        current_staff_name:
          matchedWindow.current_staff_name ??
currentWindow.current_staff_name,
        current_staff:
          typeof matchedWindow.current_staff === "number"
            ? matchedWindow.current_staff
            : currentWindow.current_staff,
      });
    },
    // eslint-disable-next-line react-hooks/exhaustive-deps
    [
      clearSelectedWindow,
      navigate,
      setSelectedWindow,
      setSelectedWindowId,
      user?.id,
    ],
  );

  useWindowSelectionSocket({
    serviceId: staffServiceId,
    enabled: !!staffServiceId,
    onWindowsUpdate: handleWindowUpdate,
  });

  useEffect(() => {
    if (selectedWindow?.id) {
      setSelectedWindowId(selectedWindow.id);
    }
  }, [selectedWindow?.id, setSelectedWindowId]);

  const { data, isLoading, isError } = useQuery<StaffDashboardResponse>({
    queryKey: ["staff-dashboard"],
    queryFn: staffApi.getDashboard,
  });

```

```

    });

    const dashboard = data?.dashboard;

    const invalidateDashboard = () =>
      queryClient.invalidateQueries({ queryKey: ["staff-dashboard"] });

    const callNextMutation = useMutation({
      mutationFn: () => staffApi.callNext({ window_id: selectedWindow?.id
    })),
      onSuccess: async (res) => {
        toast.success(res.message || "Next ticket called!");
        await invalidateDashboard();
      },
      onError: (err) => {
        toast.error(getRequestErrorMessage(err, "Failed to call next
ticket."));
      },
    });

    const startServingMutation = useMutation({
      mutationFn: ({
        ticket_number,
        windowId,
      }: {
        ticket_number: string;
        windowId: number;
      }) => staffApi.callSpecific({ ticket_number, window_id: windowId }),
      onSuccess: async (res) => {
        toast.success(res.message || "Now serving ticket.");
        await invalidateDashboard();
      },
      onError: (err) => {
        toast.error(getRequestErrorMessage(err, "Failed to start
serving."));
      },
    });

    const completeMutation = useMutation({
      mutationFn: staffApi.completeServing,

```

```

    onSuccess: async (res) => {
      toast.success(res.message || "Ticket marked as served.");
      await invalidateDashboard();
    },
    onError: (err) => {
      toast.error(getRequestErrorMessage(err, "Failed to complete
serving."));
    },
  });

const skipMutation = useMutation({
  mutationFn: (ticketId: string) =>
    staffApi.skipTicket(ticketId, {
      reason: "Customer not present",
    }),
  onSuccess: async (res) => {
    toast.success(res.message || "Ticket skipped.");
    await invalidateDashboard();
  },
  onError: (err) => {
    toast.error(getRequestErrorMessage(err, "Failed to skip ticket."));
  },
});

const removeMutation = useMutation({
  mutationFn: (ticketId: string) =>
    staffApi.removeTicket(ticketId, { reason: "Removed by staff" }),
  onSuccess: async (res) => {
    toast.success(res.message || "Ticket removed.");
    await invalidateDashboard();
  },
  onError: (err) => {
    toast.error(getRequestErrorMessage(err, "Failed to remove
ticket."));
  },
});

const recallMutation = useMutation({
  mutationFn: staffApi.recallTicket,
  onSuccess: async (res) => {

```

```

        toast.success(res.message || "Ticket recalled to queue.");
        await invalidateDashboard();
    },
    onError: (err) => {
        toast.error(getRequestErrorMessage(err, "Failed to recall
ticket."));
    },
});

const allTickets: Ticket[] = [
    ...(dashboard?.currently_serving || []),
    ...(dashboard?.waiting_list || []),
    ...(dashboard?.skipped_list || []),
];

const selectedWindowId = Number(selectedWindow?.id);
const selectedWindowNumber = Number(selectedWindow?.window_number);

const activeWindowTicketObj = dashboard?.currently_serving?.find(
    (ticket) =>
        Number(ticket.assigned_window) === selectedWindowId ||
        Number(ticket.assigned_window_info?.id) === selectedWindowId ||
        Number(ticket.assigned_window_info?.window_number) ===
            selectedWindowNumber,
);

const activeWindowFromList = dashboard?.windows?.find(
    (windowItem) =>
        windowItem.id === selectedWindowId ||
        windowItem.number === selectedWindowNumber,
);

const windowServing = activeWindowFromList?.currently_serving;
const windowServingDisplayNumber =
    typeof windowServing === "string"
        ? windowServing
        : windowServing?.display_number;

const windowServingTicketId =
    typeof windowServing === "object" && windowServing

```

```

    ? windowServing.ticket_id
    : null;

const activeTicketLabel =
  activeWindowTicketObj?.display_number || windowServingDisplayNumber;

const activeTicketId =
  activeWindowTicketObj?.ticket_id ||
  windowServingTicketId ||
  dashboard?.currently_serving?.find(
    (ticket) => ticket.display_number === windowServingDisplayNumber,
 )?.ticket_id;

const isServing = !!activeTicketLabel;
const hasWaitingTickets = (dashboard?.waiting_count ?? 0) > 0 ||
!!dashboard?.next_ticket;
const isEmptyQueue =
  (dashboard?.waiting_count ?? 0) === 0 && !dashboard?.next_ticket;
const isLastTicket = isServing && !hasWaitingTickets;

const windowStatus: ServiceWindow["status"] | undefined =
  selectedWindow?.status;
const disableCallNext = !selectedWindow || windowStatus !== "active";

const handleStartServing = (ticketNumber: string) => {
  if (disableCallNext) {
    toast.error("Cannot serve tickets while window or service is
inactive.");
    return;
  }

  if (!selectedWindow?.id) {
    toast.error("Please select a window first");
    return;
  }

  startServingMutation.mutate({
    ticket_number: ticketNumber,
    windowId: selectedWindow.id,
  });

```

```

};

// Smart main action - handles both scenarios
const handleMainActionClick = () => {
  if (disableCallNext) {
    toast.error("Please claim an active window before serving tickets.");
    return;
  }

  // Case 1: There's a serving ticket AND waiting tickets → call next (auto-completes current)
  if (isServing && hasWaitingTickets) {
    callNextMutation.mutate();
    return;
  }

  // Case 2: There's a serving ticket but NO waiting tickets → just complete it
  if (isServing && !hasWaitingTickets) {
    if (activeTicketId) {
      completeMutation.mutate(activeTicketId);
    } else {
      toast.error("No active ticket found to complete.");
    }
    return;
  }

  // Case 3: No serving ticket, just call next
  if (!isServing) {
    callNextMutation.mutate();
    return;
  }

  // Fallback
  toast.error("Unable to perform action. Please check queue status.");
};

const releaseCurrentWindow = async (): Promise<boolean> => {
  if (!selectedWindow?.id) return true;

```

```

    try {
      const result = await releaseWindowExplicit();
      if (result.skipped) return true;

      if (!result.released) {
        throw result.error;
      }

      toast.success("Window released.");
      return true;
    } catch (error: any) {
      toast.error(getRequestErrorMessage(error, "Failed to release
window."));
      const shouldRetry = window.confirm(
        "Failed to release window due to network/server issue. Retry
now?",
      );

      if (shouldRetry) {
        return releaseCurrentWindow();
      }

      return false;
    }
  };

  const handleLeaveWindow = async () => {
    const released = await releaseCurrentWindow();
    if (!released) return;
    navigate("/staff/onboarding");
  };

  const handleLogout = async () => {
    const released = await releaseCurrentWindow();
    if (!released) return;
    await logout();
  };

  return {

```

```

dashboard,
isLoading,
isError,
allTickets,
userName: user?.username,
handleLogout,
selectedWindow,
windowStatus,
disableCallNext,
activeTicketLabel,
activeTicketId,
isQueueEmpty,
isLastTicket,
pending: {
  leaveWindow: releaseLoading,
  callNext: callNextMutation.isPending,
  complete: completeMutation.isPending,
  skip: skipMutation.isPending,
  startServing: startServingMutation.isPending,
  remove: removeMutation.isPending,
  recall: recallMutation.isPending,
},
handleLeaveWindow,
handleMainActionClick,
handleStartServing,
handleComplete: (ticketId: string) =>
completeMutation.mutate(ticketId),
  handleSkip: (ticketId: string) => skipMutation.mutate(ticketId),
  handleRemove: (ticketId: string) => removeMutation.mutate(ticketId),
  handleRecall: (ticketId: string) => recallMutation.mutate(ticketId),
};
};

```

```
import { useCallback, useEffect, useRef } from "react";
import { staffApi } from "@features/staff/api";
import {
  readClaimedWindowIdFromSession,
  sendReleaseBeacon,
  sendReleaseKeepalive,
  writeClaimedWindowIdToSession,
} from "@features/staff/windowRelease";
import { useStaffWindowStore } from "@store/staffWindowStore";
import { useWindowSelectionStore } from "@store/windowSelectionStore";

interface ExplicitReleaseResult {
  released: boolean;
  skipped: boolean;
  error?: unknown;
}

export const useWindowOwnershipCleanup = () => {
  const claimedWindowId = useStaffWindowStore((state) =>
state.claimedWindowId);
  const clearSelectedWindow = useStaffWindowStore(
    (state) => state.clearSelectedWindow,
  );

  const setSelectedWindowId = useWindowSelectionStore(
    (state) => state.setSelectedWindowId,
  );
  const setReleaseLoading = useWindowSelectionStore(
    (state) => state.setReleaseLoading,
  );
  const setWindowSelectionError = useWindowSelectionStore(
    (state) => state.setError,
  );

  const latestWindowIdRef = useRef<number | null>(claimedWindowId ??
null);
  const releasedRef = useRef(false);
  const explicitReleaseInFlightRef = useRef(false);
  const unloadStartedRef = useRef(false);
```

```

useEffect(() => {
  const fromSession = readClaimedWindowIdFromSession();
  const effectiveId = claimedWindowId ?? fromSession;
  latestWindowIdRef.current = effectiveId;

  if (effectiveId) {
    releasedRef.current = false;
    writeClaimedWindowIdToSession(effectiveId);
    return;
  }

  writeClaimedWindowIdToSession(null);
}, [claimedWindowId]);

const clearLocalWindowOwnership = useCallback(() => {
  clearSelectedWindow();
  setSelectedWindowId(null);
  setWindowSelectionError(null);
  writeClaimedWindowIdToSession(null);
}, [clearSelectedWindow, setSelectedWindowId, setWindowSelectionError]);

const releaseWindowExplicit =
  useCallback(async (): Promise<ExplicitReleaseResult> => {
    const windowId =
      latestWindowIdRef.current ?? readClaimedWindowIdFromSession();

    if (!windowId) {
      return { released: false, skipped: true };
    }

    if (releasedRef.current || explicitReleaseInFlightRef.current) {
      return { released: true, skipped: false };
    }

    explicitReleaseInFlightRef.current = true;
    setReleaseLoading(true);

    try {
      await staffApi.releaseWindow({ window_id: windowId });
      releasedRef.current = true;
    }
  }, [latestWindowIdRef, releasedRef, explicitReleaseInFlightRef, staffApi]);

```

```

        latestWindowIdRef.current = null;
        clearLocalWindowOwnership();
        return { released: true, skipped: false };
    } catch (error) {
        return { released: false, skipped: false, error };
    } finally {
        explicitReleaseInFlightRef.current = false;
        setReleaseLoading(false);
    }
}, [clearLocalWindowOwnership, setReleaseLoading]);

const releaseWindowOnUnload = useCallback(() => {
    if (unloadStartedRef.current) {
        return;
    }

    unloadStartedRef.current = true;

    const windowId =
        latestWindowIdRef.current ?? readClaimedWindowIdFromSession();

    if (!windowId) {
        return;
    }

    if (releasedRef.current) {
        return;
    }

    const beaconQueued = sendReleaseBeacon(windowId);
    const keepaliveQueued = sendReleaseKeepalive(windowId);

    if (beaconQueued || keepaliveQueued) {
        releasedRef.current = true;
        latestWindowIdRef.current = null;
        writeClaimedWindowIdToSession(null);
    }
}, []);

useEffect(() => {

```

```
const handleBeforeUnload = () => {
  releaseWindowOnUnload();
};

const handlePageHide = () => {
  releaseWindowOnUnload();
};

window.addEventListener("beforeunload", handleBeforeUnload);
window.addEventListener("pagehide", handlePageHide);

return () => {
  window.removeEventListener("beforeunload", handleBeforeUnload);
  window.removeEventListener("pagehide", handlePageHide);
};
}, [releaseWindowOnUnload]);

return {
  releaseWindowExplicit,
};
};
```

```

import { useEffect, useRef } from "react";
import type {
  WindowSelectionStatus,
  WindowSelectionWindow,
} from "@features/staff/api";
import { useWindowSelectionStore } from "@store/windowSelectionStore";

interface WindowStatusUpdateMessage {
  type: "window_status_update";
  data?: {
    service_id?: number;
    windows?: Array<Partial<WindowSelectionWindow>>;
  };
}

interface UseWindowSelectionSocketOptions {
  serviceId?: number;
  enabled?: boolean;
  onWindowsUpdate?: (windows: WindowSelectionWindow[]) => void;
}

const MAX_RECONNECT_ATTEMPTS = 8;
const MAX_RECONNECT_DELAY_MS = 15_000;

const toWsUrl = (serviceId: number) => {
  const path = `/ws/service/${serviceId}/windows/`;
  const explicitWsUrl = import.meta.env.VITE_WS_URL as string | undefined;
  const backendOrigin = import.meta.env.VITE_BACKEND_ORIGIN as
    | string
    | undefined;

  if (explicitWsUrl) {
    const base = explicitWsUrl.endsWith("/")
      ? explicitWsUrl.slice(0, -1)
      : explicitWsUrl;
    return new URL(`${base}${path}`).toString();
  }

  if (backendOrigin) {

```

```

    const wsBase = backendOrigin.replace(/^http/i, "ws").replace(/\$/,
    "");
    return new URL(`${wsBase}${path}`).toString();
  }

  const protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
  return new
URL(`${protocol}://${window.location.host}${path}`).toString();
};

const normalizeWindow = (
  windowData: Partial<WindowSelectionWindow>,
): WindowSelectionWindow => {
  const status = (windowData.status as WindowSelectionStatus) ??
"inactive";
  const isInUse =
    typeof windowData.is_in_use === "boolean" ? windowData.is_in_use :
false;
  return {
    id: Number(windowData.id ?? 0),
    name: String(windowData.name ?? ""),
    number: Number(
      windowData.number ??
      (
        windowData as Partial<WindowSelectionWindow> & {
          window_number?: number;
        }
      ).window_number ??
      0,
    ),
    status,
    is_in_use: isInUse,
    is_available:
      typeof windowData.is_available === "boolean"
        ? windowData.is_available
        : status === "active" && !isInUse,
    claimed_by: windowData.claimed_by ?? null,
    current_staff_name:
      typeof windowData.current_staff_name === "string" ||
      windowData.current_staff_name === null

```

```

        ? windowData.current_staff_name
        : undefined,
    current_staff:
        typeof windowData.current_staff === "number" ||
        windowData.current_staff === null
        ? windowData.current_staff
        : undefined,
    };
};

export const useWindowSelectionSocket = ({
    serviceId,
    enabled = true,
    onWindowsUpdate,
}: UseWindowSelectionSocketOptions) => {
    const wsRef = useRef<WebSocket | null>(null);
    const reconnectTimerRef = useRef<number | null>(null);
    const reconnectAttemptsRef = useRef(0);
    // true while we are intentionally closing (unmount / serviceId change).
    // onClose should NOT schedule reconnect when this is true.
    const manualCloseRef = useRef(false);

    // Keep the callback in a ref so it never appears in effect deps.
    // The socket effect only reads from this ref, so changing the callback
    // does not tear down and recreate the WebSocket.
    const onWindowsUpdateRef = useRef(onWindowsUpdate);
    useEffect(() => {
        onWindowsUpdateRef.current = onWindowsUpdate;
    }); // intentionally no dep array - update on every render

    // Zustand setters are stable (created once by the store), but we read
    // them once here and use them inside the effect via closure.
    const { setWindows, setSocketState, setError } =
useWindowSelectionStore();

    useEffect(() => {
        if (!enabled || !serviceId) {
            return;
        }

```

```
const wsUrl = toWsUrl(serviceId);

// Guard: do not create a second socket for the same service while one
// is already OPEN or in the process of connecting.
const existing = wsRef.current;
if (
  existing &&
  (existing.readyState === WebSocket.OPEN ||
    existing.readyState === WebSocket.CONNECTING)
) {
  return;
}

// Mark this effect run as intentional (not a manual close yet).
manualCloseRef.current = false;
reconnectAttemptsRef.current = 0;

const clearReconnectTimer = () => {
  if (reconnectTimerRef.current !== null) {
    window.clearTimeout(reconnectTimerRef.current);
    reconnectTimerRef.current = null;
  }
};

const closeSocketManually = () => {
  if (manualCloseRef.current) {
    return;
  }

  manualCloseRef.current = true;
  clearReconnectTimer();

  if (wsRef.current) {
    wsRef.current.close();
    wsRef.current = null;
  }
};

const scheduleReconnect = () => {
  if (manualCloseRef.current) {
```

```

        // Intentional close - do not reconnect.
        return;
    }

    clearReconnectTimer();
    reconnectAttemptsRef.current += 1;

    if (reconnectAttemptsRef.current > MAX_RECONNECT_ATTEMPTS) {
        setSocketState(false, false);
        setError("Window connection offline. Please refresh to
reconnect.");
        return;
    }

    const delay =
        Math.min(
            1000 * 2 ** (reconnectAttemptsRef.current - 1),
            MAX_RECONNECT_DELAY_MS,
        ) + Math.floor(Math.random() * 500);

    setSocketState(false, true);
    reconnectTimerRef.current = window.setTimeout(connect, delay);
};

const connect = () => {
    if (manualCloseRef.current) {
        return;
    }

    // Another guard inside the reconnect path.
    const cur = wsRef.current;
    if (
        cur &&
        (cur.readyState === WebSocket.OPEN ||
            cur.readyState === WebSocket.CONNECTING)
    ) {
        return;
    }

    const ws = new WebSocket(wsUrl);

```

```

wsRef.current = ws;
setSocketState(false, reconnectAttemptsRef.current > 0);

ws.onopen = () => {
  reconnectAttemptsRef.current = 0;
  setSocketState(true, false);
  setError(null);
  ws.send(JSON.stringify({ type: "refresh" }));
};

ws.onmessage = (event) => {
  try {
    const payload = JSON.parse(
      String(event.data),
    ) as WindowStatusUpdateMessage;
    if (payload?.type !== "window_status_update") {
      return;
    }

    const windows = (payload.data?.windows ??
[]).map(normalizeWindow);
    setWindows(windows);
    // Read from ref - calling the latest version of the callback
    // without putting it in effect deps.
    onWindowsUpdateRef.current?.(windows);
  } catch {
    setError("Received malformed realtime update for windows.");
  }
};

ws.onerror = () => {
  // onerror is always followed by onclose, so close and let onclose
  // decide whether to reconnect.
  ws.close();
};

ws.onclose = () => {
  setSocketState(false, !manualCloseRef.current);
  scheduleReconnect();
};

```

```
};

connect();

const handleBeforeUnload = () => {
  closeSocketManually();
};

const handlePageHide = () => {
  closeSocketManually();
};

window.addEventListener("beforeunload", handleBeforeUnload);
window.addEventListener("pagehide", handlePageHide);

// Cleanup: runs on unmount OR when serviceId/enabled changes.
// Setting manualCloseRef prevents onClose from scheduling a
reconnect.
return () => {
  window.removeEventListener("beforeunload", handleBeforeUnload);
  window.removeEventListener("pagehide", handlePageHide);
  closeSocketManually();
  setSocketState(false, false);
};

// Effect ONLY re-runs when the service we are watching changes.
// Callbacks and Zustand setters must NOT be here - they change every
// render and would tear down the socket on each state update.
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [enabled, serviceId]);
};
```

Ticket hooks

```
import { useEffect, useRef } from "react";
import type { DashboardServiceDetail, DashboardServiceWindow } from
"../types";
import { formatWindowLabel } from "@lib/windowLabel";

export const useTicketAnnouncer = (
  services: DashboardServiceDetail[] | undefined,
) => {
  const previousTicketsRef = useRef<Record<string, string>>({});
  const hasInitializedRef = useRef(false);

  useEffect(() => {
    if (!services) return;

    const currentTickets: Record<string, string> = {};
    const newCalls: { ticketNumber: string; windowLabel: string }[] = [];

    services.forEach((service) => {
      if (service.currently_serving) {
        service.currently_serving.forEach((window: DashboardServiceWindow)
=> {
          if (!window.ticket_number || window.ticket_number === "---")
return;

          const windowLabel = formatWindowLabel(window, "");

          const key = `${service.id}-${windowLabel}`;

          currentTickets[key] = window.ticket_number;

          if (
            hasInitializedRef.current &&
            previousTicketsRef.current[key] !== window.ticket_number
          ) {
            newCalls.push({
              ticketNumber: window.ticket_number,
              windowLabel,
            });
          }
        });
      }
    });
  });
}
```

```

    }
  });
}
});

previousTicketsRef.current = currentTickets;
if (!hasInitializedRef.current) {
  hasInitializedRef.current = true;
  return;
}

if (newCalls.length > 0) {
  newCalls.forEach(({ ticketNumber, windowLabel }) => {
    // Format ticket number with spaces for better TTS pronunciation
    (e.g. "A 0 0 1")
    const spokenTicket = ticketNumber.split("").join(" ");
    const spokenWindow = windowLabel || "the window";

    const utterance = new SpeechSynthesisUtterance(
      `Ticket ${spokenTicket} Please proceed to ${spokenWindow}.`,
    );
    utterance.rate = 1.2; // Slightly slower for clarity
    utterance.pitch = 1;

    // Optional: Pick a good English voice if available
    const voices = window.speechSynthesis.getVoices();
    const englishVoice = voices.find(
      (v) => v.lang.includes("en-") && v.localService,
    );
    if (englishVoice) {
      utterance.voice = englishVoice;
    }

    window.speechSynthesis.speak(utterance);
  });
}, [services]);
};

```

```

import { useCallback, useMemo, useState } from "react";
import { useMutation, useQuery, useQueryClient } from
"@tanstack/react-query";
import { toast } from "sonner";

import { ticketApi } from "@features/tickets/api";
import type { TicketStatusResponse } from "@features/tickets/types";
import { useQueueWebSocket } from
"@features/shared/hooks/useQueueWebSocket";
import { formatWindowLabel } from "@lib/windowLabel";
import { getErrorMessage } from "@lib/errorUtils";

interface UseTicketStatusPageResult {
  ticketStatus: TicketStatusResponse | undefined;
  isLoading: boolean;
  isError: boolean;
  isNotifyDialogOpen: boolean;
  phoneNumber: string;
  phoneError: string | null;
  requestedNotifyNumber: string;
  isSubmittingSmsOptIn: boolean;
  currentlyServingDisplay: string | undefined;
  currentlyServingWindow: string;
  ticketDateDisplay: string;
  shouldShowNotifyMe: boolean;
  setIsNotifyDialogOpen: (open: boolean) => void;
  setPhoneError: (value: string | null) => void;
  handlePhoneChange: (value: string) => void;
  handleNotifySubmit: () => Promise<void>;
}

const validatePhPhoneNumber = (value: string) => {
  const normalized = value.replace(/\s()-/g, "");
  const isValid = /^(09\d{9}|\+?639\d{9})$/ .test(normalized);

  return {
    isValid,
    normalized,
  };
};

```

```

const getFallbackDateLabel = (createdAt?: string) => {
  if (!createdAt) return "Date unavailable";

  const parsedDate = new Date(createdAt);
  if (Number.isNaN(parsedDate.getTime())) return "Date unavailable";

  return parsedDate.toLocaleDateString([], {
    year: "numeric",
    month: "short",
    day: "numeric",
  });
};

export const useTicketStatusPage = (
  ticketId: string | undefined,
): UseTicketStatusPageResult => {
  const queryClient = useQueryClient();
  const [isNotifyDialogOpen, setIsNotifyDialogOpen] = useState(false);
  const [phoneNumber, setPhoneNumber] = useState("");
  const [phoneError, setPhoneError] = useState<string | null>(null);
  const [requestedNotifyNumber, setRequestedNotifyNumber] = useState("");

  const { mutateAsync: submitSmsOptIn, isPending: isSubmittingSmsOptIn } =
    useMutation({
      mutationFn: ({ id, phone }: { id: string; phone: string }) =>
        ticketApi.smsOptIn(id, phone),
    });

  const handleRealtimeUpdate = useCallback(
    (event: MessageEvent) => {
      if (!ticketId) return;

      if (!event.data) {
        queryClient.invalidateQueries({ queryKey: ["ticketStatus",
ticketId] });
        return;
      }

      try {

```

```

const payload = JSON.parse(String(event.data));
if (payload?.type === "ticket_update" && payload?.data) {
  const data = payload.data;
  const payloadTicketId = data?.ticket_id || data?.ticketId;

  if (!payloadTicketId || payloadTicketId === ticketId) {
    queryClient.setQueryData(
      ["ticketStatus", ticketId],
      (previous: any) => ({
        success: true,
        ticket: {
          ...(previous?.ticket || {}),
          ...data,
        },
        queue_info: {
          ...(previous?.queue_info || {}),
          ...(data?.queue_info || {}),
        },
      }),
    );
    return;
  }
}

const payloadTicketId = payload?.ticket_id || payload?.ticketId;
if (!payloadTicketId || payloadTicketId === ticketId) {
  queryClient.invalidateQueries({
    queryKey: ["ticketStatus", ticketId],
  });
}
} catch {
  queryClient.invalidateQueries({ queryKey: ["ticketStatus",
ticketId] });
}
},
[queryClient, ticketId],
);

useQueueWebSocket({
  enabled: !!ticketId,

```

```

    path: ticketId ? `/ws/ticket/${ticketId}/` : "/ws/ticket/",
    onMessage: handleRealtimeUpdate,
  });

  useQueueWebSocket({
    enabled: !!ticketId,
    path: "/ws/dashboard/",
    onMessage: () => {
      if (!ticketId) return;
      queryClient.invalidateQueries({ queryKey: ["ticketStatus", ticketId]
    });
  },
  });

  const {
    data: ticketStatus,
    isLoading,
    error,
  } = useQuery({
    queryKey: ["ticketStatus", ticketId],
    queryFn: () => ticketApi.getTicketStatus(ticketId!),
    enabled: !!ticketId,
  });

  const ticket = ticketStatus?.ticket;
  const queueInfo = ticketStatus?.queue_info;
  const currentlyServing = queueInfo?.currently_serving;

  const currentlyServingDisplay =
    typeof currentlyServing === "string"
      ? currentlyServing
      : currentlyServing?.display_number;

  const currentlyServingWindow = ticket
    ? formatWindowLabel(
      {
        ...ticket.assigned_window_info,
        window_number:
          ticket.assigned_window_info?.window_number ??
          ticket.assigned_window,
      }
    )
    : null;

```

```

    },
    "",
  )
  : "";

const ticketDateDisplay =
  ticket?.ticket_date || getFallbackDateLabel(ticket?.created_at);

const shouldShowNotifyMe = useMemo(() => {
  const parsedQueuePosition = Number(queueInfo?.position);
  return Number.isFinite(parsedQueuePosition) && parsedQueuePosition >
5;
}, [queueInfo?.position]);

const handlePhoneChange = (value: string) => {
  setPhoneNumber(value);
  if (phoneError) {
    setPhoneError(null);
  }
};

const handleNotifySubmit = async () => {
  if (!ticketId) {
    toast.error("Unable to save notification", {
      description: "Ticket ID is missing.",
    });
    return;
  }

  const { isValid, normalized } = validatePhPhoneNumber(phoneNumber);

  if (!isValid) {
    setPhoneError(
      "Enter a valid PH number starting with 09 or 639 (e.g. 09171234567
or 639171234567).",
    );
    return;
  }

  setPhoneError(null);

```

```
try {
  await submitSmsOptIn({ id: ticketId, phone: normalized });

  setRequestedNotifyNumber(normalized);
  setIsNotifyDialogOpen(false);
  setPhoneNumber("");

  toast.success("Notification request saved", {
    description:
      "You will be notified once when your ticket is near to be
called.",
  });
} catch (submitError: any) {
  toast.error("Failed to save notification request", {
    description: getErrorMessage(
      submitError,
      "Please try again in a moment.",
    ),
  });
}
};

return {
  ticketStatus,
  isLoading,
  isError: !!error,
  isNotifyDialogOpen,
  phoneNumber,
  phoneError,
  requestedNotifyNumber,
  isSubmittingSmsOptIn,
  currentlyServingDisplay,
  currentlyServingWindow,
  ticketDateDisplay,
  shouldShowNotifyMe,
  setIsNotifyDialogOpen,
  setPhoneError,
  handlePhoneChange,
  handleNotifySubmit,
```

```
};  
};
```

Pages and Components

Dashboard

```
import { useMemo, useState } from "react";
import { useQuery } from "@tanstack/react-query";
import { Link } from "react-router-dom";
import {
  MonitorUp,
  TabletSmartphone,
  CalendarDays,
  Filter,
  Download,
} from "lucide-react";
import { toast } from "sonner";

import { adminApi } from "@features/admin/api";
import DashboardPeakHoursChart from
"@features/admin/components/dashboard/DashboardPeakHoursChart";
import DashboardRecentActivityTable from
"@features/admin/components/dashboard/DashboardRecentActivityTable";
import DashboardServicesTable from
"@features/admin/components/dashboard/DashboardServicesTable";
import DashboardSummaryCards from
"@features/admin/components/dashboard/DashboardSummaryCards";
import { formatTimestamp } from
"@features/admin/components/dashboard/dashboardFormatters";
import { getErrorMessage } from "@lib/errorUtils";

import { Button } from "@components/ui/button";
import {
  Card,
  CardContent,
  CardDescription,
  CardHeader,
  CardTitle,
} from "@components/ui/card";

type AnalyticsPreset = "today" | "yesterday" | "7days" | "30days";
```

```
const Dashboard = () => {
  const [preset, setPreset] = useState<AnalyticsPreset>("today");
  const [isExporting, setIsExporting] = useState(false);

  const queryParams = useMemo(() => {
    const today = new Date();

    const formatLocalDate = (date: Date) => {
      const year = date.getFullYear();
      const month = String(date.getMonth() + 1).padStart(2, "0");
      const day = String(date.getDate()).padStart(2, "0");
      return `${year}-${month}-${day}`;
    };

    if (preset === "today") {
      return undefined;
    }

    if (preset === "yesterday") {
      const yesterday = new Date(today);
      yesterday.setDate(today.getDate() - 1);
      return { date: formatLocalDate(yesterday) };
    }

    if (preset === "7days") {
      const start = new Date(today);
      start.setDate(today.getDate() - 6);

      return {
        start_date: formatLocalDate(start),
        end_date: formatLocalDate(today),
      };
    }

    if (preset === "30days") {
      const start = new Date(today);
      start.setDate(today.getDate() - 29);

      return {
        start_date: formatLocalDate(start),
```

```

        end_date: formatLocalDate(today),
    };
}

return undefined;
}, [preset]);

const filterLabel = useMemo(() => {
    switch (preset) {
        case "today":
            return "Today's analytics overview";
        case "yesterday":
            return "Yesterday's analytics overview";
        case "7days":
            return "Last 7 days analytics overview";
        case "30days":
            return "Last 30 days analytics overview";
        default:
            return "Analytics overview";
    }
}, [preset]);

const peakHoursDescription = useMemo(() => {
    switch (preset) {
        case "today":
            return "Tickets issued by time slot for today.";
        case "yesterday":
            return "Tickets issued by time slot for yesterday.";
        case "7days":
            return "Tickets issued by time slot for the last 7 days.";
        case "30days":
            return "Tickets issued by time slot for the last 30 days.";
        default:
            return "Tickets issued by time slot.";
    }
}, [preset]);

const recentActivityDescription = useMemo(() => {
    switch (preset) {
        case "today":

```

```

        return "Most recently served tickets and their waiting times for
today.";
    case "yesterday":
        return "Most recently served tickets and their waiting times for
yesterday.";
    case "7days":
        return "Most recently served tickets within the last 7 days.";
    case "30days":
        return "Most recently served tickets within the last 30 days.";
    default:
        return "Most recently served tickets and their waiting times.";
    }
}, [preset]);

const { data, isLoading, isFetching, error, refetch } = useQuery({
  queryKey: ["admin-dashboard-analytics", preset, queryParams],
  queryFn: () => adminApi.getDashboardAnalytics(queryParams),
  refetchInterval: 60_000,
});

const handleExportCSV = async () => {
  setIsExporting(true);
  try {
    const blob = await adminApi.exportTicketsCSV(queryParams);
    const url = window.URL.createObjectURL(blob);
    const link = document.createElement("a");
    link.href = url;
    link.setAttribute(
      "download",
      `queueick_export_${new Date().toISOString().split("T")[0]}.csv`,
    );
    document.body.appendChild(link);
    link.click();
    link.remove();
    window.URL.revokeObjectURL(url);
    toast.success("Data exported successfully!");
  } catch {
    toast.error("Failed to export data");
  } finally {
    setIsExporting(false);
  }
}

```

```

    }
};

if (isLoading) {
    return (
        <div className="min-h-screen
bg-[radial-gradient(circle_at_top,#f3ffe9_0%,#f8fafc_45%,#f8fafc_100%)]">
            <div className="container mx-auto flex min-h-[70vh] items-center
justify-center p-8">
                <div className="flex flex-col items-center gap-4 rounded-3xl
border border-[#2c990f]/10 bg-white/80 px-10 py-8 shadow-xl
backdrop-blur">
                    <div className="h-12 w-12 animate-spin rounded-full border-4
border-[#2c990f]/15 border-b-[#2c990f]" />
                    <div className="space-y-1 text-center">
                        <p className="text-sm font-medium text-brand-green">
                            Loading dashboard analytics...
                        </p>
                        <p className="text-xs text-muted-foreground">
                            Preparing the latest queue insights
                        </p>
                    </div>
                </div>
            </div>
        </div>
    );
}

if (error || !data?.analytics) {
    return (
        <div className="min-h-screen
bg-[radial-gradient(circle_at_top,#f3ffe9_0%,#f8fafc_45%,#f8fafc_100%)]">
            <div className="mx-auto max-w-4xl p-8">
                <Card className="overflow-hidden border-0 shadow-xl">
                    <CardHeader className="bg-linear-to-r from-[#2c990f]/10
via-white to-[#dcb34]/10">
                        <CardTitle className="text-xl text-brand-green">
                            Dashboard Analytics
                        </CardTitle>
                        <CardDescription>

```

```

        {getErrorMessage(error, "Failed to load dashboard
analytics")}}
      </CardDescription>
    </CardHeader>
    <CardContent className="space-y-4 pt-6">
      <div className="flex flex-wrap gap-3">
        <Button
          onClick={() => refetch()}
          className="bg-brand-green text-white hover:bg-[#24820c]"
        >
          Retry
        </Button>
        <Button
          variant="outline"
          onClick={() => setPreset("today")}
          className="border-[#2c990f]/20 text-brand-green"
        >
          Reset to Today
        </Button>
      </div>
    </CardContent>
  </Card>
</div>
</div>
);
}

const { analytics } = data;

return (
  <div className="min-h-screen
bg-[radial-gradient(circle_at_top,#f3ffe9_0%,#f8fafc_45%,#f8fafc_100%)]">
    <div className="mx-auto max-w-7xl space-y-6 p-4">
      <section className="relative overflow-hidden rounded-[2rem] border
border-white/60 bg-linear-to-r from-[#2c990f]/18 via-white to-[#dcbc34]/18
px-6 py-7 shadow-[0_20px_60px_rgba(0,0,0,0.08)] backdrop-blur md:px-8">
        <div className="absolute inset-0
bg-[linear-gradient(135deg,rgba(255,255,255,0.30)_0%,rgba(255,255,255,0.05
)_100%)]" />

```

```

        <div className="absolute -right-10 -top-10 h-40 w-40
rounded-full bg-brand-green/15 blur-3xl" />
        <div className="absolute -bottom-8 left-10 h-28 w-28
rounded-full bg-[#dcbc34]/25 blur-3xl" />
        <div className="absolute right-1/3 top-0 h-full w-px
bg-linear-to-b from-transparent via-white/60 to-transparent" />

        <div className="relative flex flex-col gap-6 lg:flex-row
lg:items-end lg:justify-between">
          <div className="space-y-3">
            <div>
              <h1 className="text-3xl font-black tracking-tight
text-brand-green md:text-4xl">
                Dashboard Overview
              </h1>
              <p className="mt-1 text-sm text-slate-600 md:text-base">
                {filterLabel}
              </p>
            </div>

            <div className="inline-flex items-center gap-2 rounded-full
bg-white/70 px-3 py-1 text-xs text-slate-600 shadow-sm">
              <span className="h-2 w-2 rounded-full bg-brand-green" />
              Last updated: {formatTimestamp(analytics.timestamp)}
              {isFetching ? " (refreshing...)" : ""}
            </div>
          </div>

          <div className="flex flex-wrap items-center gap-3">
            <Button
              asChild
              variant="outline"
              className="h-10 rounded-xl border-[#2c990f]/15 bg-white/85
px-4 text-brand-green shadow-sm backdrop-blur hover:bg-[#f7fff2]"
            >
              <Link to="/kiosk">
                <TabletSmartphone className="mr-2 h-4 w-4" />
                Open Kiosk
              </Link>
            </Button>

```

```

        <Button
            asChild
            className="h-10 rounded-xl bg-brand-green px-4 text-white
shadow-sm hover:bg-[#24820c]"
        >
            <Link to="/monitoring">
                <MonitorUp className="mr-2 h-4 w-4" />
                Open Monitoring
            </Link>
        </Button>

        <Button
            onClick={handleExportCSV}
            disabled={isExporting}
            className="h-10 rounded-xl bg-brand-green px-4 text-white
shadow-sm hover:bg-[#24820c]"
        >
            <Download className="mr-2 h-4 w-4" />
            {isExporting ? "Exporting..." : "Export CSV"}
        </Button>
    </div>
</div>
</section>

<Card className="overflow-hidden rounded-[1.5rem] border-0
bg-white shadow-[0_12px_30px_rgba(0,0,0,0.06)] ring-1 ring-[#2c990f]/8">
    <div className="h-1 w-full bg-linear-to-r from-[#2c990f]
via-[#9ed640] to-[#dcbc34]" />
    <CardContent className="flex flex-col gap-4 p-4 md:flex-row
md:items-center md:justify-between">
        <div className="space-y-1">
            <div className="flex items-center gap-2 text-sm
font-semibold text-brand-green">
                <Filter className="h-4 w-4 text-[#2c990f]" />
                Quick Filters
            </div>
            <p className="text-xs text-muted-foreground">
                Switch between recent analytics periods instantly.
            </p>

```

```

</div>

<div className="flex flex-wrap items-center gap-2">
  <Button
    size="sm"
    variant={preset === "today" ? "default" : "outline"}
    onClick={() => setPreset("today")}
    className={
      preset === "today"
        ? "rounded-full bg-brand-green text-white
hover:bg-[#24820c]"
        : "rounded-full border-[#2c990f]/20 text-brand-green
hover:bg-[#f7fff2]"
      }
    >
    Today
  </Button>

  <Button
    size="sm"
    variant={preset === "yesterday" ? "default" : "outline"}
    onClick={() => setPreset("yesterday")}
    className={
      preset === "yesterday"
        ? "rounded-full bg-brand-green text-white
hover:bg-[#24820c]"
        : "rounded-full border-[#2c990f]/20 text-brand-green
hover:bg-[#f7fff2]"
      }
    >
    Yesterday
  </Button>

  <Button
    size="sm"
    variant={preset === "7days" ? "default" : "outline"}
    onClick={() => setPreset("7days")}
    className={
      preset === "7days"

```

```

        ? "rounded-full bg-[#dcbc34] text-[#3f3200]
hover:bg-[#c9ad2f]"
        : "rounded-full border-[#dcbc34]/30 text-[#8c6f00]
hover:bg-[#fff9e8]"
    }
  >
    Last 7 Days
  </Button>

  <Button
    size="sm"
    variant={preset === "30days" ? "default" : "outline"}
    onClick={() => setPreset("30days")}
    className={
      preset === "30days"
        ? "rounded-full bg-[#dcbc34] text-[#3f3200]
hover:bg-[#c9ad2f]"
        : "rounded-full border-[#dcbc34]/30 text-[#8c6f00]
hover:bg-[#fff9e8]"
    }
  >
    Last 30 Days
  </Button>

  <Button
    size="sm"
    variant="ghost"
    onClick={() => refetch()}
    className="rounded-full text-slate-600 hover:bg-slate-100"
  >
    <CalendarDays className="mr-2 h-4 w-4" />
    Refresh
  </Button>
</div>
</CardContent>
</Card>

<DashboardSummaryCards summary={analytics.summary} />

<section className="grid grid-cols-1 gap-6 xl:grid-cols-3">

```

```

        <Card className="overflow-hidden rounded-[1.5rem] border-0
bg-white shadow-[0_16px_35px_rgba(0,0,0,0.06)] ring-1 ring-[#2c990f]/8
xl:col-span-2">
            <CardHeader className="border-b border-[#2c990f]/10
bg-linear-to-r from-[#2c990f]/8 via-white to-white pb-4">
                <CardTitle className="text-lg text-brand-green">
                    Service Performance
                </CardTitle>
                <CardDescription>
                    Tickets, throughput, queue pressure, and waiting estimates
by
                    service.
                </CardDescription>
            </CardHeader>
            <CardContent className="pt-6">
                <DashboardServicesTable
                    services={analytics.services}
                    queryParams={queryParams}
                />
            </CardContent>
        </Card>

        <Card className="overflow-hidden rounded-[1.5rem] border-0
bg-white shadow-[0_16px_35px_rgba(0,0,0,0.06)]">
            <CardHeader className="border-b border-[#2c990f]/10
bg-linear-to-r from-[#2c990f]/8 via-white to-white pb-4">
                <CardTitle className="flex items-center gap-2 text-lg
text-brand-green">
                    Peak Hours
                </CardTitle>
                <CardDescription>{peakHoursDescription}</CardDescription>
            </CardHeader>
            <CardContent className="pt-6">
                <DashboardPeakHoursChart peakHours={analytics.peak_hours} />
            </CardContent>
        </Card>
    </section>

    <section>

```

```

        <Card className="overflow-hidden rounded-[1.5rem] border-0
bg-white shadow-[0_16px_35px_rgba(0,0,0,0.06)] ring-1 ring-[#dcbc34]/12">
          <CardHeader className="border-b border-[#dcbc34]/15
bg-linear-to-r from-white via-white to-[#fff8e1] pb-4">
            <CardTitle className="text-lg text-brand-green">
              Recent Activity
            </CardTitle>

<CardDescription>{recentActivityDescription}</CardDescription>
          </CardHeader>
          <CardContent className="pt-6 pb-2">
            <DashboardRecentActivityTable
              recentActivity={analytics.recent_activity}
            />
          </CardContent>
        </Card>
      </section>
    </div>
  </div>
);
};

export default Dashboard;

```

Home

```
import { Link } from "react-router-dom";
import { Ticket, Monitor, LogIn, LogOut } from "lucide-react";
import { SplitLayout } from "@components/layout/SplitLayout";
import { useAuthStore } from "@store/authStore";

const Home = () => {
  const { isAuthenticated, logout } = useAuthStore();

  return (
    <SplitLayout>
      <Link to="/kiosk" className="block focus:outline-none">
        <button className="w-full flex items-center p-6 md:p-8 bg-white border border-gray-200 rounded-3xl shadow-[0_4px_20px_rgba(0,0,0,0.04)] hover:shadow-[0_8px_30px_rgba(0,0,0,0.08)] transition-all duration-200 active:scale-[0.98] text-left group">
          <div className="shrink-0 w-16 h-16 md:w-20 md:h-20 rounded-2xl bg-[linear-gradient(135deg,rgba(44,153,15,0.14)_0%,rgba(255,255,255,1)_100%)] shadow-sm ring-1 ring-[#2c990f]/10 flex items-center justify-center mr-6 transition-colors">
            <Ticket
              className="w-8 h-8 md:w-10 md:h-10 text-brand-green"
              strokeWidth={2.5}
            />
          </div>
          <div className="flex-1">
            <h3 className="text-xl md:text-2xl font-black text-gray-900 mb-1 uppercase tracking-wide group-hover:text-brand-green transition-colors">
              GET A TICKET
            </h3>
            <p className="text-sm md:text-base text-gray-500">
              Tap here to generate your queue ticket.
            </p>
          </div>
        </button>
      </Link>

      <Link to="/monitoring" className="block focus:outline-none">
```

```

        <button className="w-full flex items-center p-6 md:p-8 bg-white
border border-gray-200 rounded-3xl shadow-[0_4px_20px_rgba(0,0,0,0.04)]
hover:shadow-[0_8px_30px_rgba(0,0,0,0.08)] transition-all duration-200
active:scale-[0.98] text-left group">
            <div className="shrink-0 w-16 h-16 md:w-20 md:h-20 rounded-2xl
bg-[linear-gradient(135deg,rgba(44,153,15,0.14)_0%,rgba(255,255,255,1)_100
%)] shadow-sm ring-1 ring-[#2c990f]/10 flex items-center justify-center
mr-6 transition-colors">
                <Monitor
                    className="w-8 h-8 md:w-10 md:h-10 text-brand-green"
                    strokeWidth={2.5}
                />
            </div>
            <div className="flex-1">
                <h3 className="text-xl md:text-2xl font-black text-gray-900
mb-1 uppercase tracking-wide group-hover:text-brand-green
transition-colors">
                    VIEW MONITOR
                </h3>
                <p className="text-sm md:text-base text-gray-500">
                    Check current queue status and updates.
                </p>
            </div>
        </button>
    </Link>

    {isAuthenticated && (
        <Link to="/dashboard" className="block focus:outline-none">
            <button className="w-full flex items-center p-6 md:p-8 bg-white
border border-gray-200 rounded-3xl shadow-[0_4px_20px_rgba(0,0,0,0.04)]
hover:shadow-[0_8px_30px_rgba(0,0,0,0.08)] transition-all duration-200
active:scale-[0.98] text-left group">
                <div className="shrink-0 w-16 h-16 md:w-20 md:h-20 rounded-2xl
bg-brand-green/20 flex items-center justify-center mr-6
transition-colors">
                    <Monitor
                        className="w-8 h-8 md:w-10 md:h-10 text-brand-green"
                        strokeWidth={2.5}
                    />
                </div>
            </button>
        </Link>
    )}

```

```

        <div className="flex-1">
            <h3 className="text-xl md:text-2xl font-black text-gray-900 mb-1 uppercase tracking-wide group-hover:text-brand-green transition-colors">
                VIEW DASHBOARD
            </h3>
            <p className="text-sm md:text-base text-gray-500">
                View Dashboard
            </p>
        </div>
    </button>
</Link>
)}

<div className="absolute bottom-4 right-4 z-50">
    {isAuthenticated ? (
        <button
            onClick={logout}
            className="p-2 hover:p-3 bg-brand-green text-white rounded-full hover:bg-brand-green/95 shadow-[0_2px_10px_rgba(0,0,0,0.05)] transition-all duration-200"
            title="Logout"
        >
            <LogOut className="size-4" />
        </button>
    ) : (
        <Link
            to="/login"
            className="p-2 hover:p-3 block bg-brand-green text-white rounded-full hover:bg-brand-green/95 shadow-[0_2px_10px_rgba(0,0,0,0.05)] transition-all duration-200"
            title="Staff Login"
        >
            <LogIn className="size-4" />
        </Link>
    )}
</div>
</SplitLayout>
);
};

```

```
export default Home;
```

Queue Management

```
import { formatWindowLabel } from "@/lib/windowLabel";
import QueueSummaryCards from
"@/features/staff/components/QueueSummaryCards";
import QueueTicketsTable from
"@/features/staff/components/QueueTicketsTable";
import QueueActionCards from
"@/features/staff/components/QueueActionCards";
import { useStaffQueueManagement } from
"@/features/staff/hooks/useStaffQueueManagement";
import { LogOut, UserRound, BadgeInfo } from "lucide-react";
import { toast } from "sonner";

const QueueManagement = () => {
  const {
    dashboard,
    isLoading,
    isError,
    allTickets,
    userName,
    handleLogout,
    selectedWindow,
    windowStatus,
    disableCallNext,
    activeTicketLabel,
    activeTicketId,
    isQueueEmpty,
    isLastTicket,
    pending,
    handleLeaveWindow,
    handleMainActionClick,
    handleStartServing,
```

```

    handleComplete,
    handleSkip,
    handleRemove,
    handleRecall,
  } = useStaffQueueManagement();

  const serviceWindowLabel = `${formatWindowLabel(selectedWindow,
  "")}`.trim();

  const handleTopSkip = () => {
    if (!activeTicketId) {
      toast.error("No active ticket found to skip.");
      return;
    }

    handleSkip(activeTicketId);
  };

  if (isLoading) {
    return (
      <div className="h-screen
bg-[radial-gradient(circle_at_top,#ecffd8_0%,#f7fdf1_26%,#fffef8_58%,#f8fafc_100%)]">
        <div className="flex min-h-[70vh] items-center justify-center
p-8">
          <div className="flex flex-col items-center gap-4 rounded-3xl
border border-[#2c990f]/15 bg-white/85 px-10 py-8
shadow-[0_20px_50px_rgba(44,153,15,0.10)] backdrop-blur">
            <div className="h-12 w-12 animate-spin rounded-full border-4
border-[#2c990f]/15 border-b-[#dcbc34]" />
            <div className="space-y-1 text-center">
              <p className="text-sm font-semibold text-brand-green">
                Loading queue dashboard...
              </p>
              <p className="text-xs text-muted-foreground">
                Preparing live staff queue controls
              </p>
            </div>
          </div>
        </div>
      </div>
    );
  }

```

```

        </div>
    );
}

if (isError || !dashboard) {
    return (
        <div className="h-screen
bg-[radial-gradient(circle_at_top,#ecffd8_0%,#f7fdf1_26%,#fffef8_58%,#f8fafc_100%)]">
            <div className="flex min-h-[70vh] items-center justify-center
p-8">
                <div className="w-full max-w-md rounded-[2rem] border
border-[#2c990f]/10 bg-white/90 p-8 text-center
shadow-[0_20px_50px_rgba(0,0,0,0.06)]">
                    <div className="mx-auto mb-4 flex h-16 w-16 items-center
justify-center rounded-full
bg-[linear-gradient(135deg,rgba(44,153,15,0.10)_0%,rgba(220,188,52,0.16)_100%)]">
                        <BadgeInfo className="h-8 w-8 text-[#2c990f]/70" />
                    </div>
                    <p className="text-xl font-bold text-brand-green">
                        Unable to load dashboard
                    </p>
                    <p className="mt-2 text-sm text-muted-foreground">
                        Make sure you are assigned to a service.
                    </p>
                </div>
            </div>
        </div>
    );
}

return (
    <div className="h-screen
bg-[radial-gradient(circle_at_top,#ecffd8_0%,#f7fdf1_24%,#fffef7_58%,#f8fafc_100%)] font-sans">
        <div className="mx-auto max-w-7xl space-y-5 p-4 md:p-6">
            <header className="relative overflow-hidden rounded-[2rem] border
border-brand-green/30 bg-white px-5 py-5
shadow-[0_20px_60px_rgba(0,0,0,0.08)] backdrop-blur md:px-6">

```

```

<div className="relative flex flex-col gap-4 md:flex-row
md:items-center md:justify-between">
  <div className="space-y-1.5">
    <h1 className="text-3xl font-black tracking-tight
text-brand-green md:text-4xl">
      Queue Management
    </h1>
    <p className="text-sm text-slate-600">
      Staff operations dashboard
    </p>
  </div>

  <div className="flex flex-wrap items-center gap-2.5">
    <div className="inline-flex items-center rounded-full
bg-[linear-gradient(135deg,rgba(44,153,15,0.14)_0%,rgba(255,255,255,1)_100
%)] px-3 py-1.5 text-xs font-semibold text-brand-green shadow-sm ring-1
ring-[#2c990f]/10">
      Service: {serviceWindowLabel}
    </div>

    <div className="inline-flex items-center gap-1.5
rounded-full border border-red-100 bg-white px-3 py-1.5 text-xs
font-medium uppercase text-slate-700">
      <UserRound className="h-3.5 w-3.5" />
      {userName || "Unknown"}
    </div>

    <button
      onClick={handleLogout}
      className="inline-flex h-8 cursor-pointer items-center
justify-center gap-2 rounded-full border border-red-100 bg-white px-4
text-sm font-semibold text-gray-700 transition-all hover:border-red-200
hover:text-red-700"
    >
      <LogOut className="h-4 w-4" />
      Logout
    </button>
  </div>
</div>
</header>

```

```

<QueueActionCards
  windowStatus={windowStatus}
  disableCallNext={disableCallNext}
  isEmpty={isEmpty}
  isLastTicket={isLastTicket}
  nextTicket={dashboard.next_ticket}
  activeTicketLabel={activeTicketLabel}
  isLeaveWindowPending={pending.leaveWindow}
  isMainActionPending={pending.callNext || pending.complete}
  isSkipPending={pending.skip}
  onLeaveWindow={handleLeaveWindow}
  onMainAction={handleMainActionClick}
  onSkip={handleTopSkip}
/>

<QueueSummaryCards
  dashboard={dashboard}
  nowServingLabel={activeTicketLabel}
/>

<QueueTicketsTable
  tickets={allTickets}
  disableStartServing={disableCallNext}
  onStartServing={handleStartServing}
  onComplete={handleComplete}
  onSkip={handleSkip}
  onRemove={handleRemove}
  onRecall={handleRecall}
  isServingPending={pending.startServing}
  isCompletingPending={pending.complete}
  isSkippingPending={pending.skip}
  isRemovingPending={pending.remove}
  isRecallingPending={pending.recall}
/>
</div>
</div>
);
};

```

```
export default QueueManagement;
```

```

import { useState, useCallback } from "react";
import {
  ChevronRight,
  LogOut,
  Mic,
  CheckCircle2,
  SkipForward,
} from "lucide-react";

interface QueueActionCardsProps {
  windowStatus?: "active" | "inactive" | "maintenance";
  disableCallNext: boolean;
  isQueueEmpty: boolean;
  isLastTicket: boolean;
  nextTicket: string | null | undefined;
  activeTicketLabel?: string;
  isLeaveWindowPending: boolean;
  isMainActionPending: boolean;
  isSkipPending: boolean;
  onLeaveWindow: () => void;
  onMainAction: () => void;
  onSkip: () => void;
}

const QueueActionCards = ({
  windowStatus,
  disableCallNext,
  isQueueEmpty,
  isLastTicket,
  nextTicket,
  activeTicketLabel,
  isLeaveWindowPending,
  isMainActionPending,
  isSkipPending,
  onLeaveWindow,
  onMainAction,
  onSkip,
}: QueueActionCardsProps) => {
  const isWindowActive = windowStatus === "active";

```

```
const isServing = !!activeTicketLabel;
const [isCooldown, setIsCooldown] = useState(false);

// Hide skip button when it's the last ticket or not serving
const showSkipButton = isServing && !isLastTicket;

// Determine if this is the last ticket action (Complete instead of Call
Next)
const isCompleteOnly = isServing && isLastTicket;

const handleMainAction = useCallback(() => {
  if (isCooldown) {
    onMainAction();
    return;
  }

  setIsCooldown(true);
  setTimeout(() => setIsCooldown(false), 3000);
  onMainAction();
}, [isCooldown, onMainAction]);

// Get button text based on state
const getButtonText = () => {
  if (isCompleteOnly) {
    return "Complete Ticket";
  }
  return "Call Next";
};

// Get button description based on state
const getButtonDescription = () => {
  if (nextTicket && !isQueueEmpty && !isCompleteOnly) {
    return isServing
      ? `Complete ${activeTicketLabel} and call ${nextTicket}`
      : `Call ${nextTicket}`;
  }
  if (isCompleteOnly) {
    return `Complete ${activeTicketLabel} || "current ticket"`;
  }
  if (isServing) {
```

```

        return `Complete ${activeTicketLabel || "current ticket"}`;
    }
    return "No tickets waiting";
};

return (
    <div
        className={`grid gap-4 ${
            showSkipButton
                ? "grid-cols-1 lg:grid-cols-3"
                : "grid-cols-1 sm:grid-cols-2"
            }`}
    >
        {/* Leave Window */}
        <button
            className="group relative h-30 cursor-pointer overflow-hidden
rounded-[1.75rem] border border-brand-green/30 bg-white transition-all
duration-200 hover:-translate-y-0.5 hover:border-brand-green/60
hover:shadow-lg active:scale-[0.99] disabled:cursor-not-allowed
disabled:opacity-60"
            onClick={onLeaveWindow}
            disabled={isLeaveWindowPending}
        >
            <div className="relative flex h-full items-center gap-4 px-5">
                <div className="flex h-13 w-13 shrink-0 items-center
justify-center rounded-2xl
bg-[linear-gradient(135deg,rgba(44,153,15,0.14)_0%,rgba(255,255,255,1)_100%)] shadow-sm ring-1 ring-[#2c990f]/10 transition-colors duration-200">
                    <LogOut className="h-6 w-6 text-brand-green transition-colors
duration-200 group-hover:text-inherit" />
                </div>

                <div className="min-w-0 text-left">
                    <h3 className="text-lg font-bold text-slate-900 md:text-xl">
                        Leave Window
                    </h3>
                    <p className="mt-1 text-xs leading-relaxed text-slate-500">
                        {isWindowActive
                            ? "Release your active window and return to selection"
                            : "Return to window selection"}
                    </p>
                </div>
            </div>
        </button>
    </div>
);

```

```

    </p>
  </div>

  <div className="ml-auto flex h-9 w-9 items-center justify-center rounded-full bg-white/85 shadow-sm transition-transform duration-200 group-hover:translate-x-0.5">
    <ChevronRight className="h-5 w-5 text-slate-400 group-hover:text-brand-green" />
  </div>
</div>
</button>

{/* Main Action - Changes based on isCompleteOnly */}
<button
  className={`group relative h-30 cursor-pointer overflow-hidden rounded-[1.75rem] border transition-all duration-200 hover:-translate-y-0.5 active:scale-[0.99] hover:shadow-lg disabled:cursor-not-allowed disabled:opacity-60 ${
    isCompleteOnly
      ? "border-emerald-600/40 bg-emerald-600"
      : "border-[#2c990f]/30 bg-brand-green"
  }`}
  onClick={handleMainAction}
  disabled={
    isMainActionPending ||
    isCooldown ||
    (isQueueEmpty && !isServing) ||
    disableCallNext
  }
>
  <div
    className={`absolute inset-0 ${
      isCompleteOnly ? "bg-emerald-600" : "bg-brand-green"
    }`}
  />

  <div className="relative flex h-full items-center gap-4 px-5">
    <div className="flex h-13 w-13 shrink-0 items-center justify-center rounded-2xl bg-white/12 shadow-inner ring-1 ring-white/15 backdrop-blur-sm">

```

```

        {isCompleteOnly ? (
            <CheckCircle2 className="h-6 w-6 text-white" />
        ) : (
            <Mic className="h-6 w-6 text-white" />
        )}
    </div>

    <div className="min-w-0 flex-1 text-left">
        <h3 className="text-lg font-black text-white md:text-xl">
            {getButtonText()}
        </h3>
        <p className="mt-1 text-xs leading-relaxed text-white/85">
            {getDescription()}
        </p>
    </div>

    <div className="ml-auto flex h-9 w-9 items-center justify-center rounded-full bg-white/12 ring-1 ring-white/15 transition-transform duration-200 group-hover:translate-x-0.5">
        <ChevronRight className="h-5 w-5 text-white/70" />
    </div>
</div>
</button>

{/* Skip Button - Orange color */}
{showSkipButton && (
    <button
        className="group relative h-30 cursor-pointer overflow-hidden rounded-[1.75rem] border border-orange-300 bg-orange-500 transition-all duration-200 hover:-translate-y-0.5 hover:border-orange-400 hover:bg-orange-600 hover:shadow-lg active:scale-[0.99] disabled:cursor-not-allowed disabled:opacity-60"
        onClick={onSkip}
        disabled={isSkipPending}
    >
        <div className="absolute inset-0 bg-orange-500" />

        <div className="relative flex h-full items-center gap-4 px-5">

```

```

        <div className="flex h-13 w-13 shrink-0 items-center
justify-center rounded-2xl bg-white/12 shadow-inner ring-1 ring-white/15
backdrop-blur-sm">
            <SkipForward className="h-6 w-6 text-white" />
        </div>

        <div className="min-w-0 flex-1 text-left">
            <h3 className="text-lg font-black text-white md:text-xl">
                Skip Ticket
            </h3>
            <p className="mt-1 text-xs leading-relaxed text-white/85">
                Mark {activeTicketLabel || "current ticket"} as skipped
            </p>
        </div>

        <div className="ml-auto flex h-9 w-9 items-center
justify-center rounded-full bg-white/12 ring-1 ring-white/15
transition-transform duration-200 group-hover:translate-x-0.5">
            <ChevronRight className="h-5 w-5 text-white/70" />
        </div>
    </div>
</button>
    )}
</div>
);
};

export default QueueActionCards;

```

```

import React from "react";
import type { Ticket } from "@features/tickets/types";
import { Badge } from "@components/ui/badge";
import { Button } from "@components/ui/button";
import {
    PlayCircle,
    CheckCircle2,
    Trash2,
    RotateCcw,

```

```

    Ticket as TicketIcon,
    SkipForward,
  } from "lucide-react";

const statusConfig: Record<string, { label: string; className: string }> =
{
  waiting: {
    label: "Waiting",
    className: "border-[#dcb334]/25 bg-[#fff7dd] text-[#8c6f00]",
  },
  notified: {
    label: "Notified",
    className: "border-blue-200 bg-blue-50 text-blue-700",
  },
  serving: {
    label: "Serving",
    className: "border-emerald-200 bg-emerald-50 text-emerald-700",
  },
  served: {
    label: "Served",
    className: "border-slate-200 bg-slate-100 text-slate-500",
  },
  cancelled: {
    label: "Cancelled",
    className: "border-red-200 bg-red-50 text-red-600",
  },
  skipped: {
    label: "Skipped",
    className: "border-orange-200 bg-orange-50 text-orange-600",
  },
};

interface QueueTicketsTableProps {
  tickets: Ticket[];
  disableStartServing?: boolean;
  onStartServing: (ticketNumber: string) => void;
  onComplete: (ticketId: string) => void;
  onSkip: (ticketId: string) => void;
  onRemove: (ticketId: string) => void;
  onRecall: (ticketId: string) => void;
}

```

```

    isServingPending: boolean;
    isCompletingPending: boolean;
    isSkippingPending: boolean;
    isRemovingPending: boolean;
    isRecallingPending: boolean;
  }

const GRID_COLS = "grid grid-cols-[1fr_120px_auto] items-center gap-4
px-6";

const ColumnHeaders = () => (
  <div
    className={` ${GRID_COLS} bg-white py-3 text-[11px] font-semibold
uppercase tracking-wider text-muted-foreground/70`}
  >
    <span>Ticket #</span>
    <span className="text-center">Status</span>
    <span className="text-center sm:w-[280px]">Actions</span>
  </div>
);

const QueueTicketsTable: React.FC<QueueTicketsTableProps> = ({
  tickets,
  disableStartServing = false,
  onStartServing,
  onComplete,
  onSkip,
  onRemove,
  onRecall,
  isServingPending,
  isCompletingPending,
  isSkippingPending,
  isRemovingPending,
  isRecallingPending,
}) => {
  const visibleTickets = tickets.filter(
    (ticket) => ticket.status !== "serving",
  );

  if (visibleTickets.length === 0) {

```

```

    return (
      <div className="overflow-hidden rounded-[1.75rem] border
border-brand-green/30 bg-white">
        <ColumnHeaders />
        <div className="flex flex-col items-center justify-center gap-3
border-t border-[#2c990f]/8 py-16 text-sm text-muted-foreground">
          <div className="flex h-16 w-16 items-center justify-center
rounded-full
bg-[linear-gradient(135deg,rgba(44,153,15,0.10)_0%,rgba(220,188,52,0.14)_1
00%)]">
            <TicketIcon className="size-8 text-[#2c990f]/55" />
          </div>
          <div className="text-center">
            <p className="font-medium text-brand-green">No tickets in
queue</p>
            <p className="mt-1 text-xs text-muted-foreground/70">
              Tickets will appear here when customers join the queue
            </p>
          </div>
        </div>
      </div>
    );
  }

  const renderRowActions = (ticket: Ticket) => {
    const s = ticket.status;

    return (
      <div className="flex flex-wrap items-center justify-end gap-1.5">
        {(s === "waiting" || s === "notified" || s === "skipped") && (
          <Button
            size="sm"
            className="h-8 cursor-pointer rounded-lg bg-brand-green px-3
text-xs font-semibold text-white shadow-sm hover:bg-[#24820c]"
            onClick={() => onStartServing(ticket.display_number)}
            disabled={isServingPending || disableStartServing}
          >
            <PlayCircle className="mr-1 h-3.5 w-3.5" />
            Serve
          </Button>

```

```

    })

    {s === "serving" && (
      <>
        <Button
          size="sm"
          className="h-8 cursor-pointer rounded-lg bg-emerald-600 px-3
text-xs font-semibold text-white shadow-sm hover:bg-emerald-700"
          onClick={() => onComplete(ticket.ticket_id)}
          disabled={isCompletingPending}
        >
          <CheckCircle2 className="mr-1 h-3.5 w-3.5" />
          Complete
        </Button>

        <Button
          size="sm"
          className="h-8 cursor-pointer rounded-lg bg-orange-500 px-3
text-xs font-semibold text-white shadow-sm hover:bg-orange-600"
          onClick={() => onSkip(ticket.ticket_id)}
          disabled={isSkippingPending}
        >
          <SkipForward className="mr-1 h-3.5 w-3.5" />
          Skip
        </Button>
      </>
    )}

    {(s === "waiting" ||
      s === "notified" ||
      s === "serving" ||
      s === "skipped") && (
      <Button
        size="sm"
        variant="outline"
        className="h-8 cursor-pointer rounded-lg border-red-200
bg-white px-3 text-xs font-semibold text-red-600 hover:bg-red-50"
        onClick={() => onRemove(ticket.ticket_id)}
        disabled={isRemovingPending}
      >

```

```

        <Trash2 className="h-3.5 w-3.5" />
      </Button>
    )}

    {s === "cancelled" && (
      <Button
        size="sm"
        variant="outline"
        className="h-8 cursor-pointer rounded-lg border-blue-200
bg-white px-3 text-xs font-semibold text-blue-600 hover:bg-blue-50"
        onClick={() => onRecall(ticket.ticket_id)}
        disabled={isRecallingPending}
      >
        <RotateCcw className="mr-1 h-3.5 w-3.5" />
        Recall
      </Button>
    )}
  </div>
);
};

return (
  <div className="overflow-hidden rounded-[1.75rem] border
border-brand-green/30 bg-white">
    <ColumnHeaders />
    <div className="max-h-[40vh] overflow-y-auto divide-y
divide-[#2c990f]/8">
      {visibleTickets.map((ticket) => {
        const config = statusConfig[ticket.status] || {
          label: ticket.status,
          className: "border-slate-200 bg-slate-100 text-slate-600",
        };
      })}

      return (
        <div
          key={ticket.ticket_id}
          className={`group ${GRID_COLS} py-3.5 transition-all
duration-150
hover:bg-[linear-gradient(90deg,rgba(44,153,15,0.04)_0%,rgba(255,255,255,1
)_60%,rgba(220,188,52,0.05)_100%)]`}

```

```

        >
        <div className="flex items-center gap-3">
            <div className="hidden sm:flex h-10 min-w-10 items-center
justify-center rounded-xl
bg-[linear-gradient(135deg,rgba(44,153,15,0.14)_0%,rgba(255,255,255,1)_100
%)] shadow-sm ring-1 ring-[#2c990f]/10">
                <TicketIcon className="h-4 w-4 text-brand-green" />
            </div>
            <span className="text-base font-black tracking-tight">
                {ticket.display_number}
            </span>
        </div>

        <div>
            <Badge
                variant="outline"
                className={` ${config.className} rounded-full px-2.5 py-1
font-semibold`}
            >
                {config.label}
            </Badge>
        </div>

        <div className="text-right">{renderRowActions(ticket)}</div>
    </div>

    );
    })}
</div>
</div>
);
};

export default QueueTicketsTable;

```

Staff Window Card

```
import { Archive, FileEdit, FileText } from "lucide-react";
import type { WindowSelectionWindow } from "@/features/staff/api";

const getServiceIcon = (name: string) => {
  const lower = name.toLowerCase();
  if (lower.includes("cashier"))
    return <Archive className="size-5 text-brand-green" strokeWidth={2} />;
  if (lower.includes("registrar"))
    return <FileEdit className="size-5 text-brand-green" strokeWidth={2} />;
  return <FileText className="size-5 text-brand-green" strokeWidth={2} />;
};

export interface StaffWindowCardProps {
  windowData: WindowSelectionWindow;
  serviceName: string;
  currentUserName: string;
  claimLoading: number | null;
  selectedWindowId: number | null;
  isPending: boolean;
  onSelect: (windowData: WindowSelectionWindow) => void;
}

export const StaffWindowCard = ({
  windowData,
  serviceName,
  currentUserName,
  claimLoading,
  selectedWindowId,
  isPending,
  onSelect,
}: StaffWindowCardProps) => {
  const claimedByCurrentUser =
    !!windowData.claimed_by &&
    windowData.claimed_by.toLowerCase() === currentUserName.toLowerCase();

  const blocked =
    windowData.status === "inactive" ||
```

```

windowData.status === "maintenance" ||
!windowData.is_available ||
(windowData.is_in_use && !claimedByCurrentUser);

const occupiedBy =
  windowData.claimed_by || windowData.current_staff_name || null;

const statusText =
  windowData.status === "inactive"
    ? "Unavailable"
    : !windowData.is_available
    ? "Temporarily unavailable"
    : windowData.status === "maintenance"
    ? "Under maintenance"
    : windowData.is_in_use
    ? occupiedBy
      ? `In use by ${occupiedBy}`
      : "In use"
    : "Available";

const overlayLabel =
  windowData.status === "maintenance"
    ? "Maintenance"
    : windowData.status === "inactive"
    ? "Inactive"
    : null;

return (
  <button
    className={`group relative overflow-hidden bg-white rounded-xl p-6
border border-brand-green/30 shadow-[0_4px_20px_rgba(0,0,0,0.04)]
transition-all duration-200 text-center flex items-center gap-4 ${
      blocked
        ? "opacity-60 cursor-not-allowed bg-gray-50"
        : "cursor-pointer hover:shadow-[0_8px_30px_rgba(0,0,0,0.08)]
hover:border-brand-green/30 active:scale-[0.98]"
      }`}
    disabled={blocked || isPending}
    onClick={() => {
      if (!blocked) {

```

```

        onSelect(windowData);
    }
    }}
    >
    <div className="flex-shrink-0 size-10 rounded-xl
bg-[linear-gradient(135deg,rgba(44,153,15,0.14)_0%,rgba(255,255,255,1)_100
%)] shadow-sm ring-1 ring-[#2c990f]/10 flex items-center justify-center
transition-colors relative">
    {getServiceIcon(serviceName)}
</div>
<div className="flex flex-col items-start">
    <h3 className="text-sm md:text-base font-black text-gray-900
uppercase tracking-wide group-hover:text-brand-green transition-colors">
        {windowData.name}
    </h3>
    <p className="mt-1 text-xs md:text-sm font-medium text-gray-400
group-hover:text-gray-500 transition-colors">
        {statusText}
    </p>
    {claimLoading === windowData.id && (
        <div className="mt-3 flex items-center gap-2 text-sm
font-semibold text-brand-green">
            <div className="h-4 w-4 animate-spin rounded-full border-2
border-brand-green border-t-transparent" />
            Claiming...
        </div>
    )}
    {selectedWindowId === windowData.id && (
        <span className="mt-3 rounded-full border border-brand-green/25
bg-brand-green/10 px-3 py-1 text-xs font-semibold text-brand-green">
            Selected
        </span>
    )}

    {overlayLabel && (
        <div className="absolute inset-0 z-10 bg-white/60
backdrop-blur-[1px] flex items-center justify-center">
            <span className="bg-red-50 text-red-600 border border-red-200
px-3 py-1 rounded-md font-bold text-xs uppercase tracking-wide shadow-sm">
                {overlayLabel}
            </span>
        </div>
    )}

```

```
        </span>
      </div>
    ) }
  </div>
</button>
);
};
```

State Management

Auth

```
import { create } from "zustand";
import { persist, createJSONStorage } from "zustand/middleware";
import type { User } from "@features/user/types";
import { authApi } from "@features/user/api";
import { useStaffWindowStore } from "../staffWindowStore";

interface AuthState {
  user: User | null;
  loading: boolean;
  isAuthenticated: boolean;
  checkAuth: () => Promise<void>;
  logout: () => Promise<void>;
  setUser: (user: User | null) => void;
  setLoading: (loading: boolean) => void;
}

export const useAuthStore = create<AuthState>() (
  persist(
    (set) => ({
      user: null,
      loading: true,
      isAuthenticated: false,

      setUser: (user) => set({ user, isAuthenticated: !!user }),
      setLoading: (loading) => set({ loading }),

      checkAuth: async () => {
        try {
          set({ loading: true });
          const response = await authApi.getCurrentUser();
          if (response.success) {
            set({ user: response.user, isAuthenticated: true });
          } else {
            set({ user: null, isAuthenticated: false });
          }
        } catch (error) {
```

```

        console.error("Auth check failed:", error);
        set({ user: null, isAuthenticated: false });
    } finally {
        set({ loading: false });
    }
},

logout: async () => {
    try {
        await authApi.logout();
    } catch (error) {
        console.error("Logout failed:", error);
    } finally {
        localStorage.removeItem("ws_access_token");
        localStorage.removeItem("auth-storage");
        localStorage.removeItem("staff-window-storage");
        useStaffWindowStore.getState().clearAll();
        set({ user: null, isAuthenticated: false });
        window.location.href = "/";
    }
},
}),
{
    name: "auth-storage",
    storage: createJSONStorage(() => localStorage),
    partialize: (state) => ({
        user: state.user,
        isAuthenticated: state.isAuthenticated,
    }),
},
),
);

```

Staff Window Store

```
import { create } from "zustand";
import { persist, createJSONStorage } from "zustand/middleware";
import type { ServiceWindow } from "@features/services/types";
import { writeClaimedWindowIdToSession } from
"@features/staff/windowRelease";

export interface StaffServiceInfo {
  id: number;
  name: string;
  prefix: string;
}

interface StaffWindowState {
  selectedWindow: ServiceWindow | null;
  claimedWindowId: number | null;
  serviceInfo: StaffServiceInfo | null;
  setSelectedWindow: (window: ServiceWindow) => void;
  setServiceInfo: (service: StaffServiceInfo) => void;
  // Clears only the selected window (leave-window / release flow).
  // serviceInfo is kept so the onboarding page can still show windows.
  clearSelectedWindow: () => void;
  // Clears everything - use only on full logout.
  clearAll: () => void;
}

export const useStaffWindowStore = create<StaffWindowState>()(
  persist(
    (set) => ({
      selectedWindow: null,
      claimedWindowId: null,
      serviceInfo: null,
      setSelectedWindow: (window) => {
        const maybeWrapped = window as ServiceWindow & {
          window?: ServiceWindow;
        };
        const normalizedWindow = maybeWrapped.window || window;
        writeClaimedWindowIdToSession(normalizedWindow.id);
        set({
          selectedWindow: normalizedWindow,
```

```
        claimedWindowId: normalizedWindow.id,
      });
    },
    setServiceInfo: (service) =>
      set({
        serviceInfo: {
          id: service.id,
          name: service.name,
          prefix: service.prefix,
        },
      }),
    clearSelectedWindow: () => {
      writeClaimedWindowIdToSession(null);
      set({ selectedWindow: null, claimedWindowId: null });
    },
    clearAll: () => {
      writeClaimedWindowIdToSession(null);
      set({ selectedWindow: null, claimedWindowId: null, serviceInfo:
null });
    },
  }),
  {
    name: "staff-window-storage",
    storage: createJSONStorage(() => localStorage),
  },
),
);
```

Window Selection State

```
import { create } from "zustand";
import type { WindowSelectionWindow } from "@features/staff/api";

interface WindowSelectionState {
  windows: WindowSelectionWindow[];
  selectedWindowId: number | null;
  claimLoading: number | null;
  releaseLoading: boolean;
  socketConnected: boolean;
  socketReconnecting: boolean;
  lastSyncAt: string | null;
  error: string | null;
  setWindows: (windows: WindowSelectionWindow[]) => void;
  setSelectedWindowId: (windowId: number | null) => void;
  setClaimLoading: (windowId: number | null) => void;
  setReleaseLoading: (isLoading: boolean) => void;
  setSocketState: (connected: boolean, reconnecting: boolean) => void;
  setError: (error: string | null) => void;
  resetSelectionState: () => void;
}

export const useWindowSelectionStore = create<WindowSelectionState>() (
  (set) => ({
    windows: [],
    selectedWindowId: null,
    claimLoading: null,
    releaseLoading: false,
    socketConnected: false,
    socketReconnecting: false,
    lastSyncAt: null,
    error: null,
    setWindows: (windows) =>
      set({
        windows,
        lastSyncAt: new Date().toISOString(),
        error: null,
      }),
    setSelectedWindowId: (selectedWindowId) => set({ selectedWindowId }),
    setClaimLoading: (claimLoading) => set({ claimLoading }),
  })
);
```

```
setReleaseLoading: (releaseLoading) => set({ releaseLoading }),
setSocketState: (socketConnected, socketReconnecting) =>
  set({ socketConnected, socketReconnecting }),
setError: (error) => set({ error }),
resetSelectionState: () =>
  set({
    windows: [],
    selectedWindowId: null,
    claimLoading: null,
    releaseLoading: false,
    socketConnected: false,
    socketReconnecting: false,
    lastSyncAt: null,
    error: null,
  }),
  },
);
```

QUEUICK Backend Code

Models

```
import uuid
from django.db import models
from django.utils import timezone
from django.db.models import Max
from django.db.models.signals import post_migrate
from django.dispatch import receiver
from django.contrib.auth.models import User

# =====
# SERVICE MODELS
# =====

class Service(models.Model):
    name = models.CharField(max_length=100, unique=True)
    description = models.TextField(blank=True)
    prefix = models.CharField(max_length=10, default="", blank=True,
unique=True)

    is_active = models.BooleanField(default=True)
    average_service_time = models.PositiveIntegerField(default=5)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['name']

    def __str__(self):
        return self.name

    def get_next_queue_number(self):
        today = timezone.now().date()

        last_ticket = self.tickets.filter(
            ticket_date=today
        ).aggregate(Max('queue_number'))['queue_number__max']

        if last_ticket:
            return last_ticket + 1
```

```

        return 1

    def get_display_number(self, queue_number):
        if self.prefix:
            return f"{self.prefix}{queue_number:03d}"
        return f"{queue_number:03d}"

    @property
    def waiting_count(self):
        today = timezone.now().date()
        return self.tickets.filter(
            ticket_date=today,
            status__in=['waiting', 'notified']
        ).count()

    @property
    def currently_serving(self):
        today = timezone.now().date()
        return self.tickets.filter(
            ticket_date=today,
            status='serving'
        ).first()

    def update_active_status(self):
        # Service availability is managed explicitly via
service.is_active.
        return False

    def can_accept_tickets(self, current_time=None):
        #Service can accept tickets if it's active
        return self.is_active

class ServiceWindow(models.Model):
    WINDOW_STATUS = [
        ('active', 'Active'),
        ('inactive', 'Inactive'),
        ('maintenance', 'Under Maintenance'),
    ]

```

```

        service = models.ForeignKey(Service, on_delete=models.CASCADE,
related_name='windows')
        window_number = models.PositiveIntegerField()
        name = models.CharField(max_length=100, blank=True)
        status = models.CharField(max_length=20, choices=WINDOW_STATUS,
default='active')
        description = models.TextField(blank=True)

        current_staff = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, blank=True, related_name='assigned_window')

        created_at = models.DateTimeField(auto_now_add=True)
        updated_at = models.DateTimeField(auto_now=True)

        class Meta:
            ordering = ['service', 'window_number']
            unique_together = ['service', 'window_number']

        def __str__(self):
            return f"{self.service.name} - {self.name or f'Window {self.window_number}'}"

        @property
        def is_available(self):
            return self.status == 'active' and self.current_staff_id is None

        @property
        def is_in_use(self):
            return self.current_staff_id is not None

        def assign_staff(self, staff_user):
            # Clear this staff from any other window
            ServiceWindow.objects.filter(current_staff=staff_user).update(current_staff=None)

            # Assign to this window
            self.current_staff = staff_user
            self.save()

```

```

        # Update staff profile
        if hasattr(staff_user, 'staff_profile'):
            staff_user.staff_profile.current_window = self
            staff_user.staff_profile.save()

# =====
# STAFF PROFILE
# =====
class StaffProfile(models.Model):
    ROLE_CHOICES = [
        ('admin', 'Administrator'),
        ('staff', 'Staff Member'),
    ]

    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='staff_profile')
    role = models.CharField(max_length=20, choices=ROLE_CHOICES,
default='staff')
    assigned_service = models.ForeignKey(Service,
on_delete=models.SET_NULL, null=True, blank=True)
    can_manage_queue = models.BooleanField(default=True)

    last_login_at = models.DateTimeField(null=True, blank=True)
    last_logout_at = models.DateTimeField(null=True, blank=True)

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        verbose_name = 'Staff Profile'
        verbose_name_plural = 'Staff Profiles'

    def __str__(self):
        service_name = self.assigned_service.name if self.assigned_service
else "No Service"
        return f"{self.user.username} - {service_name}"

    def set_current_window(self, window_id):
        # Set the window this staff is currently manning

```

```

        try:
            window = ServiceWindow.objects.get(
                id = window_id,
                service = self.assigned_service,
                status = 'active'
            )

            # Clear previous staff from this window

ServiceWindow.objects.filter(current_staff=self.user).update(current_staff
=None)

            # Assign this staff to new window
            window.current_staff = self.user
            window.save()

            # Update profile
            self.current_window = window
            self.save()

            return window
        except ServiceWindow.DoesNotExist:
            return None

    def clear_current_window(self):
        if self.current_window:
            self.current_window.current_staff = None
            self.current_window.save()
            self.current_window = None
            self.save()

# =====
# TICKET MODEL
# =====
class Ticket(models.Model):
    STATUS_CHOICES = [
        ('waiting', 'Waiting'),
        ('notified', 'Notified'),
        ('serving', 'Currently Serving'),

```

```

        ('served', 'Served'),
        ('cancelled', 'Cancelled'),
        ('skipped', 'Skipped'),
    ]

    ticket_id = models.UUIDField(default=uuid.uuid4, editable=False,
unique=True)
    service = models.ForeignKey(Service, on_delete=models.CASCADE,
related_name='tickets')
    queue_number = models.PositiveIntegerField(default=0)
    display_number = models.CharField(max_length=20, blank=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES,
default='waiting')

    assigned_window = models.ForeignKey(ServiceWindow,
on_delete=models.SET_NULL, null=True, blank=True, related_name='tickets')

    ticket_date = models.DateField(default=timezone.now)

    called_by = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, blank=True, related_name='called_tickets')
    served_by = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, blank=True, related_name='served_tickets')

    created_at = models.DateTimeField(auto_now_add=True)
    called_at = models.DateTimeField(null=True, blank=True)
    served_at = models.DateTimeField(null=True, blank=True)
    skipped_at = models.DateTimeField(null=True, blank=True)

    notes = models.TextField(blank=True)

    sms_phone = models.CharField(max_length=15, blank=True, null=True)
    sms_sent = models.BooleanField(default=False)
    sms_sent_at = models.DateTimeField(null=True, blank=True)

    class Meta:
        ordering = ['service', 'ticket_date', 'queue_number']
        unique_together = ['service', 'queue_number', 'ticket_date']
        indexes = [
            models.Index(fields=['status', 'ticket_date']),

```

```

        models.Index(fields=['service', 'status']),
        models.Index(fields=['assigned_window', 'status']),
    ]

    def __str__(self):
        return f"{self.service.name} - {self.display_number}"

    def save(self, *args, **kwargs):
        if self._state.adding:
            self.ticket_date = timezone.now().date()
            self.queue_number = self.service.get_next_queue_number()
            self.display_number =
self.service.get_display_number(self.queue_number)

            super().save(*args, **kwargs)

    @property
    def people_ahead(self):
        if self.status in ['serving', 'served', 'cancelled', 'skipped']:
            return 0

        return Ticket.objects.filter(
            service=self.service,
            ticket_date=self.ticket_date,
            status__in=['waiting', 'notified'],
            queue_number__lt=self.queue_number
        ).count()

    @property
    def is_today(self):
        return self.ticket_date == timezone.now().date()

    @property
    def wait_time_minutes(self):
        if self.status in ['served', 'cancelled', 'skipped']:
            return 0

        return self.people_ahead * self.service.average_service_time

    @classmethod
    def cancel_old_tickets(cls):

```

```

        return cls.objects.filter(
            ticket_date__lt=timezone.now().date(),
            status__in=['waiting', 'notified', 'serving']
        ).update(status='cancelled')

# =====
# POST MIGRATE INITIAL DATA
# =====
@receiver(post_migrate)
def create_initial_data(sender, **kwargs):
    if sender.name != 'queueing':
        return

    admin_user, created = User.objects.get_or_create(
        username='admin',
        defaults={
            'email': 'admin@school.edu',
            'first_name': 'System',
            'last_name': 'Administrator',
            'is_superuser': True,
            'is_staff': True
        }
    )

    if created:
        admin_user.set_password('admin123')
        admin_user.save()

    StaffProfile.objects.get_or_create(user=admin_user, defaults={'role':
'admin'})

class SMSSettings(models.Model):
    """Global and per-service SMS configuration"""

    is_global = models.BooleanField(default=True)

    sms_enabled = models.BooleanField(default=True)

```

```

notification_threshold = models.PositiveIntegerField(default = 5)

service = models.OneToOneField(Service, on_delete=models.CASCADE,
null=True, blank=True, related_name='sms_settings')

updated_at = models.DateTimeField(auto_now=True)
updated_by = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, related_name='sms_settings_updated')

class Meta:
    verbose_name = "SMS Setting"
    verbose_name_plural = "SMS Settings"

def __str__(self):
    if self.service:
        return f"SMS Settings for {self.service.name}"
    return "Global SMS Settings"

@classmethod
def get_global_settings(cls):
    """Get or create global settings"""
    settings, created = cls.objects.get_or_create(is_global=True,
defaults={'sms_enabled': True, 'notification_threshold': 5})
    return settings

@classmethod
def get_service_settings(cls, service):
    """Get settings for a specific service (falls back to global)"""
    try:
        return cls.objects.get(service=service)
    except cls.DoesNotExist:
        return cls.get_global_settings()

# =====
# GLOBAL SYSTEM SETTINGS
# =====
class SystemSettings(models.Model):
    """Global system settings"""
    id = models.IntegerField(primary_key=True, default=1, editable=False)

```

```

# Auto-open and auto-shutdown settings
auto_schedule_enabled = models.BooleanField(default=True)
opening_time = models.TimeField(default="06:00") # 8:00 AM default
shutdown_time = models.TimeField(default="16:50") # 4:50 PM default


# Track changes
updated_at = models.DateTimeField(auto_now=True)
updated_by = models.ForeignKey(User, on_delete=models.SET_NULL,
null=True, related_name='system_settings_updated')


class Meta:
    verbose_name = "System Settings"
    verbose_name_plural = "System Settings"


def save(self, *args, **kwargs):
    self.pk = 1
    super().save(*args, **kwargs)


@classmethod
def get_settings(cls):
    settings, created = cls.objects.get_or_create(
        pk=1,
        defaults={
            'auto_schedule_enabled': True,
            'opening_time': "08:00",
            'shutdown_time': "16:50"
        }
    )
    return settings


def __str__(self):
    return f"System Settings (Open: {self.opening_time}, Close: {self.shutdown_time})"

```

Serializers(Auth Serializer)

```
from rest_framework import serializers
from django.contrib.auth.models import User
from django.contrib.auth.password_validation import validate_password
from rest_framework.validators import UniqueValidator
from .models import StaffProfile

class UserSerializer(serializers.ModelSerializer):
    assigned_service = serializers.SerializerMethodField()

    class Meta:
        model = User
        field = ['id', 'username', 'is_staff', 'is_superuser',
'assigned_service']
        read_only_fields = ['id', 'is_staff']

    def get_assigned_service(self, obj):
        try:
            profile = StaffProfile.objects.get(user=obj)
            return {
                "id": profile.assigned_service.id,
                "name": profile.assigned_service.name
            } if profile.assigned_service else None
        except StaffProfile.DoesNotExist:
            return None

class LoginSerializer(serializers.Serializer):
    username = serializers.CharField(required=True,
error_messages={'required': 'Username is required', 'blank': 'Username
cannot be empty'})
    password = serializers.CharField(required=True, write_only = True,
style={'input_type': 'password'},
                                error_messages={'required': 'Password
is required', 'blank': 'Password cannot be empty'})

class RegisterStaffSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, required=True,
validators=[validate_password])
    password2 = serializers.CharField(write_only=True, required=True)
    service_id = serializers.IntegerField(required=True, write_only=True)
```

```

class Meta:
    model = User
    fields = ['username', 'password', 'password2', 'service_id']

    def validate(self, attrs):
        if attrs['password'] != attrs['password2']:
            raise serializers.ValidationError({"password": "Password
fields didn't match."})

        from .models import Service
        service_id = attrs.get('service_id')
        if not Service.objects.filter(id=service_id).exists():
            raise serializers.ValidationError({"service_id": "Service does
not exist"})

        return attrs

    def create(self, validated_data):
        service_id = validated_data.pop('service_id')
        validated_data.pop('password2')

        user = User.objects.create(username=validated_data['username'],
is_staff=True)
        user.set_password(validated_data['password'])
        user.save()

        from .models import StaffProfile, Service
        service = Service.objects.get(id=service_id)
        StaffProfile.objects.create(user=user, assigned_service=service,
role='staff', can_manage_queue=True)

        return user

class ChangePasswordSerializer(serializers.Serializer):
    old_password = serializers.CharField(required=True)
    new_password = serializers.CharField(required=True,
validators=[validate_password])
    confirm_password = serializers.CharField(required=True)

```

```

def validate(self, attrs):
    if attrs['new_password'] != attrs['confirm_password']:
        raise serializers.ValidationError({"new_password": "New
passwords don't match."})
    return attrs

class CreateAdminSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, required=True,
validators=[validate_password])
    password2 = serializers.CharField(write_only=True, required=True)

    class Meta:
        model = User
        fields = ['username', 'password', 'password2']

    def validate(self, attrs):
        if attrs['password'] != attrs['password2']:
            raise serializers.ValidationError({"password": "Password
fields didn't match."})
        return attrs

    def create(self, validated_data):
        user = User.objects.create(
            username=validated_data['username'],
            is_staff=True,
            is_superuser=True
        )
        user.set_password(validated_data['password'])
        user.save()

        from .models import StaffProfile
        StaffProfile.objects.create(
            user=user,
            role='admin',
            can_manage_queue=True
        )

        return user

```

```
class AdminResetStaffPasswordSerializer(serializers.Serializer):
    user_id = serializers.IntegerField(required=True)
    new_password = serializers.CharField(required=True,
validators=[validate_password])
    confirm_password = serializers.CharField(required=True)

    def validate(self, attrs):
        if attrs["new_password"] != attrs["confirm_password"]:
            raise serializers.ValidationError({"confirm_password":
"Passwords do not match."})

        try:
            user = User.objects.get(id=attrs["user_id"])
        except User.DoesNotExist:
            raise serializers.ValidationError({"user_id": "User not
found."})

        if user.is_superuser:
            raise serializers.ValidationError({"user_id": "This endpoint
cannot reset another admin password."})

        attrs["target_user"] = user
        return attrs
```

Serializer(Main)

```
# queueing/serializers.py
from rest_framework import serializers
from .models import Service, ServiceWindow, Ticket, SystemSettings
from django.contrib.auth.models import User

class ServiceWindowSerializer(serializers.ModelSerializer):
    service_name = serializers.CharField(source='service.name',
read_only=True)
    is_available = serializers.BooleanField(read_only=True)
    is_in_use = serializers.BooleanField(read_only=True)
    current_staff_name =
serializers.CharField(source='current_staff.username', read_only=True,
allow_null=True)

    class Meta:
        model = ServiceWindow
        fields = [
            'id', 'service', 'service_name', 'window_number', 'name',
            'status', 'description', 'current_staff',
'current_staff_name',
            'is_available', 'is_in_use', 'created_at', 'updated_at'
        ]
        read_only_fields = ['id', 'created_at', 'updated_at']

class ServiceSerializer(serializers.ModelSerializer):
    windows_count = serializers.IntegerField(read_only=True)
    waiting_count = serializers.IntegerField(read_only=True)
    currently_serving = serializers.SerializerMethodField()

    class Meta:
        model = Service
        fields = [
            'id',
            'name',
            'description',
            'prefix',
            'is_active',
            'average_service_time',
            'windows_count',
```

```

        'waiting_count',
        'currently_serving',
        'created_at',
        'updated_at',
    ]
    read_only_fields = [
        'id',
        'windows_count',
        'waiting_count',
        'currently_serving',
        'created_at',
        'updated_at',
    ]
    extra_kwargs = {
        'prefix': {
            'allow_blank': False,
            'required': True,
            'trim_whitespace': True
        }
    }

    def validate_prefix(self, value):
        #Validate that prefix is unique
        # Check if prefix already exists
        queryset = Service.objects.filter(prefix=value)

        # Exclude current instance if updating
        if self.instance:
            queryset = queryset.exclude(pk=self.instance.pk)

        if queryset.exists():
            raise serializers.ValidationError('Prefix already exists.
Please use a unique prefix.')

        return value

    def get_currently_serving(self, obj):
        serving = obj.currently_serving
        return serving.display_number if serving else None

```

```

class TicketSerializer(serializers.ModelSerializer):
    display_number = serializers.CharField(read_only=True)
    is_today = serializers.BooleanField(read_only=True)
    people_ahead = serializers.IntegerField(read_only=True)
    wait_time_minutes = serializers.IntegerField(read_only=True)
    service_name = serializers.CharField(source='service.name',
read_only=True)
    assigned_window_info = serializers.SerializerMethodField()

    class Meta:
        model = Ticket
        fields = [
            'ticket_id', 'queue_number', 'display_number',
            'service', 'service_name', 'status', 'ticket_date',
            'assigned_window', 'assigned_window_info',
            'called_by', 'served_by', 'called_at', 'served_at',
            'created_at', 'is_today', 'people_ahead', 'wait_time_minutes',
            'notes'
        ]
        read_only_fields = [
            'ticket_id', 'queue_number', 'display_number',
            'ticket_date', 'created_at', 'is_today',
            'people_ahead', 'wait_time_minutes'
        ]

    def get_assigned_window_info(self, obj):
        if obj.assigned_window:
            return {
                'id': obj.assigned_window.id,
                'name': obj.assigned_window.name,
                'window_number': obj.assigned_window.window_number
            }
        return None

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'first_name', 'last_name',
'is_staff', 'is_active', 'assigned_window']
        read_only_fields = ['id', 'is_staff']

```

```

class SystemSettingsSerializer(serializers.ModelSerializer):
    class Meta:
        model = SystemSettings
        fields = [
            'auto_schedule_enabled',
            'opening_time',
            'shutdown_time',
            'updated_at'
        ]
        read_only_fields = ['updated_at']

```

Views

```

from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import AllowAny, IsAuthenticated,
IsAdminUser
from django.utils import timezone
from .models import Service, Ticket, SMSSettings, SystemSettings
from .serializers import ServiceSerializer, TicketSerializer,
SystemSettingsSerializer, ServiceWindowSerializer
from drf_spectacular.utils import extend_schema, OpenApiParameter,
OpenApiResponse, OpenApiExample
from drf_spectacular.types import OpenApiTypes
from .websocket_utils import send_dashboard_update, send_service_update,
send_ticket_update

@extend_schema(
    summary="Public Service List",
    description="Get list of active services for ticket generation.
Optional status filter.",
    tags=['Public Endpoints'],
    parameters=[
        OpenApiParameter(
            name='status',

```

```

        description='Filter by service status (active/inactive)',
        required=False,
        type=str,
        enum=['active', 'inactive', 'all']
    )
]
)
@api_view(['GET'])
@permission_classes([AllowAny])
def public_service_list(request):
    """
    Public: Get services for ticket generation.
    Can filter by status: ?status=active (default), ?status=inactive,
    ?status=all
    """
    status_filter = request.query_params.get('status', 'active')

    services = list(Service.objects.all().order_by('name'))

    if status_filter == 'active':
        services = [service for service in services if
service.can_accept_tickets()]
    elif status_filter == 'inactive':
        services = [service for service in services if not
service.can_accept_tickets()]
    # 'all' shows all services

    serializer = ServiceSerializer(services, many=True)
    return Response({'success': True, 'count': len(services), 'services':
serializer.data, 'filter': status_filter})

@extend_schema(
    summary="Generate Ticket",
    description="Generate a new ticket for a specific service",
    tags=['Public Endpoints']
)
@api_view(['POST'])
@permission_classes([AllowAny])
def generate_ticket(request):

```

```

Ticket.cancel_old_tickets()
service_id = request.data.get('service_id')

if not service_id:
    return Response({'success': False, 'message': 'service_id is
required'}, status=status.HTTP_400_BAD_REQUEST)

try:
    service = Service.objects.get(id=service_id)
except Service.DoesNotExist:
    return Response({'success': False, 'message': 'Service not
found'}, status=status.HTTP_404_NOT_FOUND)

if not service.is_active:
    return Response({'success': False, 'message': 'This service is
currently unavailable.'}, status=status.HTTP_400_BAD_REQUEST)

ticket = Ticket.objects.create(service=service)

send_dashboard_update()
send_service_update(service.id)

windows = ServiceWindowSerializer(service.windows.all(),
many=True).data # Use .all() to include inactive too

response_data = {
    'success': True,
    'message': f'Ticket {ticket.display_number} generated for
{service.name}',
    'ticket': {
        'ticket_id': str(ticket.ticket_id),
        'service': service.name,
        'service_id': service.id,
        'queue_number': ticket.queue_number,
        'display_number': ticket.display_number,
        'ticket_date': str(ticket.ticket_date),
        'is_today': ticket.is_today,
        'status': ticket.status,
        'people_ahead': ticket.people_ahead,
        'wait_time_minutes': ticket.wait_time_minutes,

```

```

        'created_at': ticket.created_at.isoformat(),
    },
    'windows': windows
}

return Response(response_data, status=status.HTTP_201_CREATED)

@extend_schema(
    summary="Generate Ticket",
    description="Generate a new ticket for a specific service",
    tags=['Public Endpoints']
)
@api_view(['GET'])
@permission_classes([AllowAny])
def ticket_status(request, ticket_id):
    # Public: Check ticket status
    Ticket.cancel_old_tickets()
    try:
        ticket = Ticket.objects.get(ticket_id=ticket_id)
        serializer = TicketSerializer(ticket)

        # Get queue info
        service_today = Ticket.objects.filter(
            service=ticket.service,
            ticket_date=ticket.ticket_date
        ).order_by('queue_number')

        current_serving = service_today.filter(status='serving').first()

        return Response({
            'success': True,
            'ticket': serializer.data,
            'queue_info': {
                'position': ticket.people_ahead + 1,
                'total_in_queue':
service_today.filter(status__in=['waiting', 'notified']).count(),
                'currently_serving': current_serving.display_number if
current_serving else None,
                'estimated_wait_minutes': ticket.wait_time_minutes
            }
        })

```

```

        }
    })
    except Ticket.DoesNotExist:
        return Response({'success': False, 'message': 'Ticket not found'},
            status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    summary="Public Dashboard Status",
    description="Get dashboard status for display screens. Shows all
currently serving tickets per service.",
    tags=['Public Endpoints']
)
@api_view(['GET'])
@permission_classes([AllowAny])
def dashboard_status(request):
    #Public: Get dashboard status for display screens (TV Monitor)
    #Shows all currently serving tickets across all windows
    Ticket.cancel_old_tickets()
    services = Service.objects.filter(is_active=True).order_by('name')

    service_data = []
    for service in services:
        today = timezone.now().date()
        tickets_today = service.tickets.filter(ticket_date=today)

        # Get ALL currently serving tickets with their window info
        serving_tickets =
tickets_today.filter(status='serving').select_related('assigned_window').o
rder_by('assigned_window__window_number')

        # Format serving tickets with window details
        currently_serving_list = []
        for ticket in serving_tickets:
            if ticket.assigned_window:
                currently_serving_list.append({'ticket_number':
ticket.display_number, 'window_name':
ticket.assigned_window.name, 'window_number':
ticket.assigned_window.window_number})
            else:

```

```

        currently_serving_list.append({'ticket_number':
ticket.display_number, 'window_name': 'Unknown Window', 'window_number':
None})

    # Get waiting tickets
    waiting_tickets = tickets_today.filter(
        status__in=['waiting', 'notified']
    ).order_by('queue_number')

    # Calculate estimated wait time for next ticket
    next_wait_time = None
    if waiting_tickets.exists():
        next_wait_time = waiting_tickets.first().wait_time_minutes

    service_data.append({
        'id': service.id,
        'name': service.name,
        'prefix': service.prefix,
        'currently_serving': currently_serving_list,
        'serving_count': len(currently_serving_list),
        'next_in_line': waiting_tickets.first().display_number if
waiting_tickets.exists() else None,
        'waiting_count': waiting_tickets.count(),
        'estimated_next_wait': next_wait_time,
        'average_wait_time': service.average_service_time *
waiting_tickets.count()
    })

    # Also get overall stats for the TV display
    total_waiting = sum(s['waiting_count'] for s in service_data)
    total_serving = sum(s['serving_count'] for s in service_data)

    return Response({
        'success': True,
        'timestamp': timezone.now().isoformat(),
        'summary': {
            'total_waiting': total_waiting,
            'total_serving': total_serving,
        },
        'services': service_data
    })

```

```

    })

@extend_schema(
    summary="Opt-in for SMS notifications",
    description='Store phone number for ticket notifications. Expects
JSON: {"phone": "09171234567"}',
    tags=['Public Endpoints'],
    request={
        "type": "object",
        "properties": {
            "phone": {"type": "string", "description": "Phone number in
digits, e.g. 09171234567"}
        },
        "required": ["phone"]
    },
    responses={
        200: {
            "description": "SMS notifications enabled",
            "type": "object",
            "properties": {"success": {"type": "boolean"}, "message":
{"type": "string"}}
        },
        400: {
            "description": "Bad request (phone missing/invalid)",
            "type": "object",
            "properties": {"success": {"type": "boolean"}, "message":
{"type": "string"}}
        },
        404: {
            "description": "Ticket not found",
            "type": "object",
            "properties": {"success": {"type": "boolean"}, "message":
{"type": "string"}}
        }
    }
)

@api_view(['POST'])
@permission_classes([AllowAny])
def sms_opt_in(request, ticket_id):

```

```

"""
Store phone number for ticket notifications.
Expects JSON: {"phone": "09171234567"}
"""
try:
    ticket = Ticket.objects.get(ticket_id=ticket_id)
except Ticket.DoesNotExist:
    return Response({'success': False, 'message': 'Ticket not found'},
status=404)

phone = request.data.get('phone')
if not phone:
    return Response({'success': False, 'message': 'Phone number
required'}, status=400)

# Basic validation
cleaned = ''.join(filter(str.isdigit, phone))
if len(cleaned) < 10:
    return Response({'success': False, 'message': 'Invalid phone
number'}, status=400)

ticket.sms_phone = cleaned
ticket.sms_sent = False
ticket.save(update_fields=['sms_phone', 'sms_sent'])

return Response({'success': True, 'message': 'SMS notifications
enabled'})

@extend_schema(
    tags=['Admin'],
    summary='Get SMS settings',
    description='Get global and per-service SMS notification settings'
)
@api_view(['GET'])
@permission_classes([IsAdminUser])
def get_sms_settings(request):
    """Get all SMS settings (global + per service)"""

    global_settings = SMSSettings.get_global_settings()

```

```

# Get per-service settings
services = Service.objects.all()
per_service = []

for service in services:
    try:
        settings = SMSSettings.objects.get(service=service)
        per_service.append({
            'service_id': service.id,
            'service_name': service.name,
            'sms_enabled': settings.sms_enabled,
            'threshold': settings.notification_threshold,
        })
    except SMSSettings.DoesNotExist:
        # Using global defaults
        per_service.append({
            'service_id': service.id,
            'service_name': service.name,
            'sms_enabled': global_settings.sms_enabled,
            'threshold': global_settings.notification_threshold,
            'using_global': True
        })

return Response({
    'success': True,
    'global': {
        'sms_enabled': global_settings.sms_enabled,
        'threshold': global_settings.notification_threshold,
    },
    'per_service': per_service
})

@extend_schema(
    tags=['Admin'],
    summary='Update global SMS settings',
    description='Update global SMS notification settings'
)

@api_view(['PUT', 'PATCH'])
@permission_classes([IsAdminUser])

```

```

def update_global_sms_settings(request):
    """Update global SMS settings"""
    settings = SMSSettings.get_global_settings()

    sms_enabled = request.data.get('sms_enabled')
    threshold = request.data.get('threshold')

    if sms_enabled is not None:
        settings.sms_enabled = sms_enabled
    if threshold is not None:
        settings.notification_threshold = threshold

    settings.updated_by = request.user
    settings.save()

    return Response({
        'success': True,
        'message': 'Global SMS settings updated',
        'settings': {
            'sms_enabled': settings.sms_enabled,
            'threshold': settings.notification_threshold,
        }
    })

@extend_schema(
    tags=['Admin'],
    summary='Update service SMS settings',
    description='Update SMS settings for a specific service'
)
@api_view(['PUT', 'PATCH'])
@permission_classes([IsAdminUser])
def update_service_sms_settings(request, service_id):
    """Update SMS settings for a specific service"""
    try:
        service = Service.objects.get(id = service_id)
    except Service.DoesNotExist:
        return Response({'success': False, 'message': 'Service not found'},
            status=404)

```

```

# Get or create settings for this service
settings, created = SMSSettings.objects.get_or_create(
    service=service,
    defaults={
        'sms_enabled': True,
        'notification_threshold': 5,
        'is_global': False
    }
)

sms_enabled = request.data.get('sms_enabled')
threshold = request.data.get('threshold')

if sms_enabled is not None:
    settings.sms_enabled = sms_enabled
if threshold is not None:
    settings.notification_threshold = threshold

settings.updated_by = request.user
settings.save()

return Response({
    'success': True,
    'message': f'SMS settings updated for {service.name}',
    'settings': {
        'service_id': service.id,
        'service_name': service.name,
        'sms_enabled': settings.sms_enabled,
        'threshold': settings.notification_threshold,
    }
})

))

@extend_schema(
    tags=['Admin'],
    summary='Reset service to global settings',
    description='Delete service-specific settings and revert to global'
)

@api_view(['DELETE'])
@permission_classes([IsAdminUser])
def reset_service_sms_settings(request, service_id):
    """Delete service-specific settings (revert to global)"""

```

```

    try:
        settings = SMSSettings.objects.get(service_id=service_id)
        settings.delete()
        return Response({'success': True, 'message': 'Service reverted to
global SMS settings'})
    except SMSSettings.DoesNotExist:
        return Response({'success': True, 'message': 'Service already
using global settings'})

@extend_schema(
    tags=['Admin'],
    summary='Get system settings',
    description='Get global system settings including auto-shutdown time'
)
@api_view(['GET'])
@permission_classes([IsAdminUser])
def get_system_settings(request):
    """Get global system settings"""
    settings = SystemSettings.get_settings()
    serializer = SystemSettingsSerializer(settings)

    return Response({'success': True, 'settings': serializer.data})

@extend_schema(
    tags=['Admin'],
    summary='Update system settings',
    description='Update global system settings including auto-shutdown
time'
)
@api_view(['PUT', 'PATCH'])
@permission_classes([IsAdminUser])
def update_system_settings(request):
    """Update global system settings"""
    settings = SystemSettings.get_settings()

    serializer = SystemSettingsSerializer(settings, data=request.data,
partial=True)

```

```

    if serializer.is_valid():
        serializer.save(updated_by=request.user)

        # Log the change
        print(f"System settings updated by {request.user.username}: New
shutdown time = {settings.shutdown_time}")

        return Response({'success': True, 'message': 'System settings
updated successfully', 'settings': serializer.data})

    return Response({'success': False, 'message': 'Invalid data',
'errors': serializer.errors}, status=400)

```

Views(Analytics)

```

from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework.permissions import IsAdminUser
from django.utils import timezone
from django.db.models import Count, Avg, Q
from datetime import timedelta
from .models import Service, Ticket
from drf_spectacular.utils import extend_schema, OpenApiParameter,
OpenApiResponse, OpenApiExample
from drf_spectacular.types import OpenApiTypes
from datetime import datetime, timedelta
import csv
from django.http import HttpResponse

@extend_schema(
    tags=['Admin Analytics'],
    summary='Get admin dashboard analytics',
    description='Returns comprehensive analytics including total tickets,
average wait times, peak hours, and recent activity.',
    responses={
        200: OpenApiResponse(
            response=OpenApiTypes.OBJECT,
            description="Analytics data retrieved successfully",
            examples=[

```

```
OpenApiExample(  
    'Success Response',  
    value={  
        'success': True,  
        'analytics': {  
            'date': '2024-03-17',  
            'summary': {  
                'total_tickets_issued': 150,  
                'total_tickets_served': 120,  
                'completion_rate': 80.0,  
                'currently_waiting': 25,  
                'currently_serving': 5  
            },  
            'services': [  
                {  
                    'service_id': 1,  
                    'service_name': 'Cashier',  
                    'prefix': 'C',  
                    'tickets_today': 50,  
                    'served_today': 40,  
                    'waiting_now': 8,  
                    'serving_now': 2,  
                    'average_wait_minutes': 4.5,  
                    'estimated_total_wait': 32  
                }  
            ],  
            'peak_hours': [  
                {'hour': '10:00', 'tickets_issued': 25},  
                {'hour': '11:00', 'tickets_issued': 22}  
            ],  
            'recent_activity': [  
                {  
                    'ticket': 'C045',  
                    'service': 'Cashier',  
                    'served_at': '14:30:25',  
                    'wait_time': 3.5  
                }  
            ],  
            'timestamp': '2024-03-17T14:30:45.123Z'  
        }  
    )
```

```

        }
    )
]
),
401: OpenApiResponse(description="✗ Unauthorized - Admin access
required"),
403: OpenApiResponse(description="✗ Forbidden - Insufficient
permissions")
}
)
@api_view(['GET'])
@permission_classes([IsAdminUser])
def admin_analytics(request):
    """Get analytics data for admin dashboard with date filtering"""
    today = timezone.now().date()

    date_param = request.GET.get("date")
    start_date_param = request.GET.get("start_date")
    end_date_param = request.GET.get("end_date")

    try:
        if date_param:
            selected_date = datetime.strptime(date_param,
"%Y-%m-%d").date()
            tickets = Ticket.objects.filter(ticket_date=selected_date)
            date_label = str(selected_date)

        elif start_date_param and end_date_param:
            start_date = datetime.strptime(start_date_param,
"%Y-%m-%d").date()
            end_date = datetime.strptime(end_date_param,
"%Y-%m-%d").date()

            if start_date > end_date:
                return Response(
                    {"success": False, "message": "start_date cannot be
greater than end_date"},
                    status=400
                )
    
```

```

        tickets =
Ticket.objects.filter(ticket_date__range=[start_date, end_date])
        date_label = f"{start_date} to {end_date}"

    else:
        tickets = Ticket.objects.filter(ticket_date=today)
        date_label = str(today)

except ValueError:
    return Response(
        {"success": False, "message": "Invalid date format. Use
YYYY-MM-DD."},
        status=400
    )

total_tickets = tickets.count()
served_total = tickets.filter(status='served').count()
waiting_total = tickets.filter(status='waiting').count()
serving_total = tickets.filter(status='serving').count()

services_data = []
for service in Service.objects.all():
    service_tickets = tickets.filter(service=service)
    served_tickets = service_tickets.filter(status='served')

    wait_times = []
    for ticket in served_tickets:
        if ticket.called_at and ticket.created_at:
            wait_time = (ticket.called_at -
ticket.created_at).total_seconds() / 60
            wait_times.append(wait_time)

    avg_wait = sum(wait_times) / len(wait_times) if wait_times else 0

    services_data.append({
        'service_id': service.id,
        'service_name': service.name,
        'prefix': service.prefix,
        'tickets': service_tickets.count(),
        'served': served_tickets.count(),

```

```

        'waiting': service_tickets.filter(status='waiting').count(),
        'serving': service_tickets.filter(status='serving').count(),
        'average_wait_minutes': round(avg_wait, 1),
        'estimated_total_wait':
service_tickets.filter(status='waiting').count() *
service.average_service_time
    })

    peak_hours = []
    for hour in range(0, 24):
        hour_count = tickets.filter(created_at__hour=hour).count()
        if hour_count > 0:
            peak_hours.append({
                'hour': f'{hour:02d}:00',
                'tickets_issued': hour_count
            })

    peak_hours.sort(key=lambda x: x['tickets_issued'], reverse=True)

    recent_served =
tickets.filter(status='served').order_by('-served_at')[:10]
    recent_activity = []

    for ticket in recent_served:
        recent_activity.append({
            'ticket': ticket.display_number,
            'service': ticket.service.name,
            'served_at': ticket.served_at.strftime('%Y-%m-%d %H:%M:%S') if
ticket.served_at else None,
            'wait_time': round(
                (ticket.called_at - ticket.created_at).total_seconds() /
60, 1
            ) if ticket.called_at and ticket.created_at else 0
        })

    return Response({
        'success': True,
        'analytics': {
            'date': date_label,
            'summary': {

```

```

        'total_tickets_issued': total_tickets,
        'total_tickets_served': served_total,
        'completion_rate': round((served_total / total_tickets *
100), 1) if total_tickets > 0 else 0,
        'currently_waiting': waiting_total,
        'currently_serving': serving_total
    },
    'services': services_data,
    'peak_hours': peak_hours[:5],
    'recent_activity': recent_activity,
    'timestamp': timezone.now().isoformat()
}
}))

```

```

@extend_schema(
    tags=['Admin Analytics'],
    summary='Get service-specific analytics',
    description='Returns detailed analytics for a specific service
including daily stats and window performance.',
    parameters=[
        OpenApiParameter(
            name='service_id',
            type=OpenApiTypes.INT,
            location=OpenApiParameter.PATH,
            description='ID of the service to get analytics for',
            required=True
        )
    ],
    responses={
        200: OpenApiResponse(
            response=OpenApiTypes.OBJECT,
            description="✅ Service analytics retrieved successfully",
            examples=[
                OpenApiExample(
                    'Success Response',
                    value={
                        'success': True,
                        'analytics': {
                            'service': {
                                'id': 1,

```

```

        'name': 'Cashier',
        'prefix': 'C'
    },
    'daily_stats': [
        {
            'date': '2024-03-17',
            'total': 45,
            'served': 38,
            'cancelled': 2
        }
    ],
    'window_performance': [
        {
            'window_id': 6,
            'window_name': 'Cashier Window 6',
            'window_number': 6,
            'tickets_served': 15,
            'currently_serving': True
        },
        {
            'window_id': 7,
            'window_name': 'Cashier Window 7',
            'window_number': 7,
            'tickets_served': 23,
            'currently_serving': False
        }
    ],
    'average_service_time': 4,
    'total_waiting_today': 5
    }
    )
]
),
401: OpenApiResponse(description="✗ Unauthorized - Admin access
required"),
403: OpenApiResponse(description="✗ Forbidden - Insufficient
permissions"),
404: OpenApiResponse(description="✗ Service not found")
}

```

```

)
@api_view(['GET'])
@permission_classes([IsAdminUser])
def service_analytics(request, service_id):
    """Get detailed analytics for a specific service"""
    try:
        service = Service.objects.get(id=service_id)
    except Service.DoesNotExist:
        return Response({'success': False, 'message': 'Service not found'}, status=404)

    today = timezone.now().date()
    week_ago = today - timedelta(days=7)

    # Last 7 days data
    daily_stats = []
    for i in range(7):
        day = today - timedelta(days=i)
        day_tickets = Ticket.objects.filter(
            service=service,
            ticket_date=day
        )

        daily_stats.append({
            'date': day,
            'total': day_tickets.count(),
            'served': day_tickets.filter(status='served').count(),
            'cancelled': day_tickets.filter(status='cancelled').count()
        })

    # Window performance with window_number added
    window_stats = []
    for window in service.windows.all():
        window_tickets = Ticket.objects.filter(
            assigned_window=window,
            ticket_date=today
        )

        window_stats.append({
            'window_id': window.id,

```

```

        'window_name': window.name,
        'window_number': window.window_number,
        'tickets_served':
window_tickets.filter(status='served').count(),
        'currently_serving':
window_tickets.filter(status='serving').exists()
    })

    return Response({
        'success': True,
        'analytics': {
            'service': {
                'id': service.id,
                'name': service.name,
                'prefix': service.prefix
            },
            'daily_stats': daily_stats,
            'window_performance': window_stats,
            'average_service_time': service.average_service_time,
            'total_waiting_today': service.waiting_count
        }
    })

@extend_schema(
    tags=['Admin Analytics'],
    summary='Get window analytics',
    description='Returns per-window analytics for a specific service',
    parameters=[
        OpenApiParameter(
            name='service_id',
            type=OpenApiTypes.INT,
            location=OpenApiParameter.PATH,
            description='ID of the service',
            required=True
        ),
        OpenApiParameter(
            name='start_date',
            type=str,

```

```

        location=OpenApiParameter.QUERY,
        description='Filter by start date (YYYY-MM-DD)',
        required=False
    ),
    OpenApiParameter(
        name='end_date',
        type=str,
        location=OpenApiParameter.QUERY,
        description='Filter by end date (YYYY-MM-DD)',
        required=False
    ),
]
)
@api_view(['GET'])
@permission_classes([IsAdminUser])
def window_analytics(request, service_id):
    """Get detailed per-window analytics for a specific service"""
    try:
        service = Service.objects.get(id=service_id)
    except Service.DoesNotExist:
        return Response({'success': False, 'message': 'Service not found'}, status=404)

    # Date filtering
    start_date_param = request.GET.get("start_date")
    end_date_param = request.GET.get("end_date")

    try:
        if start_date_param and end_date_param:
            start_date = datetime.strptime(start_date_param,
"%Y-%m-%d").date()
            end_date = datetime.strptime(end_date_param,
"%Y-%m-%d").date()
            tickets = Ticket.objects.filter(
                ticket_date__range=[start_date, end_date],
                service=service
            )
            date_label = f"{start_date} to {end_date}"
        elif start_date_param:

```

```

        start_date = datetime.strptime(start_date_param,
"%Y-%m-%d").date()
        tickets = Ticket.objects.filter(ticket_date=start_date,
service=service)
        date_label = str(start_date)
    elif end_date_param:
        end_date = datetime.strptime(end_date_param,
"%Y-%m-%d").date()
        tickets = Ticket.objects.filter(ticket_date=end_date,
service=service)
        date_label = str(end_date)
    else:
        today = timezone.now().date()
        tickets = Ticket.objects.filter(ticket_date=today,
service=service)
        date_label = str(today)
    except ValueError:
        return Response(
            {"success": False, "message": "Invalid date format. Use
YYYY-MM-DD."},
            status=400
        )

window_stats = []
for window in service.windows.all():
    window_tickets = tickets.filter(assigned_window=window)
    served_tickets = window_tickets.filter(status='served')

    # Calculate average wait time for this window
    wait_times = []
    for ticket in served_tickets:
        if ticket.called_at and ticket.created_at:
            wait_time = (ticket.called_at -
ticket.created_at).total_seconds() / 60
            wait_times.append(wait_time)

    avg_wait = sum(wait_times) / len(wait_times) if wait_times else 0

    # Daily breakdown for this window
    daily_breakdown = []

```

```

        if not start_date_param and not end_date_param:
            # Just show today
            daily_breakdown.append({
                'date': str(today),
                'served':
served_tickets.filter(ticket_date=today).count(),
                'cancelled': window_tickets.filter(status='cancelled',
ticket_date=today).count(),
                'avg_wait_minutes': round(avg_wait, 1)
            })
        else:
            # Show breakdown by day within range
            date_range = (end_date - start_date).days + 1 if
start_date_param and end_date_param else 1
            for i in range(date_range):
                day = (start_date if start_date_param else today) +
timedelta(days=i)
                day_tickets = window_tickets.filter(ticket_date=day)
                day_served = day_tickets.filter(status='served')

                day_wait_times = []
                for ticket in day_served:
                    if ticket.called_at and ticket.created_at:
                        day_wait_times.append((ticket.called_at -
ticket.created_at).total_seconds() / 60)

                day_avg_wait = sum(day_wait_times) / len(day_wait_times)
if day_wait_times else 0

                daily_breakdown.append({
                    'date': str(day),
                    'served': day_served.count(),
                    'cancelled':
day_tickets.filter(status='cancelled').count(),
                    'avg_wait_minutes': round(day_avg_wait, 1)
                })

            window_stats.append({
                'window_id': window.id,
                'window_name': window.name,

```

```

        'window_number': window.window_number,
        'status': window.status,
        'tickets_served': served_tickets.count(),
        'tickets_served_total':
window_tickets.filter(status='served').count(),
        'tickets_cancelled':
window_tickets.filter(status='cancelled').count(),
        'avg_wait_minutes': round(avg_wait, 1),
        'currently_serving':
window_tickets.filter(status='serving').exists(),
        'daily_breakdown': daily_breakdown
    })

    # Calculate totals
    total_served = sum(w['tickets_served'] for w in window_stats)
    total_cancelled = sum(w['tickets_cancelled'] for w in window_stats)
    overall_avg_wait = sum(w['avg_wait_minutes'] for w in window_stats) /
len(window_stats) if window_stats else 0

    return Response({
        'success': True,
        'service': {
            'id': service.id,
            'name': service.name,
            'prefix': service.prefix
        },
        'date_range': date_label,
        'summary': {
            'total_windows': len(window_stats),
            'active_windows': sum(1 for w in window_stats if w['status']
== 'active'),
            'total_served': total_served,
            'total_cancelled': total_cancelled,
            'overall_avg_wait_minutes': round(overall_avg_wait, 1)
        },
        'windows': window_stats
    })

@extend_schema(

```

```

tags=['Admin Analytics'],
summary='Export analytics to CSV',
description='Export ticket data as CSV file',
parameters=[
    OpenApiParameter(
        name='start_date',
        type=str,
        location=OpenApiParameter.QUERY,
        description='Start date (YYYY-MM-DD)',
        required=False
    ),
    OpenApiParameter(
        name='end_date',
        type=str,
        location=OpenApiParameter.QUERY,
        description='End date (YYYY-MM-DD)',
        required=False
    ),
    OpenApiParameter(
        name='service_id',
        type=OpenApiTypes.INT,
        location=OpenApiParameter.QUERY,
        description='Filter by service ID',
        required=False
    ),
    OpenApiParameter(
        name='format',
        type=str,
        location=OpenApiParameter.QUERY,
        description='Export format (csv or excel)',
        required=False
    ),
]
)
@api_view(['GET'])
@permission_classes([IsAdminUser])
def export_analytics_csv(request):
    """Export ticket data as CSV file"""

    start_date_param = request.GET.get("start_date")

```

```

end_date_param = request.GET.get("end_date")
service_id = request.GET.get("service_id")

tickets = Ticket.objects.select_related('service', 'assigned_window',
'called_by', 'served_by')

    try:
        if start_date_param and end_date_param:
            start_date = datetime.strptime(start_date_param,
"%Y-%m-%d").date()
            end_date = datetime.strptime(end_date_param,
"%Y-%m-%d").date()

            if start_date > end_date:
                return Response(
                    {"success": False, "message": "start_date cannot be
later than end_date."},
                    status=400
                )

            tickets = tickets.filter(ticket_date__range=[start_date,
end_date])

        elif start_date_param:
            start_date = datetime.strptime(start_date_param,
"%Y-%m-%d").date()
            tickets = tickets.filter(ticket_date__gte=start_date)

        elif end_date_param:
            end_date = datetime.strptime(end_date_param,
"%Y-%m-%d").date()
            tickets = tickets.filter(ticket_date__lte=end_date)

        else:
            # default to today if no filters are provided
            today = timezone.now().date()
            tickets = tickets.filter(ticket_date=today)

    except ValueError:
        return Response(

```

```

        {"success": False, "message": "Invalid date format. Use
YYYY-MM-DD."},
        status=400
    )

    if service_id:
        tickets = tickets.filter(service_id=service_id)

    response = HttpResponse(content_type='text/csv')
    filename =
f"queueick_export_{timezone.now().strftime('%Y%m%d_%H%M%S')}.csv"
    response['Content-Disposition'] = f'attachment; filename="{filename}"'
    response.write('\uFEFF')

    writer = csv.writer(response)

    writer.writerow([
        'Ticket Number', 'Service', 'Window', 'Status',
        'Created At', 'Called At', 'Served At',
        'Wait Time (minutes)', 'Ticket Date'
    ])

    for ticket in tickets.order_by('ticket_date', 'created_at'):
        wait_time = 0
        if ticket.called_at and ticket.created_at:
            wait_time = round((ticket.called_at -
ticket.created_at).total_seconds() / 60, 1)

        writer.writerow([
            ticket.display_number,
            ticket.service.name if ticket.service else '',
            ticket.assigned_window.name if ticket.assigned_window else
'Not Assigned',
            ticket.status,
            ticket.created_at.strftime('%Y-%m-%d %H:%M:%S') if
ticket.created_at else '',
            ticket.called_at.strftime('%Y-%m-%d %H:%M:%S') if
ticket.called_at else '',
            ticket.served_at.strftime('%Y-%m-%d %H:%M:%S') if
ticket.served_at else '',

```

```

        wait_time,
        ticket.ticket_date.strftime('%Y-%m-%d') if ticket.ticket_date
else '',
    ])

    return response

@extend_schema(
    tags=['Admin Analytics'],
    summary='Export window performance to CSV',
    description='Export window performance data as CSV file',
    parameters=[
        OpenApiParameter(
            name='service_id',
            type=OpenApiTypes.INT,
            location=OpenApiParameter.PATH,
            description='ID of the service',
            required=True
        ),
        OpenApiParameter(
            name='start_date',
            type=str,
            location=OpenApiParameter.QUERY,
            description='Start date (YYYY-MM-DD)',
            required=False
        ),
        OpenApiParameter(
            name='end_date',
            type=str,
            location=OpenApiParameter.QUERY,
            description='End date (YYYY-MM-DD)',
            required=False
        ),
    ]
)

@api_view(['GET'])
@permission_classes([IsAdminUser])
def export_window_performance_csv(request, service_id):
    """Export window performance data as CSV"""

```

```

try:
    service = Service.objects.get(id=service_id)
except Service.DoesNotExist:
    return Response({'success': False, 'message': 'Service not
found'}, status=404)

start_date_param = request.GET.get("start_date")
end_date_param = request.GET.get("end_date")

try:
    if start_date_param and end_date_param:
        start_date = datetime.strptime(start_date_param,
"%Y-%m-%d").date()
        end_date = datetime.strptime(end_date_param,
"%Y-%m-%d").date()

        if start_date > end_date:
            return Response(
                {"success": False, "message": "start_date cannot be
later than end_date."},
                status=400
            )

        date_label = f"{start_date} to {end_date}"

    elif start_date_param:
        start_date = datetime.strptime(start_date_param,
"%Y-%m-%d").date()
        end_date = start_date
        date_label = str(start_date)

    elif end_date_param:
        end_date = datetime.strptime(end_date_param,
"%Y-%m-%d").date()
        start_date = end_date
        date_label = str(end_date)

    else:
        today = timezone.now().date()

```

```

        start_date = today
        end_date = today
        date_label = str(today)

    except ValueError:
        return Response(
            {"success": False, "message": "Invalid date format. Use
YYYY-MM-DD."},
            status=400
        )

    response = HttpResponse(content_type='text/csv')
    filename =
f"window_performance_{service.name}_{timezone.now().strftime('%Y%m%d_%H%M%S')}.csv"
    response['Content-Disposition'] = f'attachment; filename="{filename}"'
    response.write('\uFEFF')

    writer = csv.writer(response)

    writer.writerow([
        'Service',
        'Date Range',
        'Window Name',
        'Window Number',
        'Tickets Served',
        'Tickets Cancelled',
        'Avg Wait Time (minutes)'
    ])

    for window in service.windows.all():
        range_tickets = Ticket.objects.filter(
            assigned_window=window,
            ticket_date__range=[start_date, end_date]
        )

        served_tickets = range_tickets.filter(status='served')
        cancelled_tickets = range_tickets.filter(status='cancelled')

        wait_times = []

```

```

        for ticket in served_tickets:
            if ticket.called_at and ticket.created_at:
                wait_time = (ticket.called_at -
ticket.created_at).total_seconds() / 60
                wait_times.append(wait_time)

        avg_wait = sum(wait_times) / len(wait_times) if wait_times else 0

        writer.writerow([
            service.name,
            date_label,
            window.name,
            window.window_number,
            served_tickets.count(),
            cancelled_tickets.count(),
            round(avg_wait, 1)
        ])

    return response

```

Views(Auth)

```

from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status
from django.utils import timezone
from rest_framework.permissions import AllowAny, IsAuthenticated,
IsAdminUser
from drf_spectacular.utils import extend_schema, OpenApiExample,
OpenApiResponse
from drf_spectacular.types import OpenApiTypes
from django.contrib.auth import authenticate, logout
from django.contrib.auth.models import User
from .auth_serializers import (UserSerializer, LoginSerializer,
RegisterStaffSerializer, ChangePasswordSerializer, CreateAdminSerializer,
AdminResetStaffPasswordSerializer)
from .authentication import set_jwt_cookies, delete_jwt_cookies
from .serializers import ServiceWindowSerializer
from .models import Service, StaffProfile

@extend_schema(

```

```

tags=['Authentication'],
summary='User Login (JWT in HTTP-only cookies)',
description='Login user and get JWT tokens in HTTP-only cookies.',
request=LoginSerializer,
responses={
    200: OpenApiResponse(
        description="Login successful",
        examples=[
            OpenApiExample(
                'Success - Admin',
                value={
                    'success': True,
                    'role': 'admin',
                    'user': {
                        'id': 1,
                        'username': 'admin',
                        'is_staff': True,
                        'is_superuser': True
                    }
                },
            ),
            OpenApiExample(
                'Success - Staff',
                value={
                    'success': True,
                    'role': 'staff',
                    'user': {
                        'id': 2,
                        'username': 'cashier_staff'
                    },
                    'service': {
                        'id': 1,
                        'name': 'Cashier',
                        'windows': [
                            {
                                'id': 6,
                                'window_number': 6,
                                'status': 'active'
                            }
                        ]
                    }
                }
            )
        ]
    )
}

```

```

        }
    }
)
]
),
400: OpenApiResponse(description="Invalid input"),
401: OpenApiResponse(description="Invalid credentials or inactive
account"),
500: OpenApiResponse(description="Server error")
}
)
@api_view(['POST'])
@permission_classes([AllowAny])
def login_view(request):
    try:
        serializer = LoginSerializer(data=request.data)
        if not serializer.is_valid():
            return Response({'success': False, 'message': 'Validation
failed', 'errors': serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

        username = serializer.validated_data['username']
        password = serializer.validated_data['password']

        user = authenticate(request, username=username, password=password)

        if user is None:
            return Response({'success': False, 'message': 'Invalid username
or password'}, status=status.HTTP_401_UNAUTHORIZED)

        user_data = {
            'id': user.id,
            'username': user.username,
            'is_staff': user.is_staff,
            'is_superuser': user.is_superuser
        }

        if user.is_superuser:
            role = 'admin'
        else:
            role = 'staff'

```

```

        response_data = {
            'success': True,
            'message': 'Login successful',
            'role': role,
            'user': user_data
        }

        if role == 'staff' and hasattr(user, 'staff_profile'):
            service = user.staff_profile.assigned_service
            if service:
                response_data['service'] = {
                    'id': service.id,
                    'name': service.name,
                    'prefix': service.prefix,
                    'windows':
ServiceWindowSerializer(service.windows.filter(status='active'),
many=True).data}

        response = Response(response_data, status=status.HTTP_200_OK)
        response = set_jwt_cookies(response, user)
        return response

    except Exception as e:
        print(f"Login error: {str(e)}")
        return Response({'success': False, 'message': 'An unexpected error
occurred. Please try again later.'},
status=status.HTTP_500_INTERNAL_SERVER_ERROR)

@extend_schema(
    tags=['Authentication'],
    summary='Logout user',
    description="""Logout and clear JWT cookies.
Clears: access_token and refresh_token cookies
""",
    responses={
        200: OpenApiResponse(
            description="Logout successful",
            examples=[
                OpenApiExample(

```

```

        'Success Response',
        value={'success': True, 'message': 'Logout
successfully'}
    )
]
)
}
)
@api_view(['POST'])
@permission_classes([AllowAny])
def logout_view(request):
    logout(request)
    response = Response({'success': True, 'message': 'Logout
successfully'})
    response = delete_jwt_cookies(response)
    return response

@extend_schema(
    tags=['Authentication'],
    summary='Change password',
    description="""Change password for logged-in user.
After changing password, user is automatically logged out.
User must login again with new password.
""",
    request=ChangePasswordSerializer,
    responses={
        200: OpenApiResponse(
            description="Password changed",
            examples=[
                OpenApiExample(
                    'Success Response',
                    value={
                        'success': True,
                        'message': 'Password changed successfully please
login again'
                    }
                )
            ]
        ),

```

```

400: OpenApiResponse(
    description="Change failed",
    examples=[
        OpenApiExample(
            'Wrong old password',
            value={
                'success': False,
                'message': 'old password is incorrect'
            }
        )
    ]
)
},
examples=[
    OpenApiExample(
        'Example Request',
        value={
            'old_password': 'oldpass123',
            'new_password': 'newpass456',
            'confirm_password': 'newpass456'
        }
    )
]
)
@api_view(['POST'])
@permission_classes([IsAuthenticated])
def change_password_view(request):
    serializer = ChangePasswordSerializer(data=request.data)

    if serializer.is_valid():
        user = request.user
        if not
user.check_password(serializer.validated_data['old_password']):
    return Response({'success': False, 'message': 'old password is
incorrect'}, status=status.HTTP_400_BAD_REQUEST)

    user.set_password(serializer.validated_data['new_password'])
    user.save()
    logout(request)

```

```

        response = Response({'success': True, 'message': 'Password changed
successfully please login again'})
        response = delete_jwt_cookies(response)
        return response
    return Response({'success': False, 'message': 'Invalid Data',
'errors': serializer.errors}, status=400)

```

```

@extend_schema(
    tags=['Authentication'],
    summary='Get current user info',
    description="Get information about currently logged-in user",
    responses={
        200: OpenApiResponse(
            description="✔ User information",
            examples=[
                OpenApiExample(
                    'Admin User',
                    value={
                        'success': True,
                        'user': {
                            'id': 1,
                            'username': 'admin',
                            'is_staff': True,
                            'is_superuser': True
                        }
                    },
                ),
                OpenApiExample(
                    'Staff User',
                    value={
                        'success': True,
                        'user': {
                            'id': 2,
                            'username': 'cashier_staff',
                            'is_staff': True,
                            'is_superuser': False
                        }
                    },
                )
            ]
        )
    )

```

```

        ]
    )
}

@api_view(['GET'])
@permission_classes([IsAuthenticated])
def current_user_view(request):
    user = request.user
    user_data = {
        'id': user.id,
        'username': user.username,
        'is_staff': user.is_staff,
        'is_superuser': user.is_superuser
    }
    return Response({'success': True, 'user': user_data})

@extend_schema(
    tags=['Authentication'],
    summary='Refresh access token',
    description="Get new access token using refresh token.",
    responses={
        200: OpenApiResponse(
            description="✅ Token refreshed",
            examples=[
                OpenApiExample(
                    'Success Response',
                    value={'success': True, 'message': 'Token refreshed'}
                )
            ]
        ),
        400: OpenApiResponse(
            description="❌ No refresh token",
            examples=[
                OpenApiExample(
                    'Missing Token',
                    value={'success': False, 'message': 'Refresh Token not found'}
                )
            ]
        )
    ]
)

```

```

        ]
    ),
    401: OpenApiResponse(
        description="❌ Invalid refresh token",
        examples=[
            OpenApiExample(
                'Invalid Token',
                value={'success': False, 'message': 'Invalid refresh
token'}
            )
        ]
    )
}

@api_view(['POST'])
@permission_classes([IsAuthenticated])
def refresh_token_view(request):
    refresh_token = request.COOKIES.get('refresh_token')
    if not refresh_token:
        return Response({'success': False, 'message': 'Refresh Token not
found'}, status=status.HTTP_400_BAD_REQUEST)

    try:
        from rest_framework_simplejwt.tokens import RefreshToken
        refresh = RefreshToken(refresh_token)
        user = User.objects.get(id=refresh.payload['user_id'])
        response = Response({'success': True, 'message': 'Token
refreshed'})
        response = set_jwt_cookies(response, user)
        return response
    except Exception as e:
        return Response({
            'success': False,
            'message': 'Invalid refresh token'
        }, status=status.HTTP_401_UNAUTHORIZED)

# ===== ADMIN ENDPOINTS =====

@extend_schema(

```

```

tags=['Admin'],
summary='Create staff account (Admin only)',
description="Create new staff user and assign to a service in one
step.",
request=RegisterStaffSerializer,
responses={
    201: OpenApiResponse(
        description="Staff created",
        examples=[
            OpenApiExample(
                'Success Response',
                value={
                    'success': True,
                    'message': 'Staff account created for
cashier_staff and assigned to Cashier',
                    'user': {
                        'id': 2,
                        'username': 'cashier_staff',
                        'is_staff': True
                    },
                    'assigned_service': {
                        'id': 1,
                        'name': 'Cashier',
                        'prefix': 'C'
                    }
                }
            )
        ]
    )
}

@api_view(['POST'])
@permission_classes([IsAdminUser])
def create_staff_view(request):
    serializer = RegisterStaffSerializer(data=request.data)
    if serializer.is_valid():
        user = serializer.save()

        service = user.staff_profile.assigned_service

```

```

        user_data = {
            'id': user.id,
            'username': user.username,
            'is_staff': user.is_staff
        }

        service_data = {
            'id': service.id,
            'name': service.name,
            'prefix': service.prefix
        }

        return Response({
            'success': True,
            'message': f'Staff account created for {user.username} and assigned to {service.name}',
            'user': user_data,
            'assigned_service': service_data
        }, status=status.HTTP_201_CREATED)

    return Response({'success': False, 'message': 'Account failed to create', 'errors': serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

@extend_schema(
    tags=['Admin'],
    summary='List all staff (Admin only)',
    description="Get list of all staff users except yourself",
    responses={
        200: OpenApiResponse(
            description="Staff list",
            examples=[
                OpenApiExample(
                    'Success Response',
                    value={
                        'success': True,
                        'count': 2,
                        'staff': [
                            {
                                'id': 2,
                                'username': 'staff1',

```

```

        'is_staff': True
    },
    {
        'id': 3,
        'username': 'staff2',
        'is_staff': True
    }
]

    )

]

)

}

)

@api_view(['GET'])
@permission_classes([IsAdminUser])
def list_staff_view(request):
    staff_users =
User.objects.filter(is_staff=True).exclude(id=request.user.id)
    serializer = UserSerializer(staff_users, many=True)
    return Response({'success': True, 'count': staff_users.count(),
'staff': serializer.data})

@extend_schema(
    tags=['Admin'],
    summary='Delete staff (Admin only)',
    description="Delete a staff user by ID.",
    responses={
        200: OpenApiResponse(
            description="Staff deleted",
            examples=[
                OpenApiExample(
                    'Success Response',
                    value={
                        'success': True,
                        'message': 'Staff account staff1 deleted'
                    }
                )
            ]
        )
    }
)

```

```

    ),
    400: OpenApiResponse(
        description="Self-deletion attempt",
        examples=[
            OpenApiExample(
                'Error Response',
                value={
                    'success': False,
                    'message': 'Cannot delete your own account'
                }
            )
        ]
    ),
    404: OpenApiResponse(
        description="Staff not found",
        examples=[
            OpenApiExample(
                'Error Response',
                value={
                    'success': False,
                    'message': 'staff user not found'
                }
            )
        ]
    )
}

)

@api_view(['DELETE'])
@permission_classes([IsAdminUser])
def delete_staff_view(request, user_id):
    try:
        if request.user.id == user_id:
            return Response({'success': False, 'message': 'Cannot delete
your own account'}, status=status.HTTP_400_BAD_REQUEST)

        user = User.objects.get(id=user_id, is_staff=True)
        username = user.username
        user.delete()

        return Response({'success': True, 'message': f'Staff account
{username} deleted'})

```

```

except User.DoesNotExist:
    return Response({'success': False, 'message': 'staff user not
found'}, status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    tags=['Admin'],
    summary='Update staff account (Admin only)',
    description="""Update staff user information and reassign to a
different service.
**Cannot update password here.** Use change password endpoint.
""",
    request={
        'application/json': {
            'type': 'object',
            'properties': {
                'username': {'type': 'string', 'description': 'New
username (optional)'},
                'is_active': {'type': 'boolean', 'description':
'Activate/deactivate account'},
                'service_id': {'type': 'integer', 'description': 'Reassign
to different service'}
            },
            'example': {
                'username': 'new_cashier_staff',
                'is_active': True,
                'service_id': 2
            }
        }
    },
    responses={
        200: OpenApiResponse(
            description="Staff updated successfully",
            examples=[
                OpenApiExample(
                    'Success Response',
                    value={
                        'success': True,
                        'message': 'Staff account cashier_staff updated
and assigned to Permit',

```

```

        'user': {
            'id': 2,
            'username': 'new_cashier_staff',
            'is_staff': True,
            'is_active': True
        },
        'assigned_service': {
            'id': 2,
            'name': 'Permit',
            'prefix': 'P'
        }
    }
)
]
),
400: OpenApiResponse(
    description="Invalid data",
    examples=[
        OpenApiExample(
            'Validation Error',
            value={
                'success': False,
                'message': 'Invalid data',
                'errors': {
                    'username': ['A user with that username
already exists.']
                }
            }
        )
    ]
),
404: OpenApiResponse(
    description="User or Service not found",
    examples=[
        OpenApiExample(
            'User Not Found',
            value={
                'success': False,
                'message': 'Staff user not found'
            }
        )
    ]
)

```

```

        ),
        OpenApiExample(
            'Service Not Found',
            value={
                'success': False,
                'message': 'Service not found'
            }
        )
    ]
)
}
)
@api_view(['PUT', 'PATCH'])
@permission_classes([IsAdminUser])
def update_staff_view(request, user_id):
    try:
        # Get the staff user
        user = User.objects.get(id=user_id, is_staff=True)

        # Copy data and remove password fields
        data = request.data.copy()
        if 'password' in data:
            data.pop('password')
        if 'password2' in data:
            data.pop('password2')

        # Check if service_id is provided and valid
        service_id = data.pop('service_id', None)
        if service_id:
            try:
                service = Service.objects.get(id=service_id)
            except Service.DoesNotExist:
                return Response({
                    'success': False,
                    'message': 'Service not found'
                }, status=status.HTTP_404_NOT_FOUND)

        # Update user fields (username, is_active, etc.)
        serializer = UserSerializer(user, data=data, partial=True)
        if serializer.is_valid():

```

```

        serializer.save()

    # Update service assignment if provided
    if service_id:
        # Get or create staff profile
        profile, created =
StaffProfile.objects.get_or_create(user=user)
        profile.assigned_service = service
        profile.role = 'staff'
        profile.save()
        assigned_service_name = service.name
    else:
        # Keep existing service assignment
        if hasattr(user, 'staff_profile') and
user.staff_profile.assigned_service:
            assigned_service_name =
user.staff_profile.assigned_service.name
            service = user.staff_profile.assigned_service
        else:
            assigned_service_name = None
            service = None

    # Prepare response
    user_data = {
        'id': user.id,
        'username': user.username,
        'is_staff': user.is_staff,
        'is_active': user.is_active
    }

    service_data = None
    if service:
        service_data = {
            'id': service.id,
            'name': service.name,
            'prefix': service.prefix
        }

    message = f'Staff account {user.username} updated'
    if service_id:

```

```

        message += f' and assigned to {assigned_service_name}'

        return Response({'success': True, 'message': message, 'user':
user_data, 'assigned_service': service_data})

        return Response({'success': False, 'message': 'Invalid
data', 'errors': serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

    except User.DoesNotExist:
        return Response({'success': False, 'message': 'Staff user not
found'}, status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    tags=['Admin'],
    summary='Create Admin Account (Admin only)',
    description="Create another admin user. Only username and password
required.",
    request=CreateAdminSerializer,
    responses={
        201: OpenApiResponse(
            description="Admin created",
            examples=[
                OpenApiExample(
                    'Success Response',
                    value={
                        'success': True,
                        'message': 'Admin account created for new_admin',
                        'user': {
                            'id': 5,
                            'username': 'new_admin',
                            'is_staff': True,
                            'is_superuser': True
                        }
                    }
                )
            ]
        )
    }
)

```

```

@api_view(['POST'])
@permission_classes([IsAdminUser])
def create_admin_view(request):
    serializer = CreateAdminSerializer(data=request.data)
    if serializer.is_valid():
        user = serializer.save()
        user_data = {'id': user.id, 'username': user.username, 'is_staff':
user.is_staff, 'is_superuser': user.is_superuser}
        return Response({'success': True, 'message': f'Admin account
created for {user.username}', 'user': user_data},
status=status.HTTP_201_CREATED)

    return Response({'success': False, 'message': 'Failed to create
admin', 'errors': serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

@extend_schema(
    tags=['Admin'],
    summary='List All Admins',
    description="Get list of all admin users",
    responses={
        200: OpenApiResponse(
            description="List of admins",
            examples=[
                OpenApiExample(
                    'Success Response',
                    value={
                        'success': True,
                        'count': 2,
                        'admins': [
                            {
                                'id': 1,
                                'username': 'admin',
                                'is_active': True
                            },
                            {
                                'id': 5,
                                'username': 'new_admin',
                                'is_active': True
                            }
                        ]
                    }
                )
            ]
        )
    }
)

```

```

        ]
    }
    )
]
)
}
)
@api_view(['GET'])
@permission_classes([IsAdminUser])
def list_admins_view(request):
    admins =
User.objects.filter(is_superuser=True).exclude(id=request.user.id)

    admin_list = []
    for admin in admins:
        admin_list.append({'id': admin.id, 'username':
admin.username, 'is_active': admin.is_active})
    return Response({'success': True, 'count': len(admin_list), 'admins':
admin_list})

@extend_schema(
    tags=['Admin'],
    summary='Delete Admin Account',
    description="Delete an admin user (cannot delete yourself)",
    responses={
        200: OpenApiResponse(description="Admin deleted"),
        400: OpenApiResponse(description="Cannot delete yourself"),
        404: OpenApiResponse(description="Admin not found")
    }
)
@api_view(['DELETE'])
@permission_classes([IsAdminUser])
def delete_admin_view(request, user_id):
    try:
        if request.user.id == user_id:
            return Response({'success': False, 'message': 'Cannot delete
your own account'}, status=status.HTTP_400_BAD_REQUEST)

        user = User.objects.get(id=user_id, is_superuser=True)
        username = user.username

```

```

        user.delete()

        return Response({'success': True, 'message': f'Admin account
{username} deleted'})

    except User.DoesNotExist:
        return Response({'success': False, 'message': 'Admin user not
found'}, status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    tags=['Admin'],
    summary='Update Admin Account',
    description="Update admin user information (cannot change password
here)",
    responses={
        200: OpenApiResponse(description="Admin updated"),
        404: OpenApiResponse(description="Admin not found")
    }
)
@api_view(['PUT', 'PATCH'])
@permission_classes([IsAdminUser])
def update_admin_view(request, user_id):
    try:
        user = User.objects.get(id=user_id, is_superuser=True)
        # Don't allow password update here
        data = request.data.copy()
        if 'password' in data:
            data.pop('password')
        if 'password2' in data:
            data.pop('password2')

        serializer = UserSerializer(user, data=data, partial=True)

        if serializer.is_valid():
            serializer.save()
            return Response({'success': True, 'message': f'Admin account
{user.username} updated', 'user': serializer.data})

```

```

        return Response({'success': False, 'message': 'Invalid
data', 'errors': serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

    except User.DoesNotExist:
        return Response({'success': False, 'message': 'Admin not found'},
status=404)

@api_view(["POST"])
@permission_classes([IsAdminUser])
def admin_reset_staff_password_view(request, user_id):
    new_password = request.data.get("new_password")
    confirm_password = request.data.get("confirm_password")

    if not new_password or not confirm_password:
        return Response({"success": False, "message": "Password fields are
required"}, status=400)

    if new_password != confirm_password:
        return Response({"success": False, "message": "Passwords do not
match"}, status=400)

    try:
        user = User.objects.get(id=user_id, is_superuser=False)
    except User.DoesNotExist:
        return Response({"success": False, "message": "Staff user not
found"}, status=404)

    user.set_password(new_password)
    user.save()

    return Response({"success": True, "message": f"Password reset for
{user.username}"})

```

Views(service)

```
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import IsAuthenticated, IsAdminUser
from django.utils import timezone
from .models import Service, ServiceWindow, Ticket
from .serializers import ServiceSerializer, ServiceWindowSerializer
from drf_spectacular.utils import extend_schema
from .websocket_utils import send_dashboard_update, send_service_update,
send_service_status_update

@extend_schema(
    summary="Service List",
    description="Get list of all services (Admin only)",
    tags=['Service Management']
)
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def service_list(request):
    #List all services (authenticated users only)
    services = Service.objects.all().order_by('name')
    serializer = ServiceSerializer(services, many=True)

    return Response({'success': True, 'count': services.count(), 'services':
serializer.data})

@extend_schema(
    tags=['Service Management'],
    summary='Create service with optional auto-generated windows',
    description='Create a new service and optionally auto-create specified
number of windows'
)
@api_view(['POST'])
@permission_classes([IsAdminUser])
def create_service(request):
    """
    Create a service. If num_windows is provided, auto-create that many
windows.
    """
```

```

"""
data = request.data.copy()
num_windows = data.pop('num_windows', None)

serializer = ServiceSerializer(data=data)

if serializer.is_valid():
    service = serializer.save()

    windows = []
    if num_windows and int(num_windows) > 0:
        for i in range(1, int(num_windows) + 1):
            window = ServiceWindow.objects.create(
                service=service,
                window_number=i,
                name=f"{service.name} Window {i}",
                status='active',
                description=f"Auto-created window for {service.name}"
            )
            windows.append(ServiceWindowSerializer(window).data)

    response_data = {
        'success': True,
        'message': f'Service "{service.name}" created successfully',
        'service': ServiceSerializer(service).data
    }

    if windows:
        response_data['windows'] = windows
        response_data['message'] += f' with {num_windows} windows'

    return Response(response_data, status=status.HTTP_201_CREATED)

    return Response({'success': False, 'message': 'Invalid data', 'errors':
serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

@extend_schema(
    summary="Service List",
    description="Get list of all services (Admin only)",

```

```

        tags=['Service Management']
    )
@api_view(['PUT', 'PATCH'])
@permission_classes([IsAdminUser])
def update_service(request, service_id):
    #Admin-only: Update service
    try:
        service = Service.objects.get(id=service_id)
    except Service.DoesNotExist:
        return Response({'success': False, 'message': 'Service not found'},
status=status.HTTP_404_NOT_FOUND)

    # Track if status is changing
    old_is_active = service.is_active

    serializer = ServiceSerializer(service, data=request.data,
partial=True)

    if serializer.is_valid():
        service = serializer.save()

        if old_is_active != service.is_active:
            send_dashboard_update()
            send_service_update(service.id)
            send_service_status_update(service.id, service.is_active)

            if not service.is_active:
                # Optionally auto-complete any serving tickets
                Ticket.objects.filter(
                    service=service,
                    status='serving'
                ).update(
                    status='served',
                    served_at=timezone.now()
                )

        return Response({'success': True, 'message': f'Service
"{service.name}" updated successfully', 'service': serializer.data})

```

```

        return Response({'success': False, 'message': 'Invalid data',
'errors': serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

@extend_schema(
    summary="Service List",
    description="Get list of all services (Admin only)",
    tags=['Service Management']
)
@api_view(['DELETE'])
@permission_classes([IsAdminUser])
def delete_service(request, service_id):
    #Admin-only: Delete service
    try:
        service = Service.objects.get(id=service_id)
        service_name = service.name
        service.delete()

        return Response({'success': True, 'message': f'Service
"{service_name}" deleted successfully'})

    except Service.DoesNotExist:
        return Response({'success': False, 'message': 'Service not found'},
status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    summary="Service List",
    description="Get list of all services (Admin only)",
    tags=['Service Management']
)
@api_view(['GET'])
@permission_classes([IsAdminUser])
def service_stats(request, service_id):
    # Admin-only: Get service statistics
    try:
        service = Service.objects.get(id=service_id)

        today = timezone.now().date()
        tickets_today = service.tickets.filter(ticket_date=today)

        stats = {

```

```

        'service': ServiceSerializer(service).data,
        'today': {
            'total_tickets': tickets_today.count(),
            'waiting': tickets_today.filter(status__in=['waiting',
'notified']).count(),
            'serving': tickets_today.filter(status='serving').count(),
            'served': tickets_today.filter(status='served').count(),
            'cancelled':
tickets_today.filter(status='cancelled').count(),
            'skipped': tickets_today.filter(status='skipped').count(),
        },
        'average_wait_time': service.average_service_time *
service.waiting_count
    }

    return Response({'success': True, 'stats': stats})

except Service.DoesNotExist:
    return Response({'success': False, 'message': 'Service not found'},
status=status.HTTP_404_NOT_FOUND)

```

Views(staff)

```

from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status
from django.utils import timezone
from .models import Ticket, ServiceWindow
from .serializers import TicketSerializer, ServiceWindowSerializer
from .permissions import IsServiceStaff
from drf_spectacular.utils import extend_schema
from .websocket_utils import send_dashboard_update, send_service_update,
send_ticket_update, send_queue_position_updates,
debounced_send_queue_updates
from django.db import transaction
from .sms_utils import check_and_send_sms

def _get_claimed_window_for_staff(user, service, window_id):

```

```

    """Check if staff can use this window"""
    if not hasattr(user, 'staff_profile'):
        return None, Response({'success': False, 'message': 'User is not a
staff member'}, status=403)

    try:
        window = ServiceWindow.objects.get(id=window_id, service=service,
status='active')
    except ServiceWindow.DoesNotExist:
        return None, Response({'success': False, 'message': 'Window not
found or inactive'}, status=404)

    # Only block if window is claimed by a DIFFERENT user
    if window.current_staff and window.current_staff != user:
        return None, Response({
            'success': False,
            'message': f'Window {window.name} is already being used by
{window.current_staff.username}'
        }, status=403)

    # If window is not claimed OR claimed by this user, allow
    # Auto-assign if not claimed
    if not window.current_staff:
        window.current_staff = user
        window.save()

    return window, None

@extend_schema(
    summary="queue management",
    description="Staff operations for managing queues",
    tags=['Staff Queue Management']
)
@api_view(['GET'])
@permission_classes([IsServiceStaff])
def staff_dashboard(request):
    if not hasattr(request.user, 'staff_profile'):

```

```
        return Response({'success': False, 'message': 'Not staff'},
status=403)

    staff_profile = request.user.staff_profile
    service = staff_profile.assigned_service

    if not service:
        return Response({'success': False, 'message': 'No service
assigned'}, status=400)

    today = timezone.now().date()

    waiting = Ticket.objects.filter(
        service=service,
        ticket_date=today,
        status='waiting'
    ).order_by('queue_number')

    notified = Ticket.objects.filter(
        service=service,
        ticket_date=today,
        status='notified'
    ).order_by('queue_number')

    serving_tickets = Ticket.objects.filter(
        service=service,
        ticket_date=today,
        status='serving'
    ).order_by('called_at', 'queue_number')

    skipped_tickets = Ticket.objects.filter(
        service=service,
        ticket_date=today,
        status='skipped'
    ).order_by('queue_number')

    waiting_list = list(waiting) + list(notified)

    windows_status = []
    for window in service.windows.order_by('window_number'):
```

```

        serving = Ticket.objects.filter(
            service=service,
            assigned_window=window,
            ticket_date=today,
            status='serving'
        ).first()

    windows_status.append({
        'id': window.id,
        'name': window.name,
        'number': window.window_number,
        'currently_serving': {
            'ticket_id': str(serving.ticket_id),
            'display_number': serving.display_number
        } if serving else None,
        'status': window.status,
        'is_available': window.is_available,
        'is_in_use': window.is_in_use,
        'claimed_by': window.current_staff.username if
window.current_staff else None,
        'current_staff': window.current_staff.id if
window.current_staff else None,
        'current_staff_name': window.current_staff.username if
window.current_staff else None,
    })

    return Response({
        'success': True,
        'dashboard': {
            'service': service.name,
            'waiting_count': waiting.count() + notified.count(),
            'serving_count': serving_tickets.count(),
            'next_ticket': waiting.first().display_number if
waiting.exists() else (
                notified.first().display_number if notified.exists() else
None
            ),
            'currently_serving': TicketSerializer(serving_tickets,
many=True).data,

```

```

        'waiting_list': TicketSerializer(waiting_list,
many=True).data,
        'skipped_list': TicketSerializer(skipped_tickets,
many=True).data,
        'windows': windows_status
    }
})

@extend_schema(
    summary="queue management",
    description="Call next ticket",
    tags=['Staff Queue Management']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def call_next_ticket(request):
    #Call the next waiting ticket to a specific window

    # Check if user has staff profile
    if not hasattr(request.user, 'staff_profile'):
        return Response({'success': False, 'message': 'User is not a staff
member'}, status=status.HTTP_403_FORBIDDEN)

    staff_profile = request.user.staff_profile
    service = staff_profile.assigned_service

    if not service:
        return Response({'success': False, 'message': 'No service
assigned'}, status=400)

    # Check if service is active
    if not service.is_active:
        return Response({'success': False, 'message': f'Service
"{service.name}" is currently inactive. Please activate a window first.'},
status=status.HTTP_400_BAD_REQUEST)

    # Get window_id from request
    window_id = request.data.get('window_id')
    if not window_id:

```

```

        return Response({'success': False, 'message': 'window_id is
required'}, status=status.HTTP_400_BAD_REQUEST)

    # Verify window exists and belongs to this service
    window, error_response = _get_claimed_window_for_staff(request.user,
service, window_id)
    if error_response:
        return error_response

    today = timezone.now().date()

    #Use transaction to ensure data consistency
    from django.db import transaction
    with transaction.atomic():
        # STEP 1: Check if this window is already serving a ticket
        current_serving = Ticket.objects.filter(
            service=service,
            assigned_window=window,
            ticket_date=today,
            status='serving'
        ).select_for_update().first()

        completed_ticket = None
        if current_serving:
            # Auto-complete the current ticket at this window
            current_serving.status = 'served'
            current_serving.served_by = request.user
            current_serving.served_at = timezone.now()
            current_serving.save()
            completed_ticket = current_serving.display_number

        # STEP 2: Find next waiting ticket for this service
        next_ticket = Ticket.objects.filter(
            service=service,
            ticket_date=today,
            status='waiting'
        ).select_for_update().order_by('queue_number').first()

        if not next_ticket:
            # Prepare message based on whether we completed a ticket

```

```

        if completed_ticket:
            message = f'Ticket {completed_ticket} completed at
{window.name}. No more tickets waiting.'
        else:
            message = 'No tickets waiting in queue'

        # Trigger WebSocket updates even when no next ticket, after
transaction commits
        def send_empty_updates():
            send_dashboard_update()
            send_service_update(service.id)
            if completed_ticket:
                send_ticket_update(current_serving.ticket_id)

        transaction.on_commit(send_empty_updates)

        return Response({'success': False, 'message': message},
status=status.HTTP_404_NOT_FOUND)

    # STEP 3: Assign next ticket to this window
    next_ticket.status = 'serving'
    next_ticket.called_by = request.user
    next_ticket.called_at = timezone.now()
    next_ticket.assigned_window = window
    next_ticket.save()

    # STEP 4: Get queue info for response
    waiting_count = Ticket.objects.filter(
        service=service,
        ticket_date=today,
        status='waiting'
    ).count()

    next_waiting = Ticket.objects.filter(
        service=service,
        ticket_date=today,
        status='waiting'
    ).order_by('queue_number').first()

    # STEP 5: Trigger WebSocket updates after transaction commits

```

```

        from .websocket_utils import send_dashboard_update,
send_service_update, send_ticket_update

    def send_updates():
        send_dashboard_update()
        send_service_update(service.id)
        #Update ALL waiting tickets' queue positions
        debounced_send_queue_updates(service.id,
str(next_ticket.ticket_id))

        if completed_ticket:
            send_ticket_update(current_serving.ticket_id)
            send_ticket_update(next_ticket.ticket_id)

        check_and_send_sms(service.id)

    transaction.on_commit(send_updates)

    # Prepare response message
    if completed_ticket:
        message = f'{window.name}: Ticket {completed_ticket} completed.
Now serving {next_ticket.display_number}'
    else:
        message = f'{window.name}: Now serving
{next_ticket.display_number}'

    return Response({
        'success': True,
        'message': message,
        'ticket': {
            'ticket_id': next_ticket.ticket_id,
            'display_number': next_ticket.display_number,
            'queue_number': next_ticket.queue_number,
            'status': next_ticket.status,
            'people_ahead': next_ticket.people_ahead,
            'window': {
                'id': window.id,
                'name': window.name,
                'number': window.window_number
            }
        }
    })

```

```

    },
    'completed_ticket': completed_ticket,
    'window': {
        'id': window.id,
        'name': window.name,
        'number': window.window_number
    },
    'queue_info': {
        'waiting_count': waiting_count,
        'next_waiting': next_waiting.display_number if next_waiting
else None
    }
})

```

```

@extend_schema(
    summary="Call Specific Ticket",
    description="Call a specific ticket by its display number (e.g.,
'C001')",
    tags=['Staff Queue Management']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def call_specific_ticket(request):
    """Call a specific ticket by number"""
    ticket_number = request.data.get('ticket_number')
    window_id = request.data.get('window_id')

    if not ticket_number:
        return Response({'success': False, 'message': 'ticket_number is
required'}, status=status.HTTP_400_BAD_REQUEST)

    if not window_id:
        return Response({'success': False, 'message': 'window_id is
required'}, status=status.HTTP_400_BAD_REQUEST)

    staff_profile = request.user.staff_profile
    service = staff_profile.assigned_service

```

```

        window, error_response = _get_claimed_window_for_staff(request.user,
service, window_id)
        if error_response:
            return error_response

    today = timezone.now().date()

    # Variables to store IDs for WebSocket updates after transaction
    called_ticket_id = None
    completed_ticket_id = None
    called_ticket_display = None
    completed_ticket_display = None

    with transaction.atomic():
        try:
            ticket = Ticket.objects.select_for_update().get(
                service=service,
                ticket_date=today,
                display_number=ticket_number
            )
        except Ticket.DoesNotExist:
            return Response({'success': False, 'message': f'Ticket
{ticket_number} not found in today\'s queue'},
status=status.HTTP_404_NOT_FOUND)

        if ticket.status not in ['waiting', 'notified', 'skipped']:
            return Response({'success': False, 'message': f'Ticket
{ticket_number} cannot be called (current status: {ticket.status})'},
status=status.HTTP_400_BAD_REQUEST)

        current_serving = Ticket.objects.filter(
            service=service,
            assigned_window=window,
            ticket_date=today,
            status='serving'
        ).select_for_update().first()

        if current_serving:
            current_serving.status = 'served'

```

```

        current_serving.served_by = request.user
        current_serving.served_at = timezone.now()
        current_serving.save()
        completed_ticket_display = current_serving.display_number
        completed_ticket_id = str(current_serving.ticket_id)

    # Call the specific ticket
    ticket.status = 'serving'
    ticket.called_by = request.user
    ticket.called_at = timezone.now()
    ticket.assigned_window = window
    ticket.save()

    # Store IDs for WebSocket updates
    called_ticket_id = str(ticket.ticket_id)
    called_ticket_display = ticket.display_number

    # Get waiting tickets for later update (outside transaction)
    waiting_tickets = list(Ticket.objects.filter(
        service=service,
        ticket_date=today,
        status__in=['waiting', 'notified']
    ))

    # WebSocket updates outside transaction
    send_dashboard_update()
    send_service_update(service.id)
    send_queue_position_updates(service.id, called_ticket_id)

    for wt in waiting_tickets:
        send_ticket_update(str(wt.ticket_id))

    send_ticket_update(called_ticket_id)

    if completed_ticket_id:
        send_ticket_update(completed_ticket_id)

    check_and_send_sms(service.id, threshold=5)

    return Response({

```

```

        'success': True,
        'message': f'Ticket {called_ticket_display} called to
{window.name}',
        'ticket': {
            'ticket_id': called_ticket_id,
            'display_number': called_ticket_display,
            'status': 'serving',
            'window': {
                'id': window.id,
                'name': window.name,
                'number': window.window_number
            }
        },
        'completed_ticket': completed_ticket_display
    })

@extend_schema(
    summary="Start Serving",
    description="Start serving a ticket (can be from waiting or notified
status)",
    tags=['Staff Queue Management']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def start_serving(request, ticket_id):
    try:
        ticket = Ticket.objects.get(ticket_id=ticket_id)

        if ticket.service != request.user.staff_profile.assigned_service:
            return Response({'success': False, 'message': 'You do not have
permission to serve tickets from this service'},
status=status.HTTP_403_FORBIDDEN)

        if ticket.status not in ['waiting', 'notified']:
            return Response({'success': False, 'message': f'Ticket must be
in "waiting" or "notified" status. Current: {ticket.status}'},
status=status.HTTP_400_BAD_REQUEST)

        window_id = request.data.get('window_id')

```

```

        if not window_id:
            return Response({'success': False, 'message': 'window_id is
required'}, status=400)

        window, error_response =
_get_claimed_window_for_staff(request.user, ticket.service, window_id)
        if error_response:
            return error_response

        ticket.status = 'serving'
        ticket.assigned_window = window
        ticket.save()

        return Response({'success': True, 'message': f'Now serving ticket
{ticket.display_number} at {window.name}', 'ticket':
TicketSerializer(ticket).data})

    except Ticket.DoesNotExist:
        return Response({'success': False, 'message': 'Ticket not found'},
status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    summary="Complete Serving",
    description="Manually mark a ticket as served",
    tags=['Staff Queue Management']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def complete_serving(request, ticket_id):
    #manual served
    try:
        ticket = Ticket.objects.get(ticket_id=ticket_id)

        if ticket.service != request.user.staff_profile.assigned_service:
            return Response({'success': False, 'message': 'You do not have
permission to serve tickets from this service'},
status=status.HTTP_403_FORBIDDEN)

        if ticket.status != 'serving':

```

```

        return Response({'success': False, 'message': f'Ticket must be
in "serving" status. Current: {ticket.status}'},
status=status.HTTP_400_BAD_REQUEST)

    # Update ticket
    ticket.status = 'served'
    ticket.served_by = request.user
    ticket.served_at = timezone.now()
    ticket.save()

    # Trigger WebSocket updates
    send_dashboard_update()
    send_service_update(ticket.service.id)
    send_ticket_update(ticket.ticket_id)

    return Response({
        'success': True,
        'message': f'Ticket {ticket.display_number} marked as served',
        'ticket': {
            'ticket_id': ticket.ticket_id,
            'display_number': ticket.display_number,
            'status': ticket.status
        }
    })

except Ticket.DoesNotExist:
    return Response({'success': False, 'message': 'Ticket not found'},
status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    summary="queue management",
    description="Staff operations for managing queues",
    tags=['Staff Queue Management']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def remove_ticket(request, ticket_id):
    reason = request.data.get('reason', 'No reason provided')

```

```

try:
    ticket = Ticket.objects.get(ticket_id=ticket_id)

    # Check permission
    if ticket.service != request.user.staff_profile.assigned_service:
        return Response({'success': False, 'message': 'You do not have
permission to remove tickets from this service'},
status=status.HTTP_403_FORBIDDEN)

    # Can't remove served tickets
    if ticket.status == 'served':
        return Response({'success': False, 'message': 'Cannot remove a
served ticket'}, status=status.HTTP_400_BAD_REQUEST)

    # Remove ticket (mark as cancelled)
    ticket.status = 'cancelled'
    ticket.notes = f"Removed from queue: {reason}"
    ticket.save()

    return Response({'success': True, 'message': f'Ticket
{ticket.display_number} removed from queue', 'ticket':
TicketSerializer(ticket).data})

except Ticket.DoesNotExist:
    return Response({'success': False, 'message': 'Ticket not found'},
status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    summary="queue management",
    description="Staff operations for managing queues",
    tags=['Staff Queue Management']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def recall_ticket(request, ticket_id):
    try:
        ticket = Ticket.objects.get(ticket_id=ticket_id)

        # Check permission

```

```

        if ticket.service != request.user.staff_profile.assigned_service:
            return Response({'success': False, 'message': 'You do not have
permission to recall tickets from this service'},
status=status.HTTP_403_FORBIDDEN)

        # Check if ticket can be recalled
        if ticket.status not in ['notified', 'skipped', 'cancelled']:
            return Response({'success': False, 'message': f'Cannot recall
ticket in {ticket.status} status'}, status=status.HTTP_400_BAD_REQUEST)

        # Move back to waiting
        old_status = ticket.status
        ticket.status = 'waiting'
        ticket.notes = f"Recalled from {old_status}: {ticket.notes}"
        ticket.called_by = None
        ticket.called_at = None
        ticket.save()

        return Response({'success': True, 'message': f'Ticket
{ticket.display_number} recalled to waiting queue', 'ticket':
TicketSerializer(ticket).data})

    except Ticket.DoesNotExist:
        return Response({'success': False, 'message': 'Ticket not found'},
status=status.HTTP_404_NOT_FOUND)

@extend_schema(
    summary="queue management",
    description="Staff operations for managing queues",
    tags=['Staff Queue Management']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def toggle_queue_status(request):
    staff_profile = request.user.staff_profile
    service = staff_profile.assigned_service

    if not service:

```

```

        return Response({'success': False, 'message': 'No service assigned
to your account'}, status=status.HTTP_400_BAD_REQUEST)

    # Toggle service active status
    service.is_active = not service.is_active
    service.save()

    status_text = "paused" if not service.is_active else "resumed"

    return Response({
        'success': True,
        'message': f'Queue for {service.name} {status_text}',
        'service': {
            'id': service.id,
            'name': service.name,
            'is_active': service.is_active
        }
    })

@extend_schema(
    summary="Skip Ticket",
    description="Mark a currently serving ticket as skipped instead of
served",
    tags=['Staff Queue Management']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def skip_ticket(request, ticket_id):
    reason = request.data.get('reason', 'Customer not present')

    try:
        ticket = Ticket.objects.get(ticket_id=ticket_id)

        if ticket.service != request.user.staff_profile.assigned_service:
            return Response({
                'success': False,
                'message': 'You do not have permission to skip tickets
from this service'
            }, status=status.HTTP_403_FORBIDDEN)

```

```

        if ticket.status != 'serving':
            return Response({
                'success': False,
                'message': f'Ticket must be in "serving" status to be
skipped. Current: {ticket.status}'
            }, status=status.HTTP_400_BAD_REQUEST)

        ticket.status = 'skipped'
        ticket.skipped_at = timezone.now()
        ticket.notes = f"Skipped: {reason}"
        ticket.save()

        send_dashboard_update()
        send_service_update(ticket.service.id)
        send_ticket_update(str(ticket.ticket_id))

    return Response({
        'success': True,
        'message': f'Ticket {ticket.display_number} has been skipped',
        'ticket': {
            'ticket_id': str(ticket.ticket_id),
            'display_number': ticket.display_number,
            'status': ticket.status,
            'skipped_at': ticket.skipped_at,
            'notes': ticket.notes,
        }
    })

except Ticket.DoesNotExist:
    return Response({
        'success': False,
        'message': 'Ticket not found'
    }, status=status.HTTP_404_NOT_FOUND)

```

Views(windows)

```
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response
from rest_framework import status
from rest_framework.permissions import IsAuthenticated, IsAdminUser
from .models import Service, ServiceWindow, Ticket
from .serializers import ServiceWindowSerializer
from drf_spectacular.utils import extend_schema, OpenApiParameter,
OpenApiResponse, OpenApiExample
from drf_spectacular.types import OpenApiTypes
from .websocket_utils import send_dashboard_update, send_service_update,
send_service_status_update, send_ticket_update, send_windows_update
from django.utils import timezone

@extend_schema(
    summary="Service Windows List",
    description="Get list of windows for a service",
    tags=['Service Window Management']
)
@api_view(['GET'])
@permission_classes([IsAuthenticated])
def service_windows_list(request, service_id):
    # Get all windows for a specific service
    try:
        service = Service.objects.get(id=service_id)
        windows = service.windows.all().order_by('window_number')
        serializer = ServiceWindowSerializer(windows, many=True)

        return Response({'success': True, 'service': service.name,
            'count': windows.count(), 'windows': serializer.data})

    except Service.DoesNotExist:
        return Response({'success': False, 'message': 'Service not found'},
            status=404)

@extend_schema(
    summary="Service Windows List",
```

```

        description="Get list of windows for a service",
        tags=['Service Window Management']
    )
@api_view(['POST'])
@permission_classes([IsAdminUser])
def create_service_window(request, service_id):
    #Admin-only: Create new window for a service
    try:
        service = Service.objects.get(id=service_id)
    except Service.DoesNotExist:
        return Response({'success': False, 'message': 'Service not found'},
            status=status.HTTP_404_NOT_FOUND)

    # Add service to request data
    data = request.data.copy()
    data['service'] = service_id

    serializer = ServiceWindowSerializer(data=data)

    if serializer.is_valid():
        window = serializer.save()
        send_windows_update(service.id)

        return Response({
            'success': True,
            'message': f'Window "{window.name or f"Window {window.window_number}"' created for {service.name}',
            'window': serializer.data
        }, status=status.HTTP_201_CREATED)

    return Response({'success': False, 'message': 'Invalid data', 'errors':
        serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

@extend_schema(
    tags=['Service Window Management'],
    summary='Update service window status',
    description='Update window status and automatically update parent
service active status'
)

```

```

@api_view(['PUT', 'PATCH'])
@permission_classes([IsAdminUser])
def update_service_window(request, window_id):
    """
    Update service window. When status changes, automatically update
    parent service.
    """
    try:
        window = ServiceWindow.objects.get(id=window_id)
    except ServiceWindow.DoesNotExist:
        return Response({'success': False, 'message': 'Window not found'},
            status=status.HTTP_404_NOT_FOUND)

    old_status = window.status
    serializer = ServiceWindowSerializer(window, data=request.data,
        partial=True)

    if serializer.is_valid():
        window = serializer.save()

        # If a usable window is disabled, complete the current ticket and
        clear its claim.
        completed_ticket = None
        if old_status == 'active' and window.status == 'inactive':
            current_ticket = Ticket.objects.filter(assigned_window=window,
                status='serving').first()

            if current_ticket:
                current_ticket.status = 'served'
                current_ticket.served_at = timezone.now()
                current_ticket.save()
                completed_ticket = current_ticket.display_number

                send_ticket_update(str(current_ticket.ticket_id))

            if window.current_staff_id:
                window.current_staff = None
                window.save(update_fields=['current_staff'])

        # Check if status actually changed

```

```

        if old_status != window.status:
            send_dashboard_update()
            send_service_update(window.service.id)
            send_windows_update(window.service.id)

        response_data = {
            'success': True,
            'message': f'Window updated successfully',
            'window': serializer.data,
            'service_active': window.service.is_active
        }

        if completed_ticket:
            response_data['message'] += f' and completed ticket {completed_ticket}'
            response_data['completed_ticket'] = completed_ticket

        return Response(response_data)

    return Response({'success': False, 'message': 'Invalid data',
                    'errors': serializer.errors}, status=status.HTTP_400_BAD_REQUEST)

@extend_schema(
    summary="Service Windows List",
    description="Get list of windows for a service",
    tags=['Service Window Management']
)
@api_view(['DELETE'])
@permission_classes([IsAdminUser])
def delete_service_window(request, window_id):
    #Delete window and update service status
    try:
        window = ServiceWindow.objects.get(id=window_id)
        service = window.service
        window_name = window.name or f"Window {window.window_number}"
        service_name = service.name

        current_ticket = Ticket.objects.filter(assigned_window=window,
        status='serving').first()

```

```
    if current_ticket:
        current_ticket.status = 'served'
        current_ticket.served_at = timezone.now()
        current_ticket.save()

        send_ticket_update(str(current_ticket.ticket_id))

    window.delete()

    send_dashboard_update()
    send_service_update(service.id)
    send_windows_update(service.id)

    return Response({'success': True, 'message': f'Window
"{window_name}" deleted from {service_name}', 'service_active':
service.is_active})

    except ServiceWindow.DoesNotExist:
        return Response({'success': False, 'message': 'Window not found'},
status=404)
```

Views(Session)

```
from django.contrib.auth.models import User
from django.db import transaction
from django.utils import timezone
from drf_spectacular.utils import extend_schema
from rest_framework import status
from rest_framework.decorators import api_view, permission_classes
from rest_framework.response import Response

from .models import ServiceWindow, Ticket
from .permissions import IsServiceStaff
from .websocket_utils import (
    send_dashboard_update,
    send_service_status_update,
    send_service_update,
    send_ticket_update,
    send_windows_update,
)

def _parse_int(value, field_name):
    try:
        return int(value)
    except (TypeError, ValueError):
        raise ValueError(f'{field_name} must be an integer.')

def _complete_serving_ticket(window, served_by=None):
    ticket = Ticket.objects.filter(assigned_window=window,
status='serving').first()
    if not ticket:
        return None

    ticket.status = 'served'
    ticket.served_at = timezone.now()
    ticket.served_by = served_by
    ticket.save(update_fields=['status', 'served_at', 'served_by'])
    return str(ticket.ticket_id)
```

```

def _broadcast_window_state(service_id, completed_ticket_id=None):
    send_windows_update(service_id)
    send_dashboard_update()
    send_service_update(service_id)
    send_service_status_update(service_id)
    if completed_ticket_id:
        send_ticket_update(completed_ticket_id)

@extend_schema(
    summary='Claim service window',
    description='Marks a window as active and assigned to a staff
account.',
    tags=['Active Sessions']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def claim_session(request):
    window_id = request.data.get('window_id')
    staff_account_id = request.data.get('staff_account_id')

    if not window_id or not staff_account_id:
        return Response(
            {
                'error': 'invalid_request',
                'message': 'window_id and staff_account_id are required.',
            },
            status=status.HTTP_400_BAD_REQUEST,
        )

    try:
        window_id = _parse_int(window_id, 'window_id')
        staff_account_id = _parse_int(staff_account_id,
'staff_account_id')
    except ValueError as exc:
        return Response(
            {
                'error': 'invalid_request',
                'message': str(exc),
            },

```

```

        status=status.HTTP_400_BAD_REQUEST,
    )

    if not request.user.is_superuser and request.user.id !=
staff_account_id:
        return Response(
            {
                'error': 'staff_account_mismatch',
                'message': 'staff_account_id must match the authenticated
user.',
            },
            status=status.HTTP_403_FORBIDDEN,
        )

    with transaction.atomic():
        try:
            window =
ServiceWindow.objects.select_for_update().select_related('service').get(id
=window_id)
        except ServiceWindow.DoesNotExist:
            return Response(
                {
                    'error': 'window_not_found',
                    'message': 'Window not found.',
                },
                status=status.HTTP_404_NOT_FOUND,
            )

        if window.status != 'active':
            return Response(
                {
                    'error': 'window_unavailable',
                    'message': 'Window is unavailable.',
                },
                status=status.HTTP_400_BAD_REQUEST,
            )

        if not request.user.is_superuser and
request.user.staff_profile.assigned_service_id != window.service_id:
            return Response(

```

```

        {
            'error': 'forbidden_window',
            'message': 'You do not have access to this window.',
        },
        status=status.HTTP_403_FORBIDDEN,
    )

    try:
        staff_account =
User.objects.select_related('staff_profile').get(id=staff_account_id,
is_active=True)
    except User.DoesNotExist:
        return Response(
            {
                'error': 'staff_account_not_found',
                'message': 'Staff account not found.',
            },
            status=status.HTTP_404_NOT_FOUND,
        )

    if not staff_account.is_superuser:
        if not hasattr(staff_account, 'staff_profile'):
            return Response(
                {
                    'error': 'staff_profile_missing',
                    'message': 'Staff account does not have a staff
profile.',
                },
                status=status.HTTP_400_BAD_REQUEST,
            )
        if staff_account.staff_profile.assigned_service_id !=
window.service_id:
            return Response(
                {
                    'error': 'service_mismatch',
                    'message': 'Staff account is not assigned to this
service.',
                },
                status=status.HTTP_400_BAD_REQUEST,
            )

```

```

        if window.current_staff_id and window.current_staff_id !=
staff_account.id:
            claimed_by = window.current_staff.username if
window.current_staff else None
            return Response(
                {
                    'error': 'window_occupied',
                    'message': 'This window is currently in use.',
                    'window': {
                        'id': window.id,
                        'name': window.name,
                        'status': window.status,
                        'claimed_by': claimed_by,
                    },
                },
                status=status.HTTP_409_CONFLICT,
            )

        if window.current_staff_id == staff_account.id:
            return Response(
                {
                    'success': True,
                    'message': 'Window already claimed by this staff
account.',
                    'window': {
                        'id': window.id,
                        'name': window.name,
                        'number': window.window_number,
                        'status': window.status,
                        'current_staff': {
                            'id': staff_account.id,
                            'username': staff_account.username,
                        },
                    },
                },
                status=status.HTTP_200_OK,
            )

        window.current_staff = staff_account

```

```

        window.save(update_fields=['current_staff'])

    _broadcast_window_state(window.service_id)

    return Response(
        {
            'success': True,
            'message': 'Window claimed successfully.',
            'window': {
                'id': window.id,
                'name': window.name,
                'number': window.window_number,
                'status': window.status,
                'current_staff': {
                    'id': staff_account.id,
                    'username': staff_account.username,
                },
            },
        },
        status=status.HTTP_200_OK,
    )

@extend_schema(
    summary='Release service window',
    description='Marks a window as inactive and clears assigned staff.',
    tags=['Active Sessions']
)
@api_view(['POST'])
@permission_classes([IsServiceStaff])
def release_session(request):
    window_id = request.data.get('window_id')
    if not window_id:
        return Response(
            {
                'error': 'invalid_request',
                'message': 'window_id is required.',
            },
            status=status.HTTP_400_BAD_REQUEST,
        )

```

```

try:
    window_id = _parse_int(window_id, 'window_id')
except ValueError as exc:
    return Response(
        {
            'error': 'invalid_request',
            'message': str(exc),
        },
        status=status.HTTP_400_BAD_REQUEST,
    )

with transaction.atomic():
    try:
        window =
ServiceWindow.objects.select_for_update().select_related('service').get(id
=window_id)
    except ServiceWindow.DoesNotExist:
        return Response(
            {
                'error': 'window_not_found',
                'message': 'Window not found.',
            },
            status=status.HTTP_404_NOT_FOUND,
        )

    if not request.user.is_superuser and
request.user.staff_profile.assigned_service_id != window.service_id:
        return Response(
            {
                'error': 'forbidden_window',
                'message': 'You do not have access to this window.',
            },
            status=status.HTTP_403_FORBIDDEN,
        )

    if not window.current_staff_id:
        return Response(
            {
                'error': 'session_not_found',

```

```

        'message': 'This window is not currently claimed.',
    },
    status=status.HTTP_400_BAD_REQUEST,
)

if not request.user.is_superuser and window.current_staff_id and
window.current_staff_id != request.user.id:
    return Response(
        {
            'error': 'session_owner_mismatch',
            'message': 'Only the assigned staff can release this
window.',
        },
        status=status.HTTP_403_FORBIDDEN,
    )

    completed_ticket_id = _complete_serving_ticket(window,
served_by=request.user)

    window.current_staff = None
    window.save(update_fields=['current_staff'])

    _broadcast_window_state(window.service_id, completed_ticket_id)

    return Response(
        {
            'success': True,
            'message': 'Window released successfully.',
            'window': {
                'id': window.id,
                'name': window.name,
                'number': window.window_number,
                'status': window.status,
                'current_staff': None,
            },
            'completed_ticket_id': completed_ticket_id,
        },
        status=status.HTTP_200_OK,
    )

```

Authentication

```
from rest_framework_simplejwt.authentication import JWTAuthentication
from rest_framework_simplejwt.exceptions import InvalidToken,
AuthenticationFailed
from rest_framework_simplejwt.tokens import RefreshToken
from django.conf import settings

class JWTCookieAuthentication(JWTAuthentication):

    #Checks Authorization header first (Bearer token)

    def authenticate(self, request):
        # Check Authorization Header
        header = self.get_header(request)

        if header is not None:
            raw_token = self.get_raw_token(header)
            if raw_token is not None:
                validated_token = self.get_validated_token(raw_token)
                return self.get_user(validated_token), validated_token

        # If no header, check cookies
        access_token = request.COOKIES.get(
            settings.SIMPLE_JWT.get('AUTH_COOKIE', 'access_token')
        )

        if not access_token:
            return None

        try:
            validated_token = self.get_validated_token(access_token)
            return self.get_user(validated_token), validated_token
        except (InvalidToken, AuthenticationFailed):
            return None

    def set_jwt_cookies(response, user):
```

```

#Set JWT tokens as HTTP-only cookies in the response
#Also return tokens in response body for API testing

refresh = RefreshToken.for_user(user)
access_token = str(refresh.access_token)
refresh_token = str(refresh)

# Set Cookies
response.set_cookie(
    key=settings.SIMPLE_JWT.get('AUTH_COOKIE', 'access_token'),
    value=access_token,
    httponly=settings.SIMPLE_JWT.get('AUTH_COOKIE_HTTP_ONLY', True),
    secure=settings.SIMPLE_JWT.get('AUTH_COOKIE_SECURE', not
settings.DEBUG),
    samesite=settings.SIMPLE_JWT.get('AUTH_COOKIE_SAMESITE',
'Strict'),
    max_age=settings.SIMPLE_JWT.get('ACCESS_TOKEN_LIFETIME').total_seconds(),
    path=settings.SIMPLE_JWT.get('AUTH_COOKIE_PATH', '/'),
)

response.set_cookie(
    key=settings.SIMPLE_JWT.get('AUTH_COOKIE_REFRESH',
'refresh_token'),
    value=refresh_token,
    httponly=settings.SIMPLE_JWT.get('AUTH_COOKIE_HTTP_ONLY', True),
    secure=settings.SIMPLE_JWT.get('AUTH_COOKIE_SECURE', not
settings.DEBUG),
    samesite=settings.SIMPLE_JWT.get('AUTH_COOKIE_SAMESITE',
'Strict'),
    max_age=settings.SIMPLE_JWT.get('REFRESH_TOKEN_LIFETIME').total_seconds(),
    path=settings.SIMPLE_JWT.get('AUTH_COOKIE_PATH', '/'),
)

response.data["access"] = access_token
response.data["refresh"] = refresh_token

return response

```

```

def delete_jwt_cookies(response):
    cookie_name = settings.SIMPLE_JWT.get('AUTH_COOKIE', 'access_token')
    refresh_cookie_name = settings.SIMPLE_JWT.get('AUTH_COOKIE_REFRESH',
    'refresh_token')
    cookie_path = settings.SIMPLE_JWT.get('AUTH_COOKIE_PATH', '/')
    cookie_domain = settings.SIMPLE_JWT.get('AUTH_COOKIE_DOMAIN', None)
    cookie_samesite = settings.SIMPLE_JWT.get('AUTH_COOKIE_SAMESITE',
    'Lax')

    # Delete Access Token
    response.delete_cookie(
        key=cookie_name,
        path=cookie_path,
        domain=cookie_domain,
        samesite=cookie_samesite,
    )

    # Delete Refresh Token
    response.delete_cookie(
        key=refresh_cookie_name,
        path=cookie_path,
        domain=cookie_domain,
        samesite=cookie_samesite,
    )

    return response

```

Consumers

```

import json
from channels.generic.websocket import AsyncWebsocketConsumer
from channels.db import database_sync_to_async
from django.utils import timezone
from django.contrib.auth import get_user_model
from rest_framework_simplejwt.tokens import AccessToken
from rest_framework_simplejwt.exceptions import TokenError
from .models import Service, Ticket, ServiceWindow
from .serializers import TicketSerializer

```

```

User = get_user_model()

class DashboardConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.group_name = 'public_dashboard'
        await self.channel_layer.group_add(self.group_name,
self.channel_name)
        await self.accept()
        await self.send_dashboard_update()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(self.group_name,
self.channel_name)

    async def dashboard_update(self, event):
        await self.send_dashboard_update()

    async def receive(self, text_data):
        data = json.loads(text_data)
        if data.get('action') == 'refresh':
            await self.send_dashboard_update()

    async def send_dashboard_update(self):
        data = await self.get_dashboard_data()
        await self.send(text_data=json.dumps({
            'type': 'dashboard_update',
            'data': data
        }))

@database_sync_to_async
def get_dashboard_data(self):
    services = Service.objects.filter(is_active=True).order_by('name')
    service_data = []

    for service in services:
        today = timezone.now().date()
        tickets_today = service.tickets.filter(ticket_date=today)

```

```

        serving_tickets =
tickets_today.filter(status='serving').select_related('assigned_window').o
rder_by('assigned_window__window_number')
        currently_serving_list = [
            {
                'ticket_number': ticket.display_number,
                'window_name': ticket.assigned_window.name,
                'window_number': ticket.assigned_window.window_number
            }
            for ticket in serving_tickets if ticket.assigned_window
        ]

        waiting_tickets = tickets_today.filter(status__in=['waiting',
'notified']).order_by('queue_number')

        service_data.append({
            'id': service.id,
            'name': service.name,
            'prefix': service.prefix,
            'currently_serving': currently_serving_list,
            'serving_count': len(currently_serving_list),
            'next_in_line': waiting_tickets.first().display_number if
waiting_tickets.exists() else None,
            'waiting_count': waiting_tickets.count(),
        })

        total_waiting = sum(s['waiting_count'] for s in service_data)
        total_serving = sum(s['serving_count'] for s in service_data)

        return {
            'timestamp': timezone.now().isoformat(),
            'summary': {
                'total_waiting': total_waiting,
                'total_serving': total_serving,
                'last_updated': timezone.now().strftime('%I:%M:%S %p')
            },
            'services': service_data
        }

class StaffDashboardConsumer(AsyncWebsocketConsumer):

```

```

async def connect(self):
    self.service_id = self.scope['url_route']['kwargs']['service_id']
    self.user = self.scope['user']

    await self.accept()
    if self.user and self.user.is_authenticated:
        await self.authenticated_connect()

async def receive(self, text_data):
    try:
        data = json.loads(text_data)
        if data.get('type') == 'authenticate':
            await self.handle_authentication(data)
        elif data.get('type') == 'refresh':
            await self.send_staff_update()
    except json.JSONDecodeError:
        pass

async def handle_authentication(self, data):
    cookies = data.get('cookies', '')
    if cookies:
        self.user = await self.authenticate_from_cookies(cookies)
    if not self.user and data.get('token'):
        self.user = await
self.authenticate_from_token(data.get('token'))

    if self.user and self.user.is_authenticated:
        await self.authenticated_connect()
    else:
        await self.send(text_data=json.dumps({
            'type': 'error',
            'message': 'Authentication failed'
        }))
        await self.close()

@database_sync_to_async
def authenticate_from_cookies(self, cookie_string):
    try:
        cookies = dict(item.strip().split('=', 1) for item in
cookie_string.split(';') if '=' in item)

```

```

        access_token = cookies.get('access_token')
        if not access_token:
            return None
        token = AccessToken(access_token)
        user_id = token.payload.get('user_id')
        if user_id:
            user = User.objects.get(id=user_id)
            if user.is_active:
                return user
    except Exception:
        return None
    return None

@database_sync_to_async
def authenticate_from_token(self, token_string):
    try:
        token = AccessToken(token_string)
        user_id = token.payload.get('user_id')
        if user_id:
            user = User.objects.get(id=user_id)
            if user.is_active:
                return user
    except Exception:
        return None
    return None

@database_sync_to_async
def check_service_access(self):
    if not self.user:
        return False
    if self.user.is_superuser:
        return True
    if hasattr(self.user, 'staff_profile'):
        profile = self.user.staff_profile
        if profile.assigned_service and profile.assigned_service.id ==
int(self.service_id):
            return True
    return False

    async def authenticated_connect(self):

```

```

has_access = await self.check_service_access()
if not has_access:
    await self.send(text_data=json.dumps({
        'type': 'error',
        'message': 'You do not have access to this service'
    }))
    await self.close()
    return

self.group_name = f'service_{self.service_id}'
await self.channel_layer.group_add(self.group_name,
self.channel_name)

await self.send(text_data=json.dumps({
    'type': 'connected',
    'message': f'Connected to service {self.service_id} as
{self.user.username}',
    'user': {
        'id': self.user.id,
        'username': self.user.username,
        'is_staff': self.user.is_staff,
        'is_superuser': self.user.is_superuser
    }
}))

await self.send_staff_update()

async def disconnect(self, close_code):
    if hasattr(self, 'group_name'):
        await self.channel_layer.group_discard(self.group_name,
self.channel_name)

async def send_staff_update(self):
    data = await self.get_staff_dashboard_data()
    await self.send(text_data=json.dumps({
        'type': 'staff_update',
        'data': data
    }))

@database_sync_to_async

```

```

def get_staff_dashboard_data(self):
    try:
        service = Service.objects.get(id=self.service_id)
        today = timezone.now().date()

        waiting = Ticket.objects.filter(service=service,
ticket_date=today, status='waiting').order_by('queue_number')
        serving = Ticket.objects.filter(service=service,
ticket_date=today, status='serving').select_related('assigned_window')

        windows_status = []
        for window in service.windows.order_by('window_number'):
            window_serving =
serving.filter(assigned_window=window).first()
            windows_status.append({
                'id': window.id,
                'name': window.name,
                'number': window.window_number,
                'currently_serving': window_serving.display_number if
window_serving else None,
                'status': window.status,
                'is_available': window.is_available,
                'is_in_use': window.is_in_use,
                'claimed_by': window.current_staff.username if
window.current_staff else None,
            })

        return {
            'service': {
                'id': service.id,
                'name': service.name,
                'prefix': service.prefix
            },
            'waiting_count': waiting.count(),
            'serving_count': serving.count(),
            'next_ticket': waiting.first().display_number if
waiting.exists() else None,
            'waiting_list': TicketSerializer(waiting[:10],
many=True).data,
            'serving_list': TicketSerializer(serving, many=True).data,

```

```

        'windows': windows_status,
        'timestamp': timezone.now().isoformat()
    }
except Service.DoesNotExist:
    return {'error': 'Service not found'}

async def queue_update(self, event):
    await self.send_staff_update()

class TicketStatusConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.ticket_id = self.scope['url_route']['kwargs']['ticket_id']
        self.group_name = f'ticket_{self.ticket_id}'
        await self.channel_layer.group_add(self.group_name,
self.channel_name)
        await self.accept()
        await self.send_ticket_status()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(self.group_name,
self.channel_name)

    async def ticket_update(self, event):
        await self.send_ticket_status()

    @database_sync_to_async
    def get_ticket_data(self):
        try:
            ticket = Ticket.objects.get(ticket_id=self.ticket_id)
            data = TicketSerializer(ticket).data
            data['queue_info'] = {
                'position': ticket.people_ahead + 1,
                'total_ahead': ticket.people_ahead,
                'estimated_wait_minutes': ticket.wait_time_minutes,
                'total_in_queue':
Ticket.objects.filter(service=ticket.service,
ticket_date=ticket.ticket_date, status__in=['waiting',
'notified']).count()
            }
            return data

```

```

        except Ticket.DoesNotExist:
            return None

    async def send_ticket_status(self):
        data = await self.get_ticket_data()
        if data:
            await self.send(text_data=json.dumps({
                'type': 'ticket_update',
                'data': data
            }))

class WindowConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.window_id =
int(self.scope['url_route']['kwargs']['window_id'])
        info = await self.get_window_info()
        if not info:
            await self.close(code=4404)
            return

        self.service_id = info['service_id']

        await self.accept()

    async def disconnect(self, close_code):
        return

    async def receive(self, text_data):
        try:
            data = json.loads(text_data)
        except json.JSONDecodeError:
            await self.send(text_data=json.dumps({
                'type': 'error',
                'message': 'Invalid JSON payload.'
            }))
            return

        if data.get('type') not in ['reconnect', 'refresh']:
            return

```

```

        await self.send(text_data=json.dumps({
            'type': 'reconnect_ack',
            'window_id': self.window_id,
            'status': 'ok',
        }))

    @database_sync_to_async
    def get_window_info(self):
        window =
ServiceWindow.objects.filter(id=self.window_id).select_related('service').
first()
        if not window:
            return None
        return {'service_id': window.service_id}

class WindowStatusConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.service_id = self.scope['url_route']['kwargs']['service_id']
        self.group_name = f'windows_{self.service_id}'
        await self.channel_layer.group_add(self.group_name,
self.channel_name)
        await self.accept()
        await self.send_window_status()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(self.group_name,
self.channel_name)

    async def receive(self, text_data):
        try:
            data = json.loads(text_data)
        except json.JSONDecodeError:
            return

        if data.get('type') == 'refresh':
            await self.send_window_status()

    async def window_status_update(self, event):

```

```

        await self.send_window_status()

    @database_sync_to_async
    def get_windows_payload(self):
        windows =
ServiceWindow.objects.filter(service_id=self.service_id).select_related('c
urrent_staff').order_by('window_number')
        return {
            'service_id': int(self.service_id),
            'windows': [
                {
                    'id': window.id,
                    'name': window.name,
                    'number': window.window_number,
                    'status': window.status,
                    'is_in_use': window.is_in_use,
                    'is_available': window.is_available,
                    'claimed_by': window.current_staff.username if
window.current_staff else None,
                }
                for window in windows
            ],
        }

    async def send_window_status(self):
        payload = await self.get_windows_payload()
        await self.send(text_data=json.dumps({'type':
'window_status_update', 'data': payload}))

class ServiceStatusConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.service_id = self.scope['url_route']['kwargs']['service_id']
        self.group_name = f'service_status_{self.service_id}'

        await self.channel_layer.group_add(
            self.group_name,
            self.channel_name
        )
        await self.accept()

```

```

    # Send initial status
    await self.send_service_status()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(
            self.group_name,
            self.channel_name
        )

    async def service_status_update(self, event):
        #Called when service status changes
        await self.send_service_status()

    @database_sync_to_async
    def get_service_status(self):
        try:
            service = Service.objects.get(id=self.service_id)
            return {
                'service_id': service.id,
                'service_name': service.name,
                'is_active': service.is_active,
                'active_windows_count':
service.windows.filter(status='active').count(),
                'total_windows': service.windows.count()
            }
        except Service.DoesNotExist:
            return None

    async def send_service_status(self):
        status = await self.get_service_status()
        if status:
            await self.send(text_data=json.dumps({'type':
'service_status', 'data': status}))

```

Permissions

```

from rest_framework.permissions import BasePermission

```

```

class IsServiceStaff(BasePermission):
    # Permission check for staff assigned to a specific service
    def has_permission(self, request, view):
        # Allow admin users
        if request.user.is_superuser:
            return True

        # Check if user is staff
        if not request.user.is_staff:
            return False

        # Check if user has staff profile
        if not hasattr(request.user, 'staff_profile'):
            return False

        # Staff must be assigned to a service
        return request.user.staff_profile.assigned_service is not None

class HasServicePermission(BasePermission):
    # Permission check for specific service access

    def has_object_permission(self, request, view, obj):
        # Allow admin users
        if request.user.is_superuser:
            return True

        # Get staff's assigned service
        staff_profile = getattr(request.user, 'staff_profile', None)
        if not staff_profile or not staff_profile.assigned_service:
            return False

        # Check if object belongs to staff's assigned service
        if hasattr(obj, 'service'):
            return obj.service == staff_profile.assigned_service
        elif hasattr(obj, 'assigned_service'):
            return obj.assigned_service == staff_profile.assigned_service

        return False

```

SMS

```
import requests
import json
from django.conf import settings
import logging

logger = logging.getLogger(__name__)

class PhilSMSService:
    def __init__(self):
        self.api_url = settings.PHILSMS_API_URL
        self.token = settings.PHILSMS_API_TOKEN
        self.sender_id = settings.PHILSMS_SENDER_ID
        self.message_type = settings.PHILSMS_MESSAGE_TYPE

    def send_sms(self, phone_number, message):
        # Clean phone number to format 63XXXXXXXXXX
        phone = self._clean_phone(phone_number)

        headers = {
            'Authorization': f'Bearer {self.token}',
            'Content-Type': 'application/json',
            'Accept': 'application/json',
        }

        payload = {
            'recipient': phone,
            'sender_id': self.sender_id,
            'type': self.message_type,
            'message': message,
        }

        try:
            response = requests.post(
                self.api_url,
                headers=headers,
                json=payload,
```

```

        timeout=10
    )
    response.raise_for_status()
    result = response.json()

    if result.get('status') == 'success':
        logger.info(f"SMS sent to {phone}: {message[:30]}...")
        return True, result
    else:
        error_msg = result.get('message', 'Unknown error')
        logger.error(f"SMS API error: {error_msg}")
        return False, error_msg

except requests.exceptions.RequestException as e:
    logger.error(f"SMS request failed: {e}")
    return False, str(e)

def _clean_phone(self, phone):
    """Convert Philippine mobile number to 63 format."""
    # Remove all non-digits
    digits = ''.join(filter(str.isdigit, phone))
    # If it starts with 0, replace with 63
    if digits.startswith('0'):
        digits = '63' + digits[1:]
    # If it's 10 digits (e.g., 9123456789), add 63
    elif len(digits) == 10:
        digits = '63' + digits
    # If it already starts with 63, keep as is
    return digits

```

```

from .models import Ticket, SMSSettings
from .sms_service import PhilSMSService
from django.utils import timezone
import logging

logger = logging.getLogger(__name__)

def check_and_send_sms(service_id, threshold=None):
    """

```

Check all queued tickets for a service and send SMS based on settings.

NEW LOGIC: Send SMS when ticket is within 'threshold' positions from being served.

Example with threshold=10:

- Ticket #15 will get SMS when current serving reaches #5 (15-5=10)
- Ticket #20 will get SMS when current serving reaches #10 (20-10=10)

"""

```
from .models import Service
```

```
try:
```

```
    service = Service.objects.get(id=service_id)
```

```
except Service.DoesNotExist:
```

```
    logger.warning(f"Service {service_id} not found while checking SMS")
```

```
    return
```

```
settings = SMSSettings.get_service_settings(service)
```

```
if not settings.sms_enabled:
```

```
    logger.info(f"SMS disabled for service {service_id}, skipping notifications")
```

```
    return
```

```
# Use provided threshold or get from settings
```

```
active_threshold = threshold if threshold is not None else settings.notification_threshold
```

```
logger.info(f"[SMS DEBUG] Service: {service.name}, Threshold: {active_threshold}")
```

```
sms = PhilSMSService()
```

```
today = timezone.now().date()
```

```
# Get current serving tickets for this service
```

```
current_serving_tickets = Ticket.objects.filter(  
    service_id=service_id,  
    ticket_date=today,  
    status='serving'
```

```

).order_by('queue_number')

if not current_serving_tickets.exists():
    logger.info(f"No tickets currently being served for service
{service_id}")
    return

# Get the highest queue number being served (the furthest along)
current_serving_number = current_serving_tickets.last().queue_number
logger.info(f"[SMS DEBUG] Current serving ticket numbers:
[{t.queue_number for t in current_serving_tickets}]")
logger.info(f"[SMS DEBUG] Highest serving number:
{current_serving_number}")

# Get all waiting tickets that haven't been notified yet
tickets = Ticket.objects.filter(
    service_id=service_id,
    ticket_date=today,
    status__in=['waiting', 'notified'],
    sms_phone__isnull=False,
    sms_sent=False
).select_related('service').order_by('queue_number')

for ticket in tickets:
    # Calculate how many positions away from being served
    positions_away = ticket.queue_number - current_serving_number

    logger.info(
        f"[SMS DEBUG] Ticket #{ticket.queue_number}
({ticket.display_number}), "
        f"Current serving: #{current_serving_number}, "
        f"Positions away: {positions_away}, "
        f"Threshold: {active_threshold}"
    )

# Send SMS if ticket is within threshold positions from being
served
if 0 < positions_away <= active_threshold:
    # Only send if not already sent
    if not ticket.sms_sent:

```

```

        message = (
            f"Your ticket {ticket.display_number} is now
#{positions_away} in line "
            f"at {ticket.service.name}. Current serving:
#{current_serving_number}. "
            f"Please prepare to proceed to the window."
        )

        success, response = sms.send_sms(ticket.sms_phone,
message)

        if success:
            ticket.sms_sent = True
            ticket.sms_sent_at = timezone.now()
            ticket.save(update_fields=['sms_sent', 'sms_sent_at'])

            logger.info(
                f"SMS sent for ticket {ticket.display_number} "
                f"(positions away: {positions_away})"
            )
        else:
            logger.warning(
                f"Failed to send SMS for ticket
{ticket.display_number}. "
                f"Response: {response}"
            )
        elif positions_away <= 0:
            logger.info(f"Ticket {ticket.display_number} has already been
served or passed")

```

Routing

```
from django.urls import path, include
from . import views, staff_views, window_views, auth_views, service_views,
analytics_views, session_views

app_name = 'queueing'

urlpatterns = [
    # Auth endpoints
    path('auth/', include('queueing.auth_urls')),

    # Public endpoints
    path('services/public/', views.public_service_list,
name='public-service-list'),
    path('tickets/generate/', views.generate_ticket,
name='generate-ticket'),
    path('tickets/<uuid:ticket_id>/status/', views.ticket_status,
name='ticket-status'),
    path('dashboard/status/', views.dashboard_status,
name='dashboard-status'),

    # Service management (Admin only)
    path('services/', service_views.service_list, name='service-list'),
    path('services/create/', service_views.create_service,
name='create-service'),
    path('services/<int:service_id>/update/',
service_views.update_service, name='update-service'),
    path('services/<int:service_id>/delete/',
service_views.delete_service, name='delete-service'),
    path('services/<int:service_id>/stats/', service_views.service_stats,
name='service-stats'),

    # Service Window management
    path('services/<int:service_id>/windows/',
window_views.service_windows_list, name='service-windows-list'),
    path('services/<int:service_id>/windows/create/',
window_views.create_service_window, name='create-service-window'),
    path('windows/<int>window_id>/update/',
window_views.update_service_window, name='update-service-window'),
```

```
    path('windows/<int:window_id>/delete/',
window_views.delete_service_window, name='delete-service-window'),

    # Staff queue management
    path('staff/dashboard/', staff_views.staff_dashboard,
name='staff-dashboard'),
    path('staff/call-next/', staff_views.call_next_ticket,
name='call-next-ticket'),
    path('staff/call-specific/', staff_views.call_specific_ticket,
name='call-specific-ticket'),
    path('staff/toggle-queue/', staff_views.toggle_queue_status,
name='toggle-queue'),
    path('staff/tickets/<uuid:ticket_id>/start/',
staff_views.start_serving, name='start-serving'),
    path('staff/tickets/<uuid:ticket_id>/complete/',
staff_views.complete_serving, name='complete-serving'),
    path('staff/tickets/<uuid:ticket_id>/remove/',
staff_views.remove_ticket, name='remove-ticket'),
    path('staff/tickets/<uuid:ticket_id>/recall/',
staff_views.recall_ticket, name='recall-ticket'),
    path('staff/tickets/<uuid:ticket_id>/skip/', staff_views.skip_ticket,
name='skip-ticket'),

    # Active session management
    path('sessions/claim', session_views.claim_session,
name='claim-session'),
    path('sessions/release', session_views.release_session,
name='release-session'),

    # Analytics endpoints
    path('admin/analytics/', analytics_views.admin_analytics,
name='admin-analytics'),
    path('admin/analytics/service/<int:service_id>/',
analytics_views.service_analytics, name='service-analytics'),

    # SMS endpoints
    path('tickets/<uuid:ticket_id>/sms-opt-in/', views.sms_opt_in,
name='sms-opt-in'),

    # SMS Settings (Admin only)
```

```

    path('admin/sms-settings/', views.get_sms_settings,
name='get-sms-settings'),
    path('admin/sms-settings/global/', views.update_global_sms_settings,
name='update-global-sms'),
    path('admin/sms-settings/service/<int:service_id>/',
views.update_service_sms_settings, name='update-service-sms'),
    path('admin/sms-settings/service/<int:service_id>/reset/',
views.reset_service_sms_settings, name='reset-service-sms'),

    # System Settings
    path('admin/system-settings/', views.get_system_settings,
name='get-system-settings'),
    path('admin/system-settings/update/', views.update_system_settings,
name='update-system-settings'),

    path('admin/analytics/window/<int:service_id>/',
analytics_views.window_analytics, name='window-analytics'),
    path('admin/analytics/export/csv/',
analytics_views.export_analytics_csv, name='export-analytics-csv'),
    path('admin/analytics/export/window/<int:service_id>/csv/',
analytics_views.export_window_performance_csv,
name='export-window-performance-csv'),
]

```

```

from django.urls import re_path, path
from . import consumers

websocket_urlpatterns = [
    re_path(r'^ws/dashboard/$', consumers.DashboardConsumer.as_asgi()),
    re_path(r'^ws/staff/(?P<service_id>\d+)/$',
consumers.StaffDashboardConsumer.as_asgi()),
    re_path(r'^ws/service/(?P<service_id>\d+)/windows/$',
consumers.WindowStatusConsumer.as_asgi()),
    re_path(r'^ws/ticket/(?P<ticket_id>[0-9a-f-]+)/$',
consumers.TicketStatusConsumer.as_asgi()),
    re_path(r'^ws/window/(?P<window_id>\d+)/$',
consumers.WindowConsumer.as_asgi()),
    re_path(r'^ws/service/(?P<service_id>\d+)/status/$',
consumers.ServiceStatusConsumer.as_asgi()),
]

```

```
]
```

```
from django.urls import path
from . import auth_views

app_name = 'auth'

urlpatterns = [
    # Public auth endpoints
    path('login/', auth_views.login_view, name='login'),
    path('logout/', auth_views.logout_view, name='logout'),
    path('refresh/', auth_views.refresh_token_view, name='refresh-token'),

    # Authenticated user endpoints
    path('me/', auth_views.current_user_view, name='current-user'),
    path('change-password/', auth_views.change_password_view,
name='change-password'),

    # Admin management endpoints
    path('admin/staff/create/', auth_views.create_staff_view,
name='create-staff'),
    path('admin/staff/list/', auth_views.list_staff_view,
name='list-staff'),
    path('admin/staff/<int:user_id>/delete/',
auth_views.delete_staff_view, name='delete-staff'),
    path('admin/staff/<int:user_id>/update/',
auth_views.update_staff_view, name='update-staff'),
    path('admin/create/', auth_views.create_admin_view,
name='create-admin'),
    path('admin/list/', auth_views.list_admins_view, name='list-admins'),
    path('admin/<int:user_id>/update/', auth_views.update_admin_view,
name='update-admin'),
    path('admin/<int:user_id>/delete/', auth_views.delete_admin_view,
name='delete-admin'),
    path("admin/staff/<int:user_id>/reset-password/",
auth_views.admin_reset_staff_password_view, name="reset-staff-password"),
]
```

Websocket

```
from channels.layers import get_channel_layer
from asgiref.sync import async_to_sync
from .models import Ticket
from django.utils import timezone
from time import time

_last_update_time = {}

def send_ticket_update(ticket_id):
    """Send update to a specific ticket"""
    ticket_id_str = str(ticket_id)
    group_name = f'ticket_{ticket_id_str}'

    channel_layer = get_channel_layer()
    try:
        async_to_sync(channel_layer.group_send) (
            group_name,
            {
                'type': 'ticket_update',
                'ticket_id': ticket_id_str
            }
        )
    except Exception as e:
        import traceback
        traceback.print_exc()

    return True

def send_dashboard_update():
    """Send update to public dashboard"""
    channel_layer = get_channel_layer()
    if channel_layer is None:
        return

    try:
        async_to_sync(channel_layer.group_send) (
            'public_dashboard',
            {'type': 'dashboard_update'}
        )
    
```

```

except Exception as e:
    import traceback
    traceback.print_exc()

def send_service_update(service_id):
    """Send update to all staff dashboards for a specific service"""
    channel_layer = get_channel_layer()
    try:
        async_to_sync(channel_layer.group_send)(
            f'service_{service_id}',
            {'type': 'queue_update'}
        )
    except Exception as e:
        import traceback
        traceback.print_exc()

def send_windows_update(service_id):
    """Send realtime window status updates for selection screens."""
    channel_layer = get_channel_layer()
    try:
        async_to_sync(channel_layer.group_send)(
            f'windows_{service_id}',
            {'type': 'window_status_update'}
        )
    except Exception:
        import traceback
        traceback.print_exc()

def send_queue_position_updates(service_id, current_ticket_id=None):
    """Send updates to all waiting tickets in a service"""
    today = timezone.now().date()
    waiting_tickets = Ticket.objects.filter(
        service_id=service_id,
        ticket_date=today,
        status__in=['waiting', 'notified']
    )

    for ticket in waiting_tickets:
        send_ticket_update(str(ticket.ticket_id))

```

```

    if current_ticket_id:
        send_ticket_update(current_ticket_id)

def debounced_send_queue_updates(service_id, current_ticket_id=None,
delay=0.5):
    """Prevent multiple rapid updates"""
    global _last_update_time
    key = f"queue_{service_id}"
    now = time()

    if key in _last_update_time and now - _last_update_time[key] < delay:
        return

    _last_update_time[key] = now
    send_queue_position_updates(service_id, current_ticket_id)

def send_service_status_update(service_id, is_active=None):
    """Send service status update to all subscribers"""
    channel_layer = get_channel_layer()
    async_to_sync(channel_layer.group_send)(
        f'service_status_{service_id}',
        {
            'type': 'service_status_update',
            'service_id': service_id,
            'is_active': is_active
        }
    )

    # Also update dashboards
    send_dashboard_update()
    send_service_update(service_id)

```

Websocket Tester(HTML)

```
<!DOCTYPE html>
<html>
<head>
  <title>Queueick WebSocket Tester</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 20px; background:
#f5f5f5; }
    .container { display: flex; gap: 20px; flex-wrap: wrap; }
    .card {
      background: white;
      border-radius: 8px;
      padding: 15px;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
      flex: 1;
      min-width: 300px;
    }
    h3 { margin-top: 0; color: #333; border-bottom: 2px solid #eee;
padding-bottom: 10px; }
    .status {
      padding: 8px;
      border-radius: 4px;
      margin: 10px 0;
      font-weight: bold;
    }
    .connected { background: #d4edda; color: #155724; border: 1px
solid #c3e6cb; }
    .disconnected { background: #f8d7da; color: #721c24; border: 1px
solid #f5c6cb; }
    .messages {
      height: 200px;
      overflow-y: auto;
      border: 1px solid #ddd;
      padding: 10px;
      background: #fafafa;
      font-family: monospace;
      font-size: 12px;
    }
    .msg { border-bottom: 1px solid #eee; padding: 5px; }
    .timestamp { color: #999; font-size: 10px; }
```

```

    button {
        background: #007bff;
        color: white;
        border: none;
        padding: 8px 15px;
        border-radius: 4px;
        cursor: pointer;
        margin-right: 5px;
    }
    button:hover { background: #0056b3; }
    button.danger { background: #dc3545; }
    button.danger:hover { background: #bd2130; }
    button.success { background: #28a745; }
    button.success:hover { background: #218838; }
    button:disabled {
        opacity: 0.5;
        cursor: not-allowed;
    }
    input { padding: 8px; border: 1px solid #ddd; border-radius: 4px;
width: 200px; margin: 5px 0; }
    .login-box {
        background: white;
        padding: 20px;
        margin-bottom: 20px;
        border-radius: 8px;
        box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }
    .login-status {
        margin-left: 10px;
        font-weight: bold;
    }
</style>
</head>
<body>
    <h1>Queueick WebSocket Tester</h1>
    <p>Test all WebSocket endpoints in real-time</p>

    <!-- Login Section -->
    <div class="login-box">
        <h3>Authentication</h3>

```

```

        <div>
            <input type="text" id="login-username" value="cashier_staff"
placeholder="Username">
            <input type="password" id="login-password" value="staff123"
placeholder="Password">
            <button class="success" onclick="login()">Login</button>
            <button onclick="logout()" class="danger">Logout</button>
            <span id="login-status" class="login-status">Not logged
in</span>
        </div>
        <p style="font-size: 12px; color: #666; margin-top: 10px;">
            Warning: Staff dashboard requires login. Public dashboard and
ticket status do not.
        </p>
    </div>

    <div class="container">
        <!-- Dashboard Card -->
        <div class="card">
            <h3>Public Dashboard</h3>
            <div id="dashboard-status" class="status
disconnected">Disconnected</div>
            <div>
                <button onclick="connectDashboard()">Connect</button>
                <button class="danger"
onclick="disconnectDashboard()">Disconnect</button>
            </div>
            <div class="messages" id="dashboard-messages"></div>
        </div>

        <!-- Ticket Status Card -->
        <div class="card">
            <h3>Ticket Status</h3>
            <div>
                <label>Ticket ID:</label>
                <input type="text" id="ticket-id" placeholder="uuid-here">
                <button onclick="getSampleTicketId()" style="font-size:
11px; padding: 4px 8px;">Get Sample</button>
            </div>

```

```

        <div id="ticket-status" class="status
disconnected">Disconnected</div>
        <div>
            <button onclick="connectTicket()">Connect</button>
            <button class="danger"
onclick="disconnectTicket()">Disconnect</button>
        </div>
        <div class="messages" id="ticket-messages"></div>
    </div>
</div>

<!-- Staff Dashboard Card -->
<div class="card">
    <h3>Staff Dashboard</h3>
    <div>
        <label>Service ID:</label>
        <input type="number" id="service-id" value="1" min="1">
    </div>
    <div id="staff-status" class="status
disconnected">Disconnected</div>
    <div>
        <button onclick="connectStaff()">Connect</button>
        <button class="danger"
onclick="disconnectStaff()">Disconnect</button>
    </div>
    <div class="messages" id="staff-messages"></div>
</div>

<div style="margin-top: 20px; text-align: center;">
    <button onclick="connectAll()">Connect All</button>
    <button class="danger" onclick="disconnectAll()">Disconnect
All</button>
    <button onclick="clearAll()">Clear Messages</button>
    <button onclick="generateTestTicket(this)" class="success"
id="generate-btn">Generate Test Ticket</button>
</div>

<script>
    // WebSocket instances
    let dashboardWs = null;

```

```

let staffWs = null;
let ticketWs = null;

// Login status
let isLoggedIn = false;

// Flags to prevent multiple operations
let isGenerating = false;
let isLoggingIn = false;
let lastClickTime = 0;
const DEBOUNCE_TIME = 1000; // 1 second

// Helper to add message to a messages div
function addMessage(elementId, message, isJson = false) {
    const msgs = document.getElementById(elementId);
    const msgDiv = document.createElement('div');
    msgDiv.className = 'msg';

    const time = new Date().toLocaleTimeString();

    if (isJson) {
        try {
            const formatted = JSON.stringify(JSON.parse(message),
null, 2);

            msgDiv.innerHTML = `<span
class="timestamp">${time}</span> [in] <pre style="margin:5px
0">${formatted}</pre>`;
        } catch (e) {
            msgDiv.innerHTML = `<span
class="timestamp">${time}</span> [in] ${message}`;
        }
    } else {
        msgDiv.innerHTML = `<span class="timestamp">${time}</span>
${message}`;
    }

    msgs.appendChild(msgDiv);
    msgs.scrollTop = msgs.scrollHeight;
}

```

```

// Debounce function to prevent double-clicks
function checkDebounce() {
    const now = Date.now();
    if (now - lastClickTime < DEBOUNCE_TIME) {
        console.log('Warning: Click debounced - too fast!');
        return false;
    }
    lastClickTime = now;
    return true;
}

// Login function
async function login() {
    const username =
document.getElementById('login-username').value;
    const password =
document.getElementById('login-password').value;

    try {
        const response = await
fetch('http://localhost:8000/api/auth/login/', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({username, password}),
            credentials: 'include'
        });

        const data = await response.json();

        if (data.success) {
            isLoggedIn = true;
            document.getElementById('login-status').innerHTML =
'Logged in as ' + username;

            // Check cookies after login
            console.log('Cookies after login:', document.cookie);

            if (document.cookie) {
                console.log('Cookies set successfully!');
            } else {

```

```

        console.log('Cookies still empty!');
    }
    } else {
        document.getElementById('login-status').innerHTML =
'Login failed: ' + (data.error || 'Unknown error');
    }
    } catch (error) {
        console.error('Login error:', error);
        document.getElementById('login-status').innerHTML = 'Login
error: ' + error.message;
    }
}

// Logout function
async function logout() {
    if (!checkDebounce()) return false;

    try {
        await fetch('http://localhost:8000/api/auth/logout/', {
            method: 'POST',
            credentials: 'include'
        });

        isLoggedIn = false;
        document.getElementById('login-status').innerHTML =
'Logged out';
        document.getElementById('login-status').style.color =
'black';

        // Disconnect staff if connected
        disconnectStaff();

        addMessage('staff-messages', 'Logged out');
    } catch (error) {
        console.error('Logout error:', error);
    }

    return false;
}

```

```

// Generate a test ticket - FIXED to prevent multiple generations
async function generateTestTicket(button) {
  // Debounce check
  if (!checkDebounce()) return false;

  // Prevent multiple simultaneous generations
  if (isGenerating) {
    addMessage('ticket-messages', 'Already generating a
ticket, please wait...');
    return false;
  }

  // Disable the button
  if (button) {
    button.disabled = true;
    button.innerText = 'Generating...';
  }

  isGenerating = true;

  try {
    const response = await
fetch('http://localhost:8000/api/tickets/generate/', {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({service_id: 1}),
      credentials: 'include'
    });

    const data = await response.json();
    if (data.success && data.ticket) {
      const ticketId = data.ticket.ticket_id;
      document.getElementById('ticket-id').value = ticketId;
      addMessage('ticket-messages', 'Generated ticket: ' +
data.ticket.display_number);
      addMessage('ticket-messages', 'Ticket ID: ' +
ticketId);

      console.log('Ticket generated once:',
data.ticket.display_number);
    } else {

```

```

        addMessage('ticket-messages', 'Failed to generate
ticket');
    }
} catch (error) {
    console.error('Generate ticket error:', error);
    addMessage('ticket-messages', 'Error: ' + error);
} finally {
    // Re-enable the button after a delay
    setTimeout(() => {
        isGenerating = false;
        if (button) {
            button.disabled = false;
            button.innerText = 'Generate Test Ticket';
        }
    }, 500);
}

return false;
}

// Get sample ticket ID from URL or generate one - FIXED
let isGettingSample = false;

async function getSampleTicketId() {
    if (!checkDebounce()) return false;
    if (isGettingSample) return false;

    isGettingSample = true;

    // Try to get from URL parameter
    const urlParams = new URLSearchParams(window.location.search);
    const ticketId = urlParams.get('ticket');
    if (ticketId) {
        document.getElementById('ticket-id').value = ticketId;
        addMessage('ticket-messages', 'Loaded ticket ID from URL:
' + ticketId);
    } else {
        // Get the generate button element and pass it
        const generateBtn =
document.getElementById('generate-btn');

```

```

        await generateTestTicket(generateBtn);
    }

    setTimeout(() => {
        isGettingSample = false;
    }, 1000);

    return false;
}

// Dashboard WebSocket
function connectDashboard() {
    if (dashboardWs && dashboardWs.readyState === WebSocket.OPEN)
    {
        addMessage('dashboard-messages', 'Warning: Dashboard
already connected');
        return false;
    }

    if (dashboardWs) dashboardWs.close();

    dashboardWs = new
WebSocket('ws://localhost:8000/ws/dashboard/');

    dashboardWs.onopen = function() {
        document.getElementById('dashboard-status').className =
'status connected';
        document.getElementById('dashboard-status').innerHTML =
'Connected';
        addMessage('dashboard-messages', 'Connected to public
dashboard');
    };

    dashboardWs.onmessage = function(event) {
        addMessage('dashboard-messages', event.data, true);
    };

    dashboardWs.onerror = function(error) {
        addMessage('dashboard-messages', 'Error: ' +
JSON.stringify(error));
    };
}

```

```

    };

    dashboardWs.onclose = function() {
        document.getElementById('dashboard-status').className =
'status disconnected';
        document.getElementById('dashboard-status').innerHTML =
'Disconnected';
        addMessage('dashboard-messages', 'Disconnected');
        dashboardWs = null;
    };

    return false;
}

function disconnectDashboard() {
    if (dashboardWs) dashboardWs.close();
    return false;
}

// Staff WebSocket
function connectStaff() {
    if (staffWs && staffWs.readyState === WebSocket.OPEN) {
        addMessage('staff-messages', 'Warning: Staff dashboard
already connected');
        return false;
    }

    if (staffWs) staffWs.close();

    const serviceId = document.getElementById('service-id').value;

    // Get cookies from document
    const cookies = document.cookie;
    console.log('Current cookies:', cookies); // Debug: see what
cookies exist

    // Create WebSocket with cookies in header
    staffWs = new
WebSocket(`ws://localhost:8000/ws/staff/${serviceId}/`);

```

```

// Manually set cookie header when connection opens
staffWs.onopen = function() {
    // Send cookies through WebSocket as first message
    staffWs.send(JSON.stringify({
        type: 'authenticate',
        cookies: document.cookie
    }));

    document.getElementById('staff-status').className =
'status connected';
    document.getElementById('staff-status').innerHTML =
'Connected';
    addMessage('staff-messages', 'Connected to staff dashboard
(service ' + serviceId + ')');
};

staffWs.onmessage = function(event) {
    addMessage('staff-messages', event.data, true);
};

staffWs.onerror = function(error) {
    console.error('WebSocket error:', error);
    addMessage('staff-messages', 'Error: ' +
JSON.stringify(error));
};

staffWs.onclose = function(event) {
    document.getElementById('staff-status').className =
'status disconnected';
    document.getElementById('staff-status').innerHTML =
'Disconnected';

    if (event.code === 1006) {
        addMessage('staff-messages', 'Authentication failed -
Cookies not sent');
    } else {
        addMessage('staff-messages', 'Disconnected');
    }

    staffWs = null;

```

```

    };

    return false;
}

function disconnectStaff() {
    if (staffWs) staffWs.close();
    return false;
}

// Ticket WebSocket
function connectTicket() {
    if (ticketWs && ticketWs.readyState === WebSocket.OPEN) {
        addMessage('ticket-messages', 'Warning: Ticket WebSocket
already connected');
        return false;
    }

    if (ticketWs) ticketWs.close();

    // TRIM the input to remove spaces!
    const ticketId =
document.getElementById('ticket-id').value.trim();

    if (!ticketId) {
        alert('Please enter a ticket ID');
        return false;
    }

    ticketWs = new
WebSocket(`ws://localhost:8000/ws/ticket/${ticketId}/`);

    ticketWs.onopen = function() {
        document.getElementById('ticket-status').className =
'status connected';
        document.getElementById('ticket-status').innerHTML =
'Connected';
        addMessage('ticket-messages', 'Connected to ticket ' +
ticketId);
    };

```

```

        ticketWs.onmessage = function(event) {
            addMessage('ticket-messages', event.data, true);
        };

        ticketWs.onerror = function(error) {
            addMessage('ticket-messages', 'Error: ' +
JSON.stringify(error));
        };

        ticketWs.onclose = function() {
            document.getElementById('ticket-status').className =
'status disconnected';
            document.getElementById('ticket-status').innerHTML =
'Disconnected';
            addMessage('ticket-messages', 'Disconnected');
            ticketWs = null;
        };

        return false;
    }

    function disconnectTicket() {
        if (ticketWs) ticketWs.close();
        return false;
    }

    // Utility functions
    function connectAll() {
        connectDashboard();
        if (isLoggedIn) {
            connectStaff();
        } else {
            addMessage('staff-messages', 'Login first to connect staff
dashboard');
        }
        connectTicket();
        return false;
    }

```

```
function disconnectAll() {
    disconnectDashboard();
    disconnectStaff();
    disconnectTicket();
    return false;
}

function clearAll() {
    document.getElementById('dashboard-messages').innerHTML = '';
    document.getElementById('staff-messages').innerHTML = '';
    document.getElementById('ticket-messages').innerHTML = '';
    return false;
}

// Override double-click on generate button
document.addEventListener('DOMContentLoaded', function() {
    const generateBtn = document.getElementById('generate-btn');
    if (generateBtn) {
        generateBtn.addEventListener('dblclick', function(e) {
            e.preventDefault();
            e.stopPropagation();
            console.log('Double-click prevented on generate
button');

            return false;
        });
    }
});

// Auto-connect to dashboard on page load
window.onload = function() {
    connectDashboard();
    // Check URL for ticket ID after a short delay
    setTimeout(() => {
        getSampleTicketId();
    }, 500);
};
</script>
</body>
</html>
```